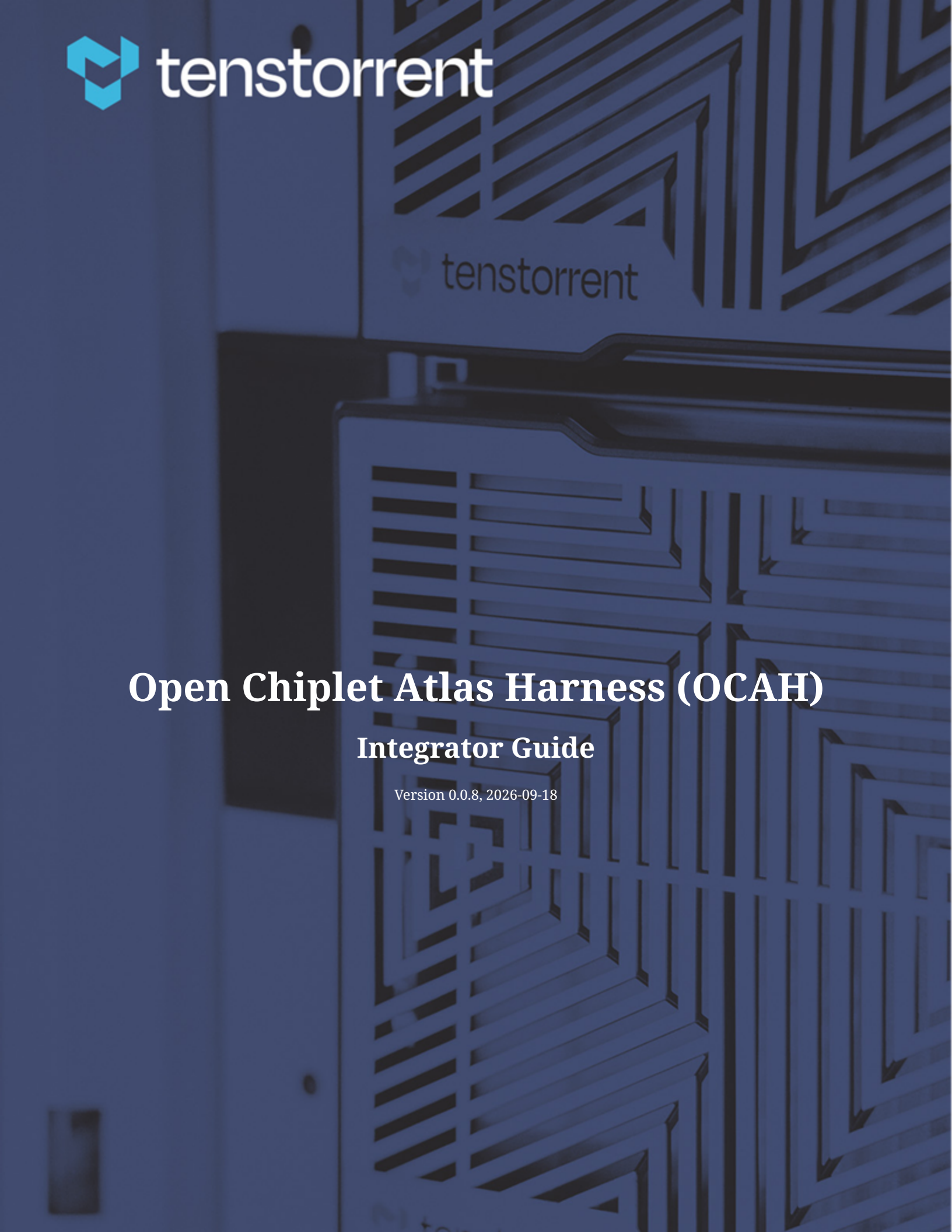


Open Chiplet Atlas Harness (OCAH)

Integrator Guide

Version 0.0.8, 2026-09-18

Table of Contents

1. About this Guide . . . 1
Initial SMU Integration . . . 2
2. Ports in `smu_wrapper` . . . 3
3. Macros needed by "integration modules" . . . 4
3.1. Macros needed by SEP . . . 4
3.2. Macros needed by SMC . . . 4
SoC Integration Checklist . . . 6
4. Integration gates . . . 7
SMU Top-Level Integration . . . 8
5. RTL Filelists . . . 9
6. Address Map Integration . . . 10
6.1. Register collateral . . . 10
7. Immutable ROM FW Integration . . . 12
7.1. Weak-symbol override method . . . 12
7.2. SMC ROM FW integration . . . 12
7.3. SEP ROM FW integration . . . 13
7.4. Key Manager ROM FW integration . . . 13
8. System Management Controller (SMC) Integration . . . 14
8.1. Clock and Reset . . . 14
8.2. External CPU cluster NDM reset . . . 14
8.3. Address Map and Aperture . . . 14
8.4. AXI Interface Configuration . . . 15
8.5. GPIO Pad Integration . . . 16
8.6. Peripheral Interface Connections . . . 16
8.6.1. Adopter Peripheral Window . . . 17
8.6.2. eFuse Interface . . . 17
8.7. Memory Subsystem Integration . . . 22
8.7.1. SRAM and Cache Array Integration . . . 22
8.8. Memory Repair Integration . . . 22
8.8.1. Control Signals . . . 23
8.8.2. Repair Data . . . 23
8.8.3. Status Monitoring . . . 23
8.8.4. Bypass Configuration . . . 23
8.8.5. Integration Requirements . . . 23
8.9. Interrupt Integration . . . 24
8.10. Telemetry and Debug Integration . . . 24
8.11. Bring-Up Sequence . . . 25
8.12. SMC Port Interface . . . 25
8.13. GPIO Pin Requirements . . . 30
9. Debug & Test Ports (DTP) Integration . . . 33
9.1. Configuration . . . 33
9.2. Instantiation . . . 33

9.3. Clock and Reset . . . 34
9.4. Constraints . . . 34
9.5. STAP Scan Chain Topology . . . 34
9.6. DTP Port Interface . . . 35
10. Security Processor (SEP) Integration . . . 39
10.1. Module Overview . . . 39
10.2. Clock and Reset Requirements . . . 39
10.3. AXI Interfaces . . . 39
10.4. Memory Interfaces . . . 40
10.5. Cryptographic Subsystem Connections . . . 40
10.5.1. External TRNG . . . 40
10.6. eFuse and Lifecycle . . . 40
10.7. SPI Host Interface . . . 41
10.7.1. SEP SPI Host Pin Assignment . . . 41
10.8. Bring-Up Considerations . . . 42
10.9. SEP Port Interface . . . 42
11. SMU Composition and Internal Wiring . . . 47
11.1. Module Variants and IP Integration . . . 47
11.1.1. IP Integration Modules . . . 49
11.1.2. Wrapper Clock Architecture . . . 49
11.2. Clock and Reset Architecture . . . 50
11.3. SMU AXI Crossbar Address Map . . . 50
11.3.1. Firmware programming order . . . 50
11.4. SMU Default Parameters . . . 51
11.5. IC_RESET TDR Structure . . . 51
11.5.1. SMC slice (TDI to TDO) . . . 51
11.5.2. SEP slice (TDI to TDO) . . . 52
11.5.3. External slice (adopter-defined) . . . 52
11.6. DTP Parameter Pass-Through . . . 52
11.6.1. JTAG Configuration Parameters . . . 52
11.6.2. JTAG ID Parameters . . . 53
11.6.3. Cross Trigger Parameters . . . 53
11.6.4. Pipeline Depth Parameters . . . 53
11.6.5. Type Parameters . . . 53
11.7. DTP Port Mapping . . . 54
11.7.1. Clock and Reset . . . 54
11.7.2. Primary JTAG TAP Interface . . . 54
11.7.3. Boundary Scan Interface . . . 54
11.7.4. I/O STAP Interface . . . 55
11.7.5. SMC Debug STAP Interface . . . 55
11.7.6. SEP Debug STAP Interface . . . 55
11.7.7. Extra STAP Interfaces . . . 55
11.7.8. Extended STAP Scan Interface . . . 55
11.7.9. DFD iJTAG Interface . . . 56

11.7.10. DFT iJTAG Interfaces . . . 56
11.7.11. SMC Fabric Debug AXI Interface . . . 56
11.7.12. SMC OTP Debug AXI-Lite Interface . . . 56
11.7.13. SEP OTP Debug AXI-Lite Interface . . . 56
11.7.14. Clock Control . . . 56
11.7.15. Boot Stall Control . . . 57
11.7.16. JTAG Reset Control . . . 57
11.7.17. JTAG State Outputs . . . 57
11.7.18. Cross Trigger CSR AXI-Lite Interface . . . 57
11.7.19. Cross Trigger Matrix Interface . . . 57
11.7.20. Clock Stop Request Interface . . . 58
11.7.21. Cross Trigger Port GPIO Interface . . . 58
11.8. SMC Port Mapping . . . 58
11.8.1. Clocks . . . 58
11.8.2. Resets . . . 59
11.8.3. System AXI Input . . . 59
11.8.4. JTAG AXI Input . . . 59
11.8.5. SMC OTP JTAG AXI-Lite . . . 59
11.8.6. SEP AXI Input . . . 59
11.8.7. AXI Output . . . 59
11.8.8. DTP CSR Interface . . . 59
11.8.9. Adopter Peripheral Window AXI-Lite . . . 60
11.8.10. eFuse Interface . . . 60
11.8.11. GPIO Data Signals . . . 60
11.8.12. GPIO External Pins . . . 60
11.8.13. ATB Telemetry . . . 60
11.8.14. DED/WDT . . . 61
11.8.15. Region Size . . . 61
11.8.16. External Interrupts . . . 61
11.8.17. Fuse Signals . . . 61
11.8.18. DFT Signals . . . 61
11.8.19. Lifecycle State . . . 61
11.8.20. Mailbox Interrupts . . . 61
11.8.21. Reset Unit Signals . . . 62
11.8.22. CPU Memory Signals . . . 62
11.8.23. Memory Init . . . 62
11.8.24. System Timer OCTS Interface . . . 62
11.8.25. Reset Overrides (from DTP) . . . 62
11.8.26. DFD Signals . . . 63
11.8.27. Cross Trigger Interface . . . 63
11.8.28. Trace Memory . . . 63
11.8.29. External Debug Bus . . . 63
11.8.30. Test Mode . . . 63
11.9. SMC-DTP Internal Interconnection . . . 63

11.10. SEP-Conditional Wiring . . . 64
11.10.1. Other Internal Tie-offs . . . 64
11.11. SMU Port Interface . . . 64
System Management Network (SMN) Integration . . . 73
12. Status . . . 74
AXI-over-UCIe Bridge (AoU) Integration . . . 75
13. Integration guide . . . 76
References . . . 76
Integration Options . . . 76
Module Overview . . . 76
AOU_TOP . . . 76
AOU_CORE_TOP . . . 77
Parameter List . . . 79
FDI_CONFIG values . . . 81
Build flags . . . 81
Interface Signals . . . 81
Clock and Reset . . . 81
APB Slave Interface . . . 82
AXI Master (RX) – AR Channel . . . 82
AXI Master (RX) – R Channel . . . 82
AXI Master (RX) – AW Channel . . . 82
AXI Master (RX) – W and B Channels . . . 83
AXI Slave (TX) – AR Channel . . . 83
AXI Slave (TX) – R Channel . . . 83
AXI Slave (TX) – AW, W, B Channels . . . 84
PHY and FDI (Flit Data Interface) . . . 84
Interrupt and Error (to Error Handler) . . . 85
UCIe / External Control and Status . . . 86
DFT . . . 86
FDI Bringup Control Interface (AOU_TOP only) . . . 87
FDI Bringup Software Control (AOU_TOP only) . . . 88
FDI Bringup Status (AOU_TOP only) . . . 88
AOU_CORE_TOP-only Ports (not on AOU_TOP) . . . 89
Software operation Guide . . . 89
Register Map . . . 89
Interrupts . . . 89
Activation Flow . . . 90
Credit Management Type . . . 90
Activation Start . . . 91
Deactivation Start . . . 91
Activation/Deactivation Interrupt . . . 94
PM Entry / LinkReset / LinkDisabled Sequence . . . 94
FDI Bringup Flow (AOU_TOP only) . . . 95
Verification Testbench . . . 99

Architecture . . . 100
Components . . . 100
Running the Testbench . . . 101
IP Integration Collateral . . . 101
Generated Outputs (Phase 1) . . . 101
Single Source of Truth . . . 102
Regeneration . . . 102
Planned: Full IP-XACT Component (Phase 2) . . . 103
Timing Constraints (SDC) . . . 103
UPF Power Intent . . . 104
Library Cell Replacement . . . 104
Project Defines . . . 107
14. SIMULATION . . . 108
15. SYNTHESIS . . . 109
16. EMULATION . . . 110
17. Tool predefined names . . . 111
18. Native DV extras . . . 112
19. Derived assertion gates . . . 113
20. Vendor aliases . . . 114
21. Adopter overlay hooks . . . 115
22. Subsystem defines . . . 116
22.1. Shared cells . . . 116
22.2. SMC . . . 116
22.2.1. Simulation hooks . . . 116
22.2.2. CPU cluster . . . 116
22.2.3. i3c-core configuration . . . 117
22.3. DTP . . . 117
22.4. SEP . . . 117
22.4.1. Adams Bridge . . . 117
22.4.2. VeeR EL2 core . . . 118
22.4.3. First-party view checks . . . 118
22.4.4. Key manager . . . 118
22.4.5. DV testbench names . . . 119
23. Synthesis filelist defines . . . 120
23.1. Synopsys Design Compiler and Fusion Compiler . . . 120
23.2. Cadence Genus . . . 120
Prim Cell Replacement and Don’t-Touch Constraints . . . 121
24. Naming Requirement . . . 122
25. Constraint Requirement . . . 123
25.1. After Synthesis . . . 123
26. Simulation . . . 125
CDC Timing Constraints . . . 126
27. Why crossings are bounded rather than timed . . . 127
28. What is constrained . . . 128

28.1. Why protocol-protected data still needs a bound . . . 128
29. The two layers . . . 130
30. CDC/RDC sign-off collateral . . . 131
31. What is covered . . . 132
32. Integration checklist . . . 133
32.1. 1. Repoint the constraints at the pin of the cell inside the prim wrapper . . . 133
32.2. 2. Point the files at your hierarchy, or regenerate . . . 133
32.3. 3. Set the default bound . . . 134
32.4. 4. Declare your own clock groups . . . 134
32.5. 5. Declare periods for generated clocks . . . 135
32.6. Also check . . . 135
33. Confirming it worked . . . 136
34. Scope and limitations . . . 137

Chapter 1. About this Guide

This guide provides instructions and best practices for integrating OCA Harness subsystems into your chiplet design.

The integrator guide is split into multiple chapters. Each of these chapters covers integration considerations, interfaces, and configuration requirements for the respective subsystem.

Initial SMU Integration

The System Management Unit (SMU) is a central part of the Open Chiplet Atlas Harness (OCAH). It comprises three peer subsystems: the System Management Controller (SMC), the Debug and Test Ports (DTP), and the Security Processor (SEP).

The `smu_wrapper` module gives an example structure for the integrator. This has ports that can be used to connect to the user subsystem and the outside of the chiplet. It also has internal "integration modules". These will need to be modified to use memories and fuses that are supplied by the integrator.

Chapter 2. Ports in `smu_wrapper`

There are many groups of ports in `smu_wrapper`. The ports are all described in detail in SMU Top-Level Integration. The following port groups are good starting points in design bring-up: they will be required in any integration.

- **Clocks and reset** The integrator needs to provide a primary system clock, together with a reference clock (for timing-sensitive operations) and a peripheral clock (used for the peripherals' interfaces). There is also a master reset for the chiplet. See *Clock and Reset* in the top-level integration guide.
- **AXI crossbar** The SMU has an internal AXI crossbar. The *SMU AXI Crossbar Address Map* section in the top-level integration guide describes the crossbar's address map. This crossbar should have two connections to the user subsystem: one as a manager and one as a subordinate.
- **GPIO pads** The SMU expects direct connections to GPIO pads. These are exposed as inout ports. The detailed requirements for GPIO pad connections are described in the *GPIO Pad Integration* section in the top-level integration guide.
- **Cross-trigger** The SMU has a cross-trigger interface, which can either be exposed as a *Cross Trigger Matrix Interface* or as GPIO-style signals.
- **JTAG** There is a group of JTAG interfaces, each connected to a TAP. These TAPs are exposed separately, to be connected and chained as appropriate by the integrator. Each TAP also exposes an output that shows whether its output is enabled. See *STAP Scan Chain Topology* for details of these interfaces.
- **Lifecycle** The SMU has a lifecycle controller. Its current lifecycle state is exposed through the `lc_state_o` port. This port is described at *Lifecycle State* in the top-level integration guide.

Chapter 3. Macros needed by "integration modules"

The smu_wrapper module instantiates two "integration modules" (sep_ip_integration and smc_ip_integration). These will need to be modified and completed for any integration.

3.1. Macros needed by SEP

If the SEP is enabled, the macros that must be supplied and instantiated for it are the following. As with the top-level ports, there are also several connections that an integrator may wish to implement (but which are not necessary for the initial structure). The following macros are definitely required:

- **Fuses** The SEP expects some fuses to exist. In sep_ip_integration, these are implemented with efuse_bank_model, which uses a model fuse instance in efuse_bank_reg. An integrator will need to implement either efuse_bank_reg (or the whole of efuse_bank_model) to perform an "APB to fuse" conversion.
- **ROM implementations** The sep_ip_integration module contains two ROMs, which are both implemented with the prim_rom module. These ROMs are: the boot rom (in u_sep_boot_rom) and a key manager ROM (in u_km_rom).
- **SRAM implementations** The SRAM implementations expected by the SEP can be split by the components using them as follows:
  - **Adams Bridge 1r1w RAMs** If SEP_ABR_EN is defined, the SEP contains a copy of the Adams Bridge module. This module needs access to either 9 or 13 1r1w RAMs (the larger number if masking is enabled). A 1r1w RAM is a memory with one read and one write port. These memories are simpler than a general dual-port RAM. Such a memory is implemented in the example with the prim_ram_1r1w module.
  - **The TCM** The VeeR core expects instruction and data memories, split across multiple banks, and communicates with them using sep_cpu_tcm_req_t / sep_cpu_tcm_rsp_t signals. There is a parameterized example wrapper in sep_tcm_wrapper, and this will need to be supplied with single-port RAM instances of appropriate sizes.
  - **Other single-port RAMs** As well as the memories described above, there are four other single port RAM instances: the SEP scratch RAM (u_sep_sram); the key manager RAM (u_km_sram); and OTBN instruction and data memories (u_otbn_imem_sram, u_otbn_dmem_sram).

3.2. Macros needed by SMC

The macros that are needed for the SMC are the following:

- **Fuses** Like the SEP, the SMC expects some fuses to exist. In smc_ip_integration, these are implemented with efuse_bank_model, which uses a model fuse instance in efuse_bank_reg. An integrator will need to implement either efuse_bank_reg (or the whole of efuse_bank_model) to perform an "APB to fuse" conversion.
- **Conversion between core and pad signals** The SMC expects to be able to communicate with GPIO pads (which are exposed through an inout port in smu_wrapper). The integrator's wrapper module must convert between the SMC's binary representation of the pad state and the value in that inout port. In the example, this translation is made by a (non-synthesizable) prim_pad_wrapper module, which will need a synthesizable implementation.
- **ROM** The four-core cluster in the SMC expects a ROM. In the example, this is implemented with a prim_rom instance.
- **SRAM implementations** The SMC expects SRAM implementations for the following components.
  - The four-core cluster in the SMC expects several SRAMs. These back a scratch memory, together with tag and

data memories for an I-cache and D-cache. In the example, these are all implemented with instances of `prim_ram_1p`.

- There are also several backing memories for each I3C instance. There is a single-port "DAT" and a "DCT" memory per I3C instance (implemented in the example by instances of `prim_ram_1p`). There is also a two-port RLT (Retransmission Limit Timer) memory, which is implemented in the example with an instance of `prim_ram_2p`.
- Finally, there are multiple trace RAM instances that can be used to capture debug data. In the example, these are implemented with instances of `prim_ram_1p_adv`.

SoC Integration Checklist

Use this checklist as a quick route through the integration work; each item links to its detailed requirements elsewhere in this guide. Record minimum evidence against each gate before sign-off.

Chapter 4. Integration gates

Done	Integration gate	Minimum evidence	Reference
[ ]	Select the delivered hierarchy, enabled subsystems, and required configuration	Selected hierarchy and configuration record reviewed	SMU Top-Level Integration
[ ]	Assign the chiplet address envelope and program the SEP and SMC apertures	Approved non-overlapping address map; aperture CSRs verified in simulation	Address Map Integration
[ ]	Connect clocks, resets, power-good, and configuration straps; follow the power-up sequence	Clock/reset/power review and sequencing test pass	SMC Integration, SEP Integration
[ ]	Connect external pin, bus, memory, eFuse, peripheral, and debug interfaces	Interface connectivity and tie-off audit complete	SMC Integration, DTP Integration, SEP Integration
[ ]	Verify boot and reset progression, including fuse sensing, memory repair or bypass, ROM execution, and SEP release when enabled	Successful boot and reset log including failure paths	SMC Integration, SEP Integration
[ ]	Apply technology-mapping and CDC constraints; run integration-level verification	Lint, CDC/RDC, constraint, and integration-test reports archived	Prim Cell Replacement, CDC Timing Constraints, OCA Standard Compliance (see Home: Why Open Chiplet Atlas Harness)
[ ]	For each adopter-owned IP, select its trust and attachment class and complete the User IP integration checklist	User IP integration checklist completed per IP block	User IP integration checklist (see the *User IP Integration* chapter)

SMU Top-Level Integration

The System Management Unit (SMU) comprises three peer subsystems: the System Management Controller (SMC), the Debug & Test Ports (DTP), and the Security Processor (SEP). The subsections below describe each subsystem's interfaces independently, followed by the SMU Composition section that explains how they are wired together inside the SMU and how to select between the `smu.sv` logic-level core and the `smu_wrapper.sv` reference integration.

Chapter 5. RTL Filelists

OCAH ships generated, repository-relative filelists under `integration/filelists/`. Run tools that consume them from the repository root.

These filelists are listed in the following table, which gives the filelist needed to synthesize or simulate each system.

System	Synthesis	Emulation	Simulation (Verilator)	Simulation (VCS)
SMU	smu.synth.f	smu.emul.f	smu.verilator.sim.f	smu.vcs.sim.f
SMC	smc.synth.f	smc.emul.f	smc.verilator.sim.f	smc.vcs.sim.f
DTP	dtp.synth.f	dtp.emul.f	dtp.verilator.sim.f	dtp.vcs.sim.f
AoU	aou.synth.f	aou.emul.f	-	-

The filelists for synthesis come from Bender's `synth` target and carry the target's `define` lines. Preserve those lines when passing the filelist to a synthesis tool; see Synthesis filelist defines.

The filelists for simulation are different for Verilator and VCS. Each is the complete filelist generated by that subsystem's DV configuration for the named simulator, including the subsystem's testbench top, supplemental models, and tool-selected stubs. AoU's DV environment is maintained in its vendor folder, so it is not provided at the OCAH level.

Like the synthesis filelists, the simulation filelists carry the target's `define` lines. Preserve these lines when passing the filelist to a simulator.

Regenerate all filelists with `make update-integration-filelists` from the repository root. Add `CHECK=1` to verify that committed outputs are current.

Chapter 6. Address Map Integration

The OCAH address map is configured in three layers, which are described in detail in the TRM at *OCAH Memory Map*.

- **Chiplet envelope (integrator).** The SoC/chiplet address map assigns each subsystem a global base address and reserved size. The decoder in the adopter's fabric routes accesses to these ranges to the SMU. The SMU crossbar preserves offsets and does not constrain envelope size, so the integrator is responsible for non-overlap and containment at the SoC level.
- **SEP and SMC chiplet apertures (firmware).** Each subsystem exposes a chiplet-visible aperture whose base and size are programmable at runtime. Inbound transactions to an aperture are remapped global-to-local before they reach the subsystem.
- **Local-alias views (fixed in RTL).** Internally each subsystem addresses its own resources through a fixed local map. The SMC is at `0xC000_0000`: see *Address Map and Aperture* section in the SMC's section of this integrator guide. The SEP accesses its registers through its own local map. The SMC and SEP internal base addresses are RTL-fixed, not integrator-assigned.

Table 1. Address-map assignment contract

Level	Owner	Assigned / programmed by
Chiplet envelope	Integrator	SoC/chiplet address map + adopter fabric decode
SEP chiplet aperture	SEP firmware	`SEP_GLOBAL_BASE_ADDR` / `SEP_REGION_SIZE` (the `sep_cpu_ctrl` CSRs)
SMC chiplet aperture	SMC firmware	`GLOBAL_BASE` / `REGION_SIZE` (the SMC CPU Control CSRs at `0xC001_0000`)
SEP / SMC local view	Fixed (RTL)	Offsets within each subsystem's local map (SMC local base `0xC000_0000`)

Programming order and constraints: firmware programs the SEP and SMC aperture CSRs during early boot, **before any cross-subsystem AXI traffic is issued**, and must keep the two apertures non-overlapping and inside the chiplet envelope the integrator assigned. The full runtime crossbar behavior and the exact CSR plumbing are in the *Firmware programming order* section of the integrator's guide.

6.1. Register collateral

The machine-readable address map ships in the repository tree; integrators consume it from the release checkout or tarball rather than a separate package. The source of truth is distributed by ownership: each subsystem or IP keeps its canonical register sources and generated outputs beside its implementation. The `integration/` directory centralizes symlinks to that distributed collateral so an integrator does not have to discover every owner-local path.

Each subsystem lists the SystemRDL files that it requires in a flat self-contained list at `integration/rdl/<subsystem>/`. Filenames remain unchanged, and shared dependencies appear in every subsystem that needs them. Non-RDL support files needed by relative includes remain at the `integration/rdl/` root. For example, `integration/rdl/i3c_defines.svh` sits there because both `integration/rdl/smc/registers.rdl` and `integration/rdl/smu/registers.rdl` reach it through the I3C core's own parent-relative include (`../i3c_defines.svh`). That relative path only resolves if the header is one directory above every subsystem folder that needs it. The matching generated IEEE 1685-2014 component descriptions use the same flat subsystem organization under `integration/ipxact/<subsystem>/`.

These directories are generated symlink indexes; run `python3 scripts/update_integration_symlinks.py` after adding or moving collateral. The targets of these symlinks are RDL files, one per subsystem, which remain at their canonical locations in `hw/sys/<sys>/regs/`, `hw/ip/<ip>/regs/`, and `vendor/⋯`. These directories are the `regs/` element of every component's directory layout, described in the *Getting Started Guide*. Other generated outputs remain under each owner's `regs/gen/{c,sv,svh,py,ral,adoc,html,ipxact}/` directory.

For the architecture-level responsibility split between the integrator, firmware, and the internal SEP/SMC maps, see the SEP's *OCAH Memory Map* section in the Technical Reference Manual.

Chapter 7. Immutable ROM FW Integration

OCAH contains three immutable ROM FW images: the SMC production ROM FW, the SEP production ROM FW, and the Key Manager application ROM FW. Each image has two integration contracts. The adopter connects and configures the corresponding ROM macro, and the final ROM FW image must be present before its processor leaves reset. Where ROM FW must initialize or access an adopter-owned peripheral outside OCAH, the image may also expose build-time integration hooks.

The firmware itself is described in the Programmer's Guide appendices: the SMC Boot ROM, the SEP Boot ROM, and the Key Manager Application ROM.

7.1. Weak-symbol override method

An immutable ROM FW image may provide selected vendor-neutral integration hooks as weak symbols. Supply an adopter implementation as a strong definition of the same symbol before the final image is frozen. The strong implementation must program the same adopter-owned IP instantiated by the RTL integration, and its signature, ABI, return values, and documented semantics must match the weak declaration.

This mechanism applies only when the ROM FW needs to interact with a peripheral outside OCAH. It is a build-time image-composition mechanism, not a runtime extension mechanism. Include the adopter implementation in the final ROM FW build so that it replaces the weak definition at link time.

Inspect the final ELF or linker map after linking. For example, `riscv64-unknown-elf-nm` must report each selected implementation as a strong symbol rather than `W`; successful linking alone does not prove that the weak default was replaced.

7.2. SMC ROM FW integration

Connect `smc_rom_intf_req_o` and `smc_rom_intf_rsp_i` to the adopter's ROM implementation. In the current four-core configuration, the request provides the ROM clock, enable, and 14-bit 64-bit-word address, and the response returns one 64-bit word. This is a direct memory interface, not AXI-Lite. The 128 KiB ROM FW image must be present before the first boot; its build is under `hw/sys/smc/bootrom/prod/`.

Connect the ROM macro configuration input reached as `rom_cfg_i` and `mem_cfg_i` according to the selected macro's contract. OCAH forwards these 11 bits without assigning technology-specific semantics; the reference `smc_ip_integration.sv` ties them to zero. Set the eFuse shadow value that drives the CPU's `rom_flip_endianness_i` path to match the byte order of the image and ROM macro. Deasserted passes each 64-bit read word unchanged; asserted reverses its eight bytes.

Table 2. SMC ROM FW override inventory

Hook	Weak default	Rationale	Override and integration action
`smc_pll_init()`	Returns `SMC_PLL_SUCCESS` without programming PLL hardware.	PLL implementation is technology- and vendor-specific, so OCAH does not include a vendor-specific PLL driver.	Override when `BL0_PLLCLK` is asserted and the adopter's PLL requires ROM-time configuration. Provide the same `smc_pll_result_t smc_pll_init(void)` signature in the final ROM FW build. Any result other than `SMC_PLL_SUCCESS` is fatal: the ROM FW records an interface error and remains in `wfi`.

Hook	Weak default	Rationale	Override and integration action
`i3c_release_reset()`, `cfg_ps()`, `init_i3c_ctrl()`, and `I3C_GetDriverInstance()`	Permit the ROM FW to boot without an active I3C implementation; I3C data operations report a hardware error.	OCAH includes an I3C implementation, but the adopter may select a different I3C target and driver for the final system.	When the ROM FW must use an active I3C target, select the open HCI implementation with `I3C_CORE=chipsalliance`, or connect the adopter's I3C target through the SMC external AXI-Lite master (`smc_external_req_o` and `smc_external_resp_i`) and link strong implementations that conform to `drivers/include/i3c_target_driver.h`.

7.3. SEP ROM FW integration

The SEP exposes the `sep_boot_rom` memory interface for its production ROM FW image. The detailed ROM-macro configuration, image-loading contract, and adopter override inventory are not yet documented here.

Table 3. SEP ROM FW override inventory

Hook	Status
To be documented	No adopter override requirement is defined here yet.

7.4. Key Manager ROM FW integration

The Key Manager exposes the `km_rom_mem` interface through SEP for its application ROM FW image. The detailed ROM-macro configuration, image-loading contract, and adopter override inventory are not yet documented here.

Table 4. Key Manager ROM FW override inventory

Hook	Status
To be documented	No adopter override requirement is defined here yet.

Chapter 8. System Management Controller (SMC) Integration

The System Management Controller (SMC) is a subsystem responsible for power management, clock control, reset sequencing, and peripheral interface coordination. The `smc.sv` module exposes logic-level interfaces; all technology-specific components (PLLs, PVT sensors, eFuses, memory macros, GPIO padrings) are instantiated externally by the adopter. See the *SMU Module Variants* section for how the reference wrappers demonstrate this integration.

For detailed architectural documentation about the SMC internal design, refer to `hw/sys/smc/doc/`. The public integration examples are `hw/top/smc_wrapper.sv` and `hw/top/smc_ip_integration.sv`.

8.1. Clock and Reset

The SMC requires three external clock inputs: `clk_smc_i` (primary system clock driving the CPU cluster, fabric, and peripherals), `clk_periph_i` (peripheral clock driving the peripherals), and `clk_ref_i` (reference clock for timing-sensitive operations). All must be generated externally by the adopter's PLL infrastructure; the SMC provides AXI-Lite control outputs for PLL configuration (see Peripheral Interface Connections). The peripheral clock is used to drive the I3C interface, I2C interface, UART interface, and AVSBus interface.

Hold `rst_cold_ni` asserted (low) while power supplies and clocks stabilize. Assert `powergood_i` once supplies reach nominal, then deassert `rst_cold_ni` (drive high) to start the boot sequence.

The SMC generates AASD (async-assert / sync-deassert) reset outputs: `rst_primary_smc_clk_no` for the main clock domain, `rst_primary_periph_clk_no` for the peripheral clock domain, and `rst_primary_ref_clk_no` for the reference clock domain. These ensure clean reset release without metastability across the system hierarchy.

Reset domain overrides are delivered through a single typed input, `jtag_reset_ctrl_i` (`smc_pkg::jtag_smc_reset_ctrl_t`), which carries all SMC reset override enables and active-low values. The DTP drives this input through the JTAG IC_RESET mechanism (see the *SMC-DTP Internal Interconnection* section); most integrations leave all override enables deasserted.

8.2. External CPU cluster NDM reset

Connect `smc_ndmreset_request_i[i]` to the NDM reset request from external CPU cluster `i`, and return `smc_ndmreset_process_o[i]` to that cluster. Set `CpuClusterCount` in `smc_config_pkg` to the number of clusters. This harness sets it to 4. The parameter sizes both ports, and `NDMRESET_CLUSTER_COUNT` reads that value back. The register fields stay 32 bits wide. Tie unused request inputs to 0.

8.3. Address Map and Aperture

The SMC has a chiplet-visible aperture and an internal local address view. Internally, the SMC uses a fixed local base address of `0xC000_0000` and a fixed local window size of 16 MiB. SMC-local addresses (registers, memories, remapped regions) are defined as offsets from that base. The chiplet-visible aperture is adopter-defined: the chiplet or SoC address map assigns the global base and size that external masters can use to reach SMC through the SMU crossbar.

When integrating the SMC into a chiplet or SoC, the adopter assigns a system base address and reserved size for the SMC window. The system base (e.g. `0x0800_0000` in a chiplet address map) may be different from the SMC-local base `0xC000_0000`. Firmware programs the SMC `GLOBAL_BASE` / `REGION_SIZE` CSRs during early boot to match that assignment.

The adopter's fabric or address decoder must route the reserved system range to the SMC block so that offsets are preserved: system base + offset maps to SMC-local `0xC000_0000` + offset.

Table 5. Address Base Mapping

View	Base Address	Description
SMC-local (internal)	`0xC000_0000`	Local alias address (fixed). Used by SMC CPU, firmware, and fabric.
System (adopter)	Adopter-defined	Base of the chiplet/SoC aperture assigned to SMC.

The SMC routes bus transactions from inside the SMU to either be local (accessing something inside the SMC) or global. This routing is configured by three of the SMC base configuration registers:

- `LOCAL_BASE` (read-only, fixed at `0xC000_0000`): The local alias base. Together with `REGION_SIZE`, this is the address space the SMC CPU and internal fabric use to access SMC resources.
- `GLOBAL_BASE` (read-write, defaults to `0x4000_0000`): The system-level base address. Firmware must program this during early boot to match the adopter's system address assignment, enabling the SMC to issue outbound transactions using global system addresses.
- `REGION_SIZE` (read-write, resets to `0x0100_0000` / 16 MiB): The size of both the local alias window and the global aperture. It sizes the SMC input fabric's local/global split as well as SMU-level routing and SEP-to-SMC decode. Firmware must not program it to zero, and must keep `LOCAL_BASE + REGION_SIZE` inside the 32-bit SMC-local address space. SMC units whose window extends past `REGION_SIZE` answer `DECERR` to the SMU and the SEP. At the reset value these are the PLIC, CLINT and bus-error units, so firmware must widen `REGION_SIZE` to cover them before either accesses them.

For full details on alias remapping and outbound routing, see the SMC Fabric documentation (`hw/sys/smc/doc/fabric.adoc`).

For a detailed breakdown of all components within the SMC local map, refer to the SMC Memory Map documentation (`hw/sys/smc/doc/memmap.adoc`).

8.4. AXI Interface Configuration

The SMC provides three distinct AXI interface groups serving different access paths. The `sys_axi_in` interface accepts requests from external system masters requiring access to SMC-managed resources. This interface uses a 56-bit address space with 64-bit data width and 6-bit transaction IDs. Connect this interface to your system fabric for normal operational access from application processors or DMA controllers.

The `sep_axi_in` interface provides dedicated access from the Security Processor (SEP) subsystem. This interface shares the same sizing parameters as the system input but operates independently to maintain security domain separation. Use this interface only when integrating alongside the SEP subsystem. The interface may remain unconnected if your integration does not include SEP.

The `output_axi` interface carries filtered and remapped requests from the SMC to external system resources. This interface uses an 8-bit transaction ID field to accommodate merged traffic from multiple input paths. Connect this interface to your system interconnect to allow SMC components access to external memory and peripherals. An AXI VIP or equivalent BFM is a suitable stand-in during integration verification before the real system fabric is available.

The `jtag_axi_in` interface provides debug access through JTAG mechanisms. This interface uses reduced 2-bit transaction IDs and connects to your JTAG debug infrastructure.

8.5. GPIO Pad Integration

GPIO pad integration requires careful attention to function assignment and pull requirements. The GPIO table below lists each pad's index, function, recommended pull, and integration notes. Default input/output direction is defined by `smc_padring_pkg::DefaultDirectionMap`, not by the table.

The public `hw/top/smc_wrapper.sv` and `hw/top/smc_ip_integration.sv` examples show the pad connections. For bi-directional pins, implement proper tri-state buffers controlled by the `core2pad_en_o` signals. Connect multiplexed UART, I2C, I3C, and SPI pins to external devices according to the function assignments in the GPIO table.

`spi_enable_i` hands the SPI pads to the SPI host. The adopter should not change it while the SPI host is active, or the pad outputs can glitch. In the SMU it is constant, set by whether the SMU has the SEP.

The SMC padring drives one GPIO pad per `u_gpio_interface` instance, so the adopter must integrate `smc_pkg::NumGpioWraps` pads: 61 bonded pads (GPIO 0-60) plus 4 unbonded pads (GPIO 61-64). If the adopter chooses to capture configuration straps from GPIO pads, this can only be done from the bonded pads. The pads for the SDA and SCL lines of I3C[1] must be unbonded for security when transmitting OCCP traffic. The pads for Cool Reset In/Out must also be unbonded as they are meant only to interface with other chiplets within a package. All of these pads must be bi-directional, except for Cool Reset In which must use an input-only pad.

The adopter must instantiate 10 additional pads to support the JTAG PTAP and STAP. This consists of two groups of TCK, TRSTN, TDI, TDO, and TMS — one group for the PTAP and one for the STAP. The TDO for both the PTAP and STAP must use tri-state capable bi-directional pads. The TDI for both the STAP and PTAP must use input-only pads. The remaining PTAP pads must be capable of input, and the remaining STAP pads must be capable of output.

The adopter must connect every cross trigger port (CTP) that `smu` exposes. A CTP that leaves the chiplet must instantiate a group of 4 pads (request out, request in, acknowledge in, acknowledge out). The two inputs must use input-only pads and the two outputs must use bi-directional pads capable of open-drain drive to conform with the cross-trigger methodology described in the *Cross-Trigger Port* section of the Technical Reference Manual.

See *Cross Trigger Port GPIO Interface* for the SMU-level ports.

OCAH also requires a reference clock, a cold reset, and a powergood indication (`clk_ref_i`, `rst_cold_ni`, and `powergood_i` on `smu`). These must be driven, but not necessarily from pads. The adopter may source them from dedicated pads as the GPIO table shows, or from another stable source.

The adopter may choose to instantiate additional pads (with or without supporting instances of the OCAH `gpio` module) to support custom features.

Configuration straps are independent of the GPIO table, and how they are sourced and captured is up to the adopter. Whatever the mechanism, the `STRAPS_LO` and `STRAPS_HI` registers must hold their values by the time the boot ROMs read them. These registers in the adopter peripheral window (see Mandatory Region) define the decoded bit assignments, including `BYPASS_SRAM_REPAIR` at `STRAPS_LO[13]` and `TEST_EN` at `STRAPS_LO[14]`.

8.6. Peripheral Interface Connections

The SMC exposes logic-level interfaces to several technology-specific subsystem components that the adopter instantiates externally. The subsections below describe each connection in turn.

8.6.1. Adopter Peripheral Window

The `smc_external_req_o` and `smc_external_resp_i` signals form a 32-bit AXI-Lite interface to the adopter's own peripheral register maps. Any technology- or product-specific block that SMC firmware needs to reach sits behind this window rather than on a dedicated per-peripheral port.

The window is the recommended home for the control/status registers of logic that the SMC must configure or monitor but that the SMC itself does not implement — padring configuration logic, clock generation, on-die sensors, memory-technology SHIMs, and similar adopter-owned blocks are typical examples. The SMC decodes the aperture and forwards the transaction; what lives behind it is entirely adopter-defined.

The window starts at `BASE + 0x040_0000` (`0xC040_0000` with `LOCAL_BASE`), and its size is the extent of the adopter's `smc_external` register map. The SMC forwards every access in the window to `smc_external_req_o` except the eFuse SHIM range at its base (see Mandatory Region); the adopter decodes everything else.

The window is split into two regions. Apart from the eFuse SHIM control block needed for security reasons, hardware treats both identically — the split is organizational only:

- **Mandatory** (`BASE + 0x040_0000`): Blocks every integration must implement. OCAH specifications dictate their function, but their implementation, register layout and placement within the region are technology- and vendor-specific.
- **Supplementary** (`BASE + 0x040_4000`): Any additional memory-mapped devices the adopter chooses to expose. Blocks currently included are purely suggestions and examples, adopters are not required to implement.

Mandatory Region

Block	Purpose
eFuse SHIM control	Technology-specific eFuse bank control, such as the time the OTP macro needs to initialize before sensing
Per-pad GPIO control	One control block per pad: pad electrical settings, the value captured at reset deassertion, and the secondary hardware function override
PLL wrapper	Control and status of the adopter's clock-generation block
Straps	Configuration strap values read by boot firmware. The capture mechanism is adopter-defined

The eFuse SHIM control CSRs must sit at the base of the mandatory region. In each subsystem, the internal fabric routes the first `*_EFUSE_SHIM_SIZE` bytes of the window into the eFuse controller, so the controller's lifecycle access control applies to them. The accesses then leave on the dedicated eFuse bank port, `smc_efuse_bank_ctrl_req_o` or `sep_efuse_bank_ctrl_req_o`, instead of `smc_external_req_o` or `sep_external_req_o`. See [efuse-bank-ctrl-window].

Supplementary Region

Block	Purpose
Controller wrapper	An adopter peripheral controller, such as an I3C controller
Extra GPIO pad interface	Register-driven data and direction for a pad outside the per-pad GPIO array
Extra GPIO pad control	Drive strength, pull enable and select, and Schmitt select for that pad
PVT wrapper	Control and status of the adopter's process, voltage and temperature sensors

8.6.2. eFuse Interface

NOTE This guide uses **eFuse** and **OTP** interchangeably for the same one-time programmable bank and the

> SHIM that fronts it. Port names mix both (`efuse_*`, **`otp`**); they refer to the same interface.

The eFuse interface consists of two parts:

- `efuse_bank_ctrl` — a 32-bit AXI-Lite interface used to access the SHIM control CSRs (for example, foundry-specific bank timing such as the initialization delay before sensing can begin). Firmware reaches these CSRs through the base of the adopter external window; see [efuse-bank-ctrl-window].
- `efuse_shim_command` — the custom fuse command interface used to issue read, program, and program/read-back operations to the eFuse bank.

Adopter responsibility. The `efuse_shim_command` interface is intentionally generic and technology-neutral. The adopter must implement an Interface SHIM that converts these generic commands into the hardware signals required by their OTP macro. The conversion is driven by the adopter's own read and program state machines inside the SHIM; OCAH only defines the command/response contract, not the macro-side signaling.

The SMC and the SEP each contain their own eFuse controller, so the `smu.sv` boundary carries two independent copies of the interface, one with an `smc_` prefix and one with a `sep_` prefix. The adopter places **two separate SHIM and macro instances**, one per controller, each fronting its own 8 Kbit bank. The banks are never shared. Each controller also drives its own sense-done output to the `smu.sv` boundary, `smc_fuse_sense_done_o` and `sep_fuse_sense_done_o`, and holds the sensed bank in its own shadow registers. eFuse connections at the smu.sv boundary shows these connections. When the SMU is built with `CFG.SEP = 0`, only the SMC instance is needed.

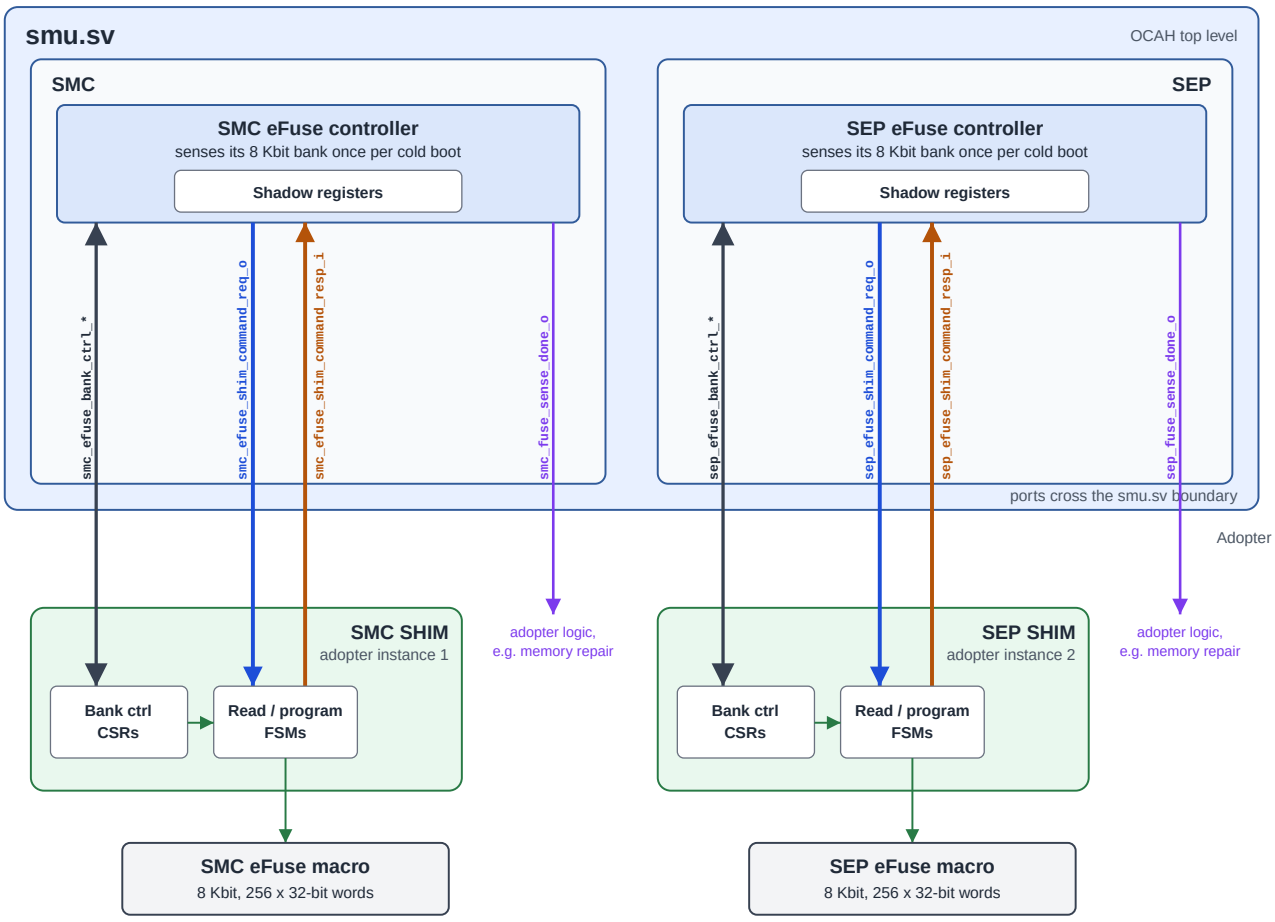


Figure 1. eFuse connections at the smu.sv boundary

eFuse controller to Interface SHIM contract shows the contract for one of those instances. Everything above the dashed line is provided by OCAH; everything below it is the adopter's. Only the two interfaces that cross the line are fixed.

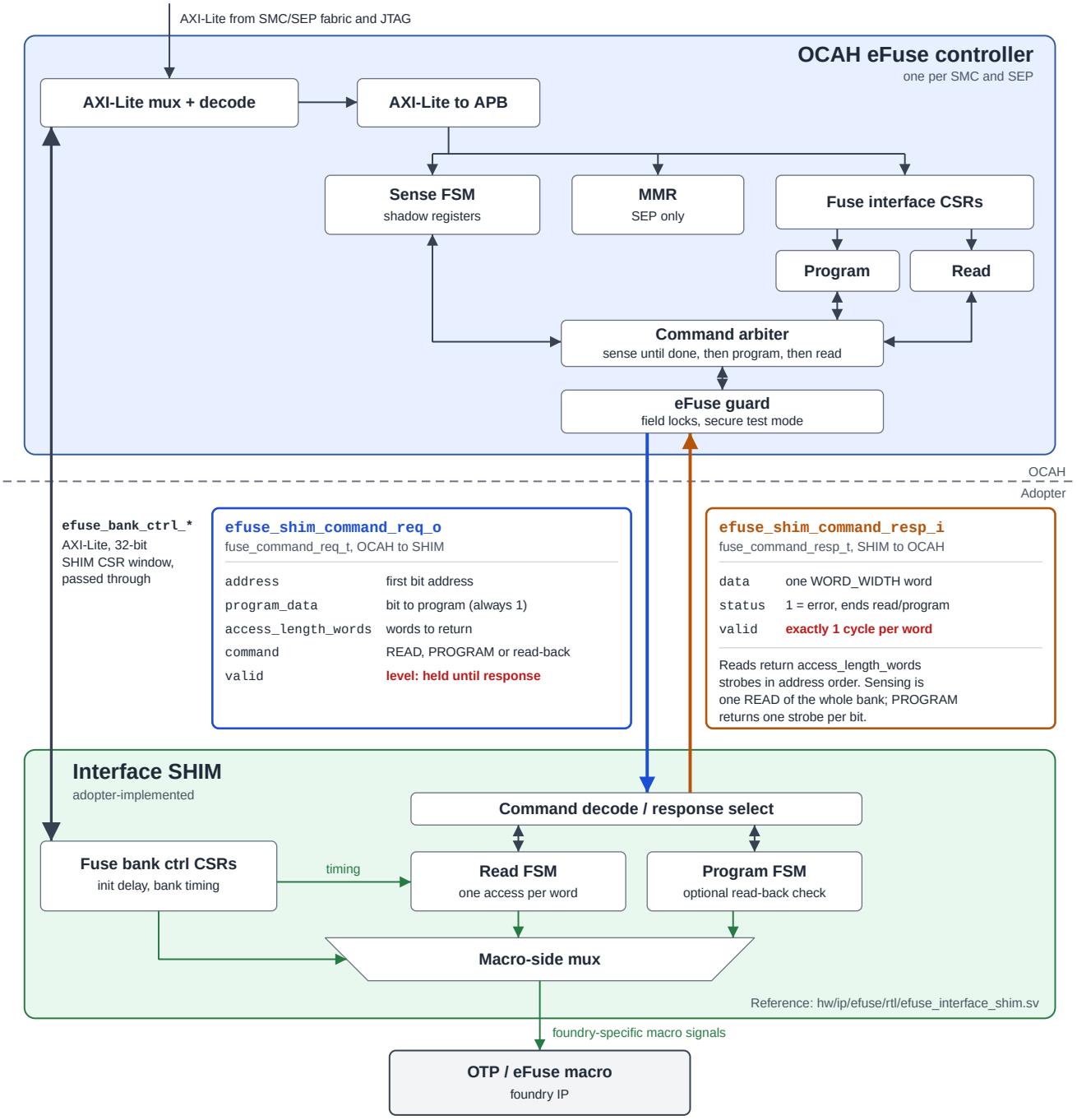


Figure 2. eFuse controller to Interface SHIM contract

The reference SHIM, `hw/ip/efuse/rtl/efuse_interface_shim.sv`, drives an APB-based behavioral bank model. Use it as a worked example of the read and program state machines, not as a template for macro-side timing.

SHIM control address window. The SHIM control CSRs are defined by the adopter, but their addresses sit at the base of the subsystem's adopter external window. The fabric carves that block off the window and routes it to the eFuse controller instead of the external port. The controller does not decode it; it forwards every access outside its own register blocks to `efuse_bank_ctrl` unchanged. The rest of the external window leaves on the subsystem's external port as usual.

Table 6. SHIM control address window

Subsystem	Window base	Size parameter (SMU default)	Rest of the window
SMC	`smc_external` base, SMC-local `0xC040_0000`	`CFG.SMC_EFUSE_SHIM_SIZE` (`0x4`)	`smc_external_req_o` / `smc_external_resp_i`
SEP	`sep_external` base, SEP-local `0x2000_0000`	`CFG.SEP_EFUSE_SHIM_SIZE` (`0x4`)	`sep_external_req_o` / `sep_external_resp_i`

The SHIM must follow these rules for this window:

- **Fit the window.** Size the parameter to cover every SHIM control register. Accesses past it go to the external port and never reach the SHIM.
- **Describe the registers in the external map.** Place the SHIM control block at offset `0x0` of the adopter's `smc_external` and `sep_external` register maps. The open SMC map reserves the whole window as opaque memory. SEP takes the window base from the `efuse_shim_ctrl` instance in `sep_external`, so an adopter map that replaces it must keep an instance of that name at offset `0x0`.

The reference register block is `efuse_shim_ctrl` (`hw/ip/efuse/dv/models/regs/efuse_shim_ctrl.rdl`), which holds a single `EFUSE_BANK_INIT_TIME` register.

The command and response payloads are exchanged as packed structs:

```
// Controller -> SHIM
typedef struct packed {
    addr_t                address;             // Target address (in bits)
    logic                 program_data;        // Data to program (when programming)
    access_length_words_t access_length_words; // Number of WORD_WIDTH words to access
    command_t             command;             // READ / PROGRAM / PROGRAM_READ_BACK
    logic                 valid;               // Request valid
} fuse_command_req_t;

// SHIM -> Controller
typedef struct packed {
    data_t data;   // Read data (one WORD_WIDTH word)
    logic  status; // Operation status (0 = OK, 1 = error)
    logic  valid;  // Response valid (qualifies `data` for one cycle)
} fuse_command_resp_t;
```

`command` is `efuse_pkg::fuse_command_e`: `FUSE_COMMAND_READ` (`2'b00`), `FUSE_COMMAND_PROGRAM` (`2'b01`) and `FUSE_COMMAND_PROGRAM_READ_BACK` (`2'b10`).

The SHIM implements two operations, reads and programs:

- **Reads.** If a request has `command` set to `FUSE_COMMAND_READ` and `access_length_words` set to *N*, respond with *N* `valid` strobes, each carrying one `WORD_WIDTH` word in `data`, reading consecutive words starting at `address`. Firmware reads set *N* to one; fuse sensing sets it to the whole bank, as described below.
- **Programs.** If a request has `command` set to `FUSE_COMMAND_PROGRAM`, program the single bit at `address` and respond with one `valid` strobe when the macro finishes. `program_data` is always 1 and `access_length_words` is always 1. For `FUSE_COMMAND_PROGRAM_READ_BACK`, also read the word back, return it in `data`, and set `status` if the programmed bit did not stick.

Both operations follow these rules:

- **Bit addressing.** `address` is a bit address. Convert it to whatever word or byte addressing the macro uses; the reference SHIM shifts it right by three.
- **One command at a time.** `valid` is a level, not a strobe. The controller holds `valid` and the whole request payload stable from the cycle it issues a command until it has received the response, so the SHIM may sample the payload at any point while the command runs.
- **When `valid` falls.** After a firmware read or program, `valid` falls in the cycle after the response strobe. After sensing it stays high until `fuse_sense_done` moves the arbiter to the firmware path, two cycles after the last strobe; do not treat that still-asserted request as a second read. The controller also withdraws `valid` without waiting for a response in these cases, and the SHIM must abandon the command it had started:
  - a firmware read or program timeout expires*
  - firmware changes the read or program enable, where program takes priority over read*
- **Errors.** Setting `status` ends a firmware read or program command with an error. The sense FSM counts only `valid` strobes, so a sensing error must still be followed by the remaining strobes.
- **No access control.** The eFuse guard zeroes requests to locked fields, and every request while the SEP is in secure test mode, in the same cycle the request is issued. A blocked request therefore never reaches the SHIM: `valid` does not rise, and no response is expected. Secure test mode is latched once per cold boot, before any firmware command can be issued, so it cannot begin in the middle of a command. The SMC controller has no secure test mode.

* These scenarios will be fixed in a future release so that the controller never withdraws `valid` before the response.

IMPORTANT

Fuse sensing happens **once per cold boot**, when the eFuse controller's primary reset is released. It runs in **both** eFuse controller instances, SMC and SEP, each against its own bank. CPU and warm resets do not repeat it.

The adopter must provide **8 Kbit of fuses (8192 bits, 256 32-bit words) for each instance**: one bank behind the SMC SHIM and a second, separate bank behind the SEP SHIM. For both, `NumShadowWords` is 256.

Fuse sensing protocol. During automatic sensing the controller issues a single read with `access_length_words` set to the entire bank: `NumShadowWords`, which is `SHADOW_REG_BITS / WORD_WIDTH`. The SHIM is expected to stream the bank back one `WORD_WIDTH` word at a time: for each OTP access it drives the read `data` onto `efuse_shim_command_resp_i.data` and asserts `valid` for **exactly one cycle**. The controller consumes one word per cycle that `valid` is high, advancing to the next eFuse word on each assertion.

eFuse sensing response stream shows the stream for the 256-word bank. The sense FSM leaves `ST_READ` after one cycle and waits in `ST_WAIT`. Each `valid` strobe stores `data` into shadow word `words_received_q` and increments the count one cycle later. Gaps between strobes are allowed, as between cycles 2 and 4, and so are strobes on consecutive cycles, as in cycles 4 and 5, provided each cycle carries a new word. `data` is sampled only while `valid` is high. The strobe for word 255 (`NumShadowWords - 1`) stores the last word and moves the FSM to `ST_FINISHED` without incrementing. `fuse_sense_done` rises one cycle after that.

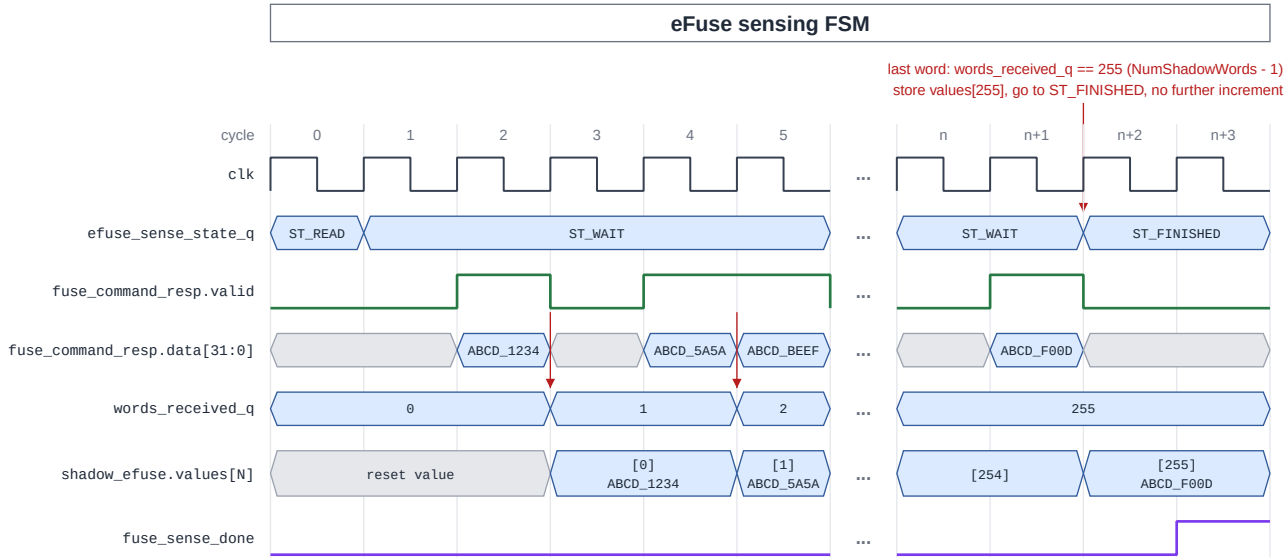


Figure 3. eFuse sensing response stream

WARNING

`valid` must be asserted for only a single cycle per returned word. If the SHIM holds `valid` high for two cycles, the controller will sample the same `data` value a second time and count it as the data for the **next** eFuse word, shifting all subsequent words and corrupting the sensed shadow contents. The adopter's SHIM FSM is responsible for producing a clean one-cycle `valid` strobe per `WORD_WIDTH` access.

During cold boot the SMC and SEP each sense their own bank into their own shadow registers. `smc_fuse_sense_done_o` and `sep_fuse_sense_done_o` indicate when each sensing operation completes.

The `smc_shadow_regs_o` output gives adopter read-only access to the sensed SMC bank. The SMU has no equivalent SEP output: the SEP bank holds secrets such as the class key and device UIDs, so its shadow registers stay inside SEP.

8.7. Memory Subsystem Integration

The SMC includes an embedded RISC-V CPU cluster with associated memory interfaces. See *Immutable ROM FW Integration* for the ROM FW integration contract.

8.7.1. SRAM and Cache Array Integration

Scratch RAM interfaces (`smc_scratch_ram_intf_req_o` and `smc_scratch_ram_intf_rsp_i`) connect to 32 independent RAM banks totaling the scratchpad memory space. Cache memory interfaces for L1 instruction and data caches (`smc_l1_icache_*` and `smc_l1_dcache_*`) connect to tag and data RAM banks as specified in the port table.

The `smc_disable_sram_auto_init_i` input controls automatic memory initialization behavior. When deasserted, the SMC automatically zeroes all SRAM contents during bring-up. The `smc_init_mem_done_o` output indicates completion of this initialization sequence. The `SRAM_AUTO_ZERO_DISABLE` strap (`STRAPS_LO[26]`) can disable this automatic zeroing behavior if your integration requires preserved SRAM contents across resets.

8.8. Memory Repair Integration

The SMC implements Built-In Redundancy Analysis (BIRA) memory repair to address manufacturing defects in SRAM

arrays. Repair uses redundant memory elements configured with data stored in eFuse memory, replacing defective cells detected during manufacturing test.

Memory repair executes automatically during boot after eFuse sensing completes and before MBIST operations. The process must complete successfully before the CPU cluster can safely access SRAM arrays. Repair is required on every cold reset but can be bypassed during Function Level Reset (FLR) operations for faster recovery.

8.8.1. Control Signals

- `smc_fuse_sense_done_o`: eFuse controller output indicating fuse sensing completion. External DFT logic monitors this signal to trigger repair operations.
- `skip_mem_repair_o`: Reset unit output indicating repair should be bypassed. It is driven by the isolation request inputs (`isolate_req_pin_sync_smc` or the SMC isolation register), not directly by the strap. When asserted, external repair logic should skip operations and immediately assert completion status.
- `ext_boot_seq_done_i`: External input indicating the boot sequence (memory repair and shadow register override) is complete. Hold low to pause reset release until external logic gives the go-ahead; required before the system can continue its boot sequence.

8.8.2. Repair Data

The SMC fuse map does not allocate storage for repair data. Your external memory repair controller must source it from wherever your manufacturing flow keeps it and apply it to SRAM arrays according to your memory architecture.

8.8.3. Status Monitoring

Repair completion and success are reported through `DFX_CTRL_STATUS.STATUS_SMU` at `0xC000_B800`. The `mem_repair_done` bit indicates completion regardless of success, while `mem_repair_success` indicates successful repair. Both bits are set by the external repair controller. The SMC ROM FW monitors these bits during initialization to verify repair completion.

8.8.4. Bypass Configuration

The `BYPASS_SRAM_REPAIR` configuration strap is reported in `STRAPS_LO[13]` of the strap registers in the adopter peripheral window. Keep these three mechanisms distinct:

- No RTL path drives `skip_mem_repair_o` from this strap directly. `skip_mem_repair_o` is driven from the isolation request path.
- Production ROM reads the strap and, when asserted, skips repair-status validation and writes `MBIST_STATUS_MEM_REPAIR_BYPASSED` to scratchpad register 15 (`SMC_SCRATCH_MBIST_STATUS`, `0xC003_90F8`). It does not cause the external repair controller to skip physical repair.
- External repair logic independently honors `skip_mem_repair_o` to decide whether to bypass physical repair operations.

Use bypass mode only when SRAM arrays are guaranteed defect-free, such as during development or with known-good components.

8.8.5. Integration Requirements

Provide an external memory repair controller that monitors `smc_fuse_sense_done_o` to initiate repair, respects `skip_mem_repair_o` for bypass control, reads repair data from eFuse shadow registers, applies repair configuration to

SRAM arrays, asserts ext_boot_seq_done_i and updates DFX_CTRL_STATUS with completion status. The controller operates independently of the SMC core and must complete before the CPU cluster accesses SRAM.

Note: For a standalone SEP, the adopter must synchronize ext_boot_seq_done_i to the system clock and ensure the signal is held until next cold reset.

For direct access to the eFuse not through the SHIM, i.e. from a DFT fuse controller, adopters must ensure the integrated logic will not interfere with accesses from the OCAH's efuse interface.

The isolation request path asserts skip_mem_repair_o during Function Level Reset (FLR) operations to bypass repair for rapid reset sequences. Normal cold reset sequences always execute repair unless isolation is already asserted.

8.9. Interrupt Integration

The SMC aggregates interrupts from multiple sources into cpu_interrupts_o, which feeds the PLIC of the embedded RISC-V CPU cluster. The interrupt vector is assembled in hw/sys/smc/rtl/smc_base.sv and hw/sys/smc/rtl/smc_peripherals/rtl/smc_peripherals.sv. The authoritative bit-level map (RTL trace and PLIC source ID note) is maintained in a single place:

See the *SMC CPU interrupt vector map* in the Technical Reference Manual.

The SMC PLIC and CPU interrupt vector are internal. sync_irq_o is **not** the interrupt aggregate — it is the software-controlled SYNC_REG.sync global synchronization bit in the reset unit, unrelated to PLIC interrupt delivery. Do not connect an external interrupt controller to sync_irq_o.

External interrupt sources visible to the adopter are smc_ext_interrupts_i (SMC external interrupt input vector, width NUM_INT_TO_SMC), smc_ext_mailbox_interrupts_o (outbound mailbox notifications, width NumMailboxes), gpio_interrupt_o (GPIO edge/level events, width NumGpioWraps), and uart_interrupt_o (UART TX/RX events, width NumUart). Drive smc_ext_interrupts_i as an active-high level or as a pulse that stays high long enough to be sampled on clk_smc. The pin is three-flop synchronized; a short asynchronous pulse can be lost. There is no edge-triggered mode. For the full PLIC source map see the SMC Technical Reference Manual.

8.10. Telemetry and Debug Integration

For system-level debug and telemetry, the SMC implements ATB (Advanced Trace Bus) receiver interfaces. The telemetry interface group (telemetry_atdata_i, telemetry_atid_i, telemetry_atvalid_i, telemetry_atready_o) operates in a dedicated clk_telemetry_i clock domain with separate rst_telemetry_ni reset. Connect these signals to ATB trace sources for debug data collection.

At the SMC boundary, ext_debug_bus_i is 512 bits wide; the SMU concatenates the adopter-visible 128-bit input with the SEP's internal 384-bit bus before driving SMC. The typed xtrigger_ss_i / xtrigger_ss_o ports are the only cross-trigger signals at the SMC boundary. The adopter-visible SMU cross-trigger interfaces are xtrig_ctm_*, xtrig_clk_stop_req_i, and the CTP groups.

Trace memory interfaces (trace_mem_req_o and trace_mem_resp_i) connect to trace RAM instances for storing captured debug data. smc_ip_integration.sv shows an example set of RAM instances that can be adapted to your debug requirements.

8.11. Bring-Up Sequence

The recommended bring-up sequence begins with asserting `rst_cold_ni` while stabilizing power supplies and clocks. Once all supplies reach nominal voltages, deassert `rst_cold_ni` to initiate the SMC boot sequence. The SMC immediately begins fuse sensing to read configuration values and security credentials.

Monitor the `smc_fuse_sense_done_o` signal to detect completion of fuse sensing. The configuration straps the boot ROMs read determine boot modes and operational settings. After fuse sensing completes, the SMC releases internal subsystem resets based on the programmed reset sequencing configuration.

The CPU cluster begins executing the ROM FW after reset release. The ROM FW performs additional initialization including peripheral configuration, memory testing (unless disabled), and system health checks. The `smc_init_mem_done_o` signal indicates completion of memory initialization if automatic SRAM zeroing is enabled.

To support Function Level Reset (FLR), assert `cfg_flr_pf_active_i` to trigger a warm reset sequence. The SMC asserts `isolate_req_o` bits for the affected subsystem indices. The adopter's power controller drives `ss_reset_complete_i` high once the subsystem reset is complete; the SMC synchronizes this into the readable `SS_RESET_COMPLETE` register. Isolation remains asserted until cleared by software: SMC firmware must poll `SS_RESET_COMPLETE` and then write `ISOLATE_REQ_SMC_REG` to clear the FLR isolation latch. Do not assume isolation releases automatically on `ss_reset_complete_i`.

See `hw/top/smu_wrapper.sv` and `hw/top/smc_ip_integration.sv` for reference bring-up wiring examples covering clock, reset, strap, and peripheral connections.

8.12. SMC Port Interface

The SMC exposes a comprehensive set of interfaces for system integration. The following table defines all SMC top-level ports including clocks, resets, AXI interfaces, peripheral connections, and control signals. Refer to the detailed SMC documentation for internal signal descriptions and timing requirements.

Table 7. SMC Port Declaration

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
`clk_smc_i`	`logic`	Input	1	Primary SMC clock domain. All SMC internal logic operates synchronously to this clock.	PLL output	N/A
`clk_ref_i`	`logic`	Input	1	Reference clock domain. Used for timing-sensitive operations and peripheral interfaces.	PLL reference clock	N/A
`clk_periph_i`	`logic`	Input	1	Peripheral clock domain.	PLL peripheral clock	N/A
`powergood_i`	`logic`	Input	1	Power-good indication. Assert after supply voltages stabilize.	Power management controller	N/A
`powergood_stable_o`	`logic`	Output	1	Stabilized power-good output. Debounced version of `powergood_i`.	Reset unit → External systems	N/A
`rst_cold_ni`	`logic`	Input	1	Cold reset input (active low). Assertion and deassertion are synchronous to `clk_ref_i` and do not propagate while that clock is stopped. Assertion is de-glitched for 32 cycles. A falling `powergood_i` asserts the downstream cold reset without that wait. Deassertion is held for 255 cycles.	System cold reset	N/A
`rst_primary_ref_clk_no`	`logic`	Output	1	Primary reset synchronized to reference clock domain (active low).	To subsystems using ref clock	N/A

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
rst_primary_smc_clk_no	logic	Output	1	Primary reset synchronized to SMC clock domain (active low).	To subsystems using SMC clock	N/A
sys_axi_in_req_i	smc_sys_in_56_64_6_12_axi_req_t	Input	-	System AXI input request (56-bit addr, 64-bit data, 6-bit ID, 12-bit user).	External system masters → Input Fabric	N/A, tie to '0 if unused
sys_axi_in_resp_o	smc_sys_in_56_64_6_12_axi_resp_t	Output	-	System AXI input response.	Input Fabric → External system masters	N/A, leave floating if unused
jtag_axi_in_req_i	smc_jtag_56_64_2_12_axi_req_t	Input	-	JTAG debug AXI request (56-bit addr, 64-bit data, 2-bit ID, 12-bit user).	JTAG debug port → Input Fabric	N/A, tie to '0 if unused
jtag_axi_in_resp_o	smc_jtag_56_64_2_12_axi_resp_t	Output	-	JTAG debug AXI response.	Input Fabric → JTAG debug port	N/A, leave floating if unused
axil_smc_otp_jtag_req_i	smc_pkg::smc_axil_32_32_req_t	Input	-	eFuse OTP/JTAG AXI-Lite request.	JTAG → eFuse controller	N/A, tie to '0 if unused
axil_smc_otp_jtag_resp_o	smc_pkg::smc_axil_32_32_resp_t	Output	-	eFuse OTP/JTAG AXI-Lite response.	eFuse controller → JTAG	N/A, leave floating if unused
sep_axi_in_req_i	smc_sep_in_56_64_6_12_axi_req_t	Input	-	SEP AXI request (56-bit addr, 64-bit data, 6-bit ID, 12-bit user).	SEP → Input Fabric	N/A, tie to '0 if unused
sep_axi_in_resp_o	smc_sep_in_56_64_6_12_axi_resp_t	Output	-	SEP AXI response.	Input Fabric → SEP	N/A, leave floating if unused
output_axi_req_o	smc_sys_out_56_64_8_12_axi_req_t	Output	-	Filtered and remapped AXI output request (56-bit addr, 64-bit data, 8-bit ID, 12-bit user).	Output Fabric → External system resources	N/A, leave floating if unused
output_axi_resp_i	smc_sys_out_56_64_8_12_axi_resp_t	Input	-	Filtered and remapped AXI output response.	External system resources → Output Fabric	N/A, tie to '0 if unused
efuse_bank_ctrl_req_o	smc_pkg::smc_axil_32_32_req_t	Output	-	eFuse bank control AXI-Lite request.	eFuse controller → eFuse Shim	N/A
efuse_bank_ctrl_resp_i	smc_pkg::smc_axil_32_32_resp_t	Input	-	eFuse bank control AXI-Lite response.	eFuse Shim → eFuse controller	N/A
efuse_shim_command_req_o	smc_efuse_pkg::fuse_command_req_t	Output	-	eFuse command interface request.	eFuse controller → eFuse Shim	N/A
efuse_shim_command_resp_i	smc_efuse_pkg::fuse_command_resp_t	Input	-	eFuse command interface response.	eFuse Shim → eFuse controller	N/A
shadow_regs_o	smc_efuse_pkg::efuse_map_t	Output	-	eFuse shadow register output.	eFuse controller → External systems	N/A
pll_clk_obs_i	logic	Input	1	PLL clock observation input.	PLL → GPIO padring (muxed)	N/A
pll_clk_obs_en_i	logic	Input	1	PLL clock observation enable.	Control logic → GPIO padring	N/A
pvt_process_clk_obs_i	logic	Input	1	PVT process monitor clock observation input.	PVT → GPIO padring (muxed)	N/A
pvt_process_clk_obs_en_i	logic	Input	1	PVT process monitor clock observation enable.	Control logic → GPIO padring	N/A
cat_therm_i	logic	Input	1	CAT thermal sensor input.	CAT thermal sensor → PVT wrap	N/A, tie to '0 if unused
lsio_interface_select_o	logic	Output	[NumGpioWraps-1:0]	LSIO interface selection for each GPIO wrap.	GPIO padring → GPIO wraps	N/A
pad2core_i	logic	Input	[NumGpioWraps-1:0]	GPIO input data from pad ring to core.	GPIO padring → SMC	N/A

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
core2pad_o	logic	Output	[NumGpioWraps-1:0]	GPIO output data from core to pad ring.	SMC → GPIO padring	N/A
pad2core_en_o	logic	Output	[NumGpioWraps-1:0]	GPIO pad-to-core path enable.	SMC → GPIO padring	N/A
core2pad_en_o	logic	Output	[NumGpioWraps-1:0]	GPIO core-to-pad path enable.	SMC → GPIO padring	N/A
rst_cool_n_from_pin_i	logic	Input	1	Cool reset input from GPIO pin 61.	GPIO pin 61 → Reset Unit	N/A
clk_telemetry_i	logic	Input	1	Telemetry clock domain. Separate clock for ATB telemetry data collection.	Telemetry clock source	N/A
rst_telemetry_ni	logic	Input	1	Telemetry reset (active low). Resets telemetry receiver logic.	Telemetry reset source	N/A
telemetry_atdata_i	telemetry_receiver_pkg::telemetry_data_t	Input	[NumTelemetryReceivers-1:0]	ATB telemetry data input array.	ATB sources → Telemetry receivers	N/A
telemetry_atid_i	telemetry_receiver_pkg::atb_id_t	Input	[NumTelemetryReceivers-1:0]	ATB telemetry ID input array.	ATB sources → Telemetry receivers	N/A
telemetry_atready_o	logic	Output	[NumTelemetryReceivers-1:0]	ATB telemetry ready output.	Telemetry receivers → ATB sources	N/A
telemetry_atvalid_i	logic	Input	[NumTelemetryReceivers-1:0]	ATB telemetry valid input.	ATB sources → Telemetry receivers	N/A
telemetry_afvalid_o	logic	Output	[NumTelemetryReceivers-1:0]	ATB flush valid output.	Telemetry receivers → ATB sources	N/A
telemetry_afready_i	logic	Input	[NumTelemetryReceivers-1:0]	ATB flush ready input.	ATB sources → Telemetry receivers	N/A
smc_cluster_ded_o	logic	Output	1	Cluster double error detection output.	CPU cluster → External systems	N/A
smc_wdt_first_timeout_o	logic	Output	1	First watchdog timeout signal.	CPU cluster WDT → Reset Unit	N/A
smc_wdt_second_timeout_o	logic	Output	1	Second watchdog timeout signal.	Reset Unit → External systems	N/A
smc_region_size_o	logic	Output	[31:0]	SMC address region size output.	SMC Internal Registers → External systems	N/A
smc_ext_interrupts_i	logic	Input	[NumExtInterrupts-1:0]	External interrupt inputs. Width is 256. Active-high level or a pulse long enough to be sampled on clk_smc. Synchronized to clk_smc (3-stage) in smc_base; a short asynchronous pulse can be lost.	External sources → Interrupt aggregator	N/A, tie to '0 if unused
sep_mailbox_interrupts_i	logic	Input	[7:0]	SEP → SMC mailbox interrupt inputs (8 channels). Routed to internal interrupts to the SMC CPU.	SEP → Interrupt aggregator	N/A, tie to '0 if unused
sep_wdt_reset_n_i	logic	Input	1	SEP watchdog reset (active low). Active-low polarity is inverted before routing to internal interrupts to the SMC CPU.	SEP → Interrupt aggregator	N/A
smc_fuse_sense_done_o	logic	Output	1	Fuse sense completion output.	eFuse controller → Memory repair logic	N/A
smc_fuse_reset_n_delayed_o	logic	Output	1	Delayed fuse reset output (after 16 pipeline stages).	eFuse controller → External systems	N/A

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
skip_mem_repair_o	logic	Output	1	Skip memory repair output.	Reset Unit → Memory repair logic	N/A
ext_boot_seq_done_i	logic	Input	1	Asynchronous external boot sequence completion (memory repair and shadow register override done). Qualified in smc_peripherals and routed to the SMC and SEP eFuse controllers.	External boot/repair controller → SMC peripherals	N/A
lc_state_i	logic	Input	[2*LcStateWidth-1:0] (8 bits)	Lifecycle state input, differentially encoded.	Lifecycle controller → SMC Internal Registers	From SEP; tie to 8'hf0 if unused (encoded TEST_DEV)
smc_ext_mailbox_interrupts_o	logic	Output	[NumMailboxes-1:0]	External mailbox interrupt outputs (32 interrupts, mapped to IDs 156-187).	Mailbox controller → External targets	N/A
cfg_flr_pf_active_i	logic	Input	1	Function Level Reset (FLR) PF active signal.	PCIe/System → Reset Unit	N/A, tie to '0 if unused
smc_ndmreset_request_i	logic	Input	[CpuClusterCount-1:0]	NDM reset request from external CPU cluster i on bit i. Synchronized onto clk_smc.	External CPU clusters → SMC	Tie to '0 if unused
smc_ndmreset_process_o	logic	Output	[CpuClusterCount-1:0]	Firmware response to external CPU cluster i on bit i.	SMC → external CPU clusters	N/A
isolate_req_o	logic	Output	[31:0]	Isolation request output for 32 subsystems.	Reset Unit → 32 subsystems	N/A
ss_reset_complete_i	logic	Input	[31:0]	Subsystem reset completion status.	32 subsystems → Reset Unit	32'hFFFFFFFF
ss_config_o	logic	Output	[31:0]	Subsystem configuration output.	Reset Unit → 32 subsystems	N/A
ss_reset_ctrl_o	smc_reset_unit_pkg::reset_ctrl_t	Output	[31:0]	Subsystem reset control output array.	Reset Unit → 32 subsystems	N/A
sync_irq_o	logic	Output	1	Software-controlled global synchronisation bit (SYNC_REG.sync in the reset unit). Not an interrupt aggregate output.	Reset unit → External	N/A
gpio_interrupt_o	logic	Output	[NumGpioWraps-1:0]	Raw GPIO interrupt vector from the padring, one bit per GPIO wrap (bonded + unbonded). Software reads GPIO status registers to identify the interrupt source. Already OR-reduced into peripheral_interrupts_o[29:28] internally.	smc_padring → external consumers	N/A, leave floating if unused
uart_interrupt_o	logic	Output	[NumUart-1:0]	Raw UART interrupt outputs, one bit per UART instance, in the peripheral clock domain. Already aggregated into peripheral_interrupts_o[21:18] internally via CDC.	uart_wrap → external consumers	N/A, leave floating if unused
smc_rom_intf_req_o	rom_req_t	Output	-	ROM memory interface request. Width depends on CPU cluster configuration	CPU cluster → Boot ROM	N/A
smc_rom_intf_rsp_i	rom_rsp_t	Input	-	ROM memory interface response. Width depends on CPU cluster configuration	Boot ROM → CPU cluster	N/A
smc_scratch_ram_intf_req_o	scratch_ram_req_t	Output	[NUM_SRAM_BANKS2-1:0]	Scratch RAM interface requests. Width depends on CPU cluster configuration	CPU cluster → Scratch RAM banks	N/A
smc_scratch_ram_intf_rsp_i	scratch_ram_rsp_t	Input	[NUM_SRAM_BANKS2-1:0]	Scratch RAM interface responses. Width depends on CPU cluster configuration	Scratch RAM banks → CPU cluster	N/A
smc_l1_icache_tag_intf_req_o	l1_icache_tag_req_t	Output	[NumIcacheTagBanks-1:0]	L1 I-cache tag interface requests. Width depends on CPU cluster configuration	CPU cluster → L1 I-cache tag banks	N/A
smc_l1_icache_tag_intf_rsp_i	l1_icache_tag_rsp_t	Input	[NumIcacheTagBanks-1:0]	L1 I-cache tag interface responses. Width depends on CPU cluster configuration	L1 I-cache tag banks → CPU cluster	N/A

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
smc_l1_icache_data_intf_req_o	l1_icache_data_req_t	Output	[NumIcacheDataBanks-1:0]	L1 I-cache data interface requests. Width depends on CPU cluster configuration	CPU cluster → L1 I-cache data banks	N/A
smc_l1_icache_data_intf_rsp_i	l1_icache_data_rsp_t	Input	[NumIcacheDataBanks-1:0]	L1 I-cache data interface responses. Width depends on CPU cluster configuration	L1 I-cache data banks → CPU cluster	N/A
smc_l1_dcache_tag_intf_req_o	l1_dcache_tag_req_t	Output	[NumDcacheTagBanks-1:0]	L1 D-cache tag interface requests. Width depends on CPU cluster configuration	CPU cluster → L1 D-cache tag banks	N/A
smc_l1_dcache_tag_intf_rsp_i	l1_dcache_tag_rsp_t	Input	[NumDcacheTagBanks-1:0]	L1 D-cache tag interface responses. Width depends on CPU cluster configuration	L1 D-cache tag banks → CPU cluster	N/A
smc_l1_dcache_data_intf_req_o	l1_dcache_data_req_t	Output	[NumDcacheDataBanks-1:0]	L1 D-cache data interface requests. Width depends on CPU cluster configuration	CPU cluster → L1 D-cache data banks	N/A
smc_l1_dcache_data_intf_rsp_i	l1_dcache_data_rsp_t	Input	[NumDcacheDataBanks-1:0]	L1 D-cache data interface responses. Width depends on CPU cluster configuration	L1 D-cache data banks → CPU cluster	N/A
smc_disable_sram_auto_init_i	logic	Input	1	Disable SRAM automatic initialization.	Control logic → CPU cluster	1'b0
smc_init_mem_done_o	logic	Output	1	Memory initialization completion signal.	CPU cluster → Control logic	N/A
chiplet_is_primary_i	logic	Input	1	Chiplet is primary signal. Used for System Timer OCTS	PRIMARY_CHIPLET configuration strap (STRAPS_LO[25])	N/A
timer_count_o	logic	Output	[63:0]	64-bit system timer count output for Open Chiplet Time Synchronization (OCTS).	System Timer OCTS → External systems	N/A
jtag_reset_ctrl_i	smc_pkg::jtag_smc_reset_ctrl_t	Input	-	Typed reset-override control struct carrying all SMC reset override enables and active-low values. Driven by DTP via JTAG IC_RESET. Tie all enables to 0 when not used.	DTP → SMC reset unit	All enables 0
cla_gpio_enable_o	logic	Output	1	CLA GPIO enable output.	DFD controller → CLA	N/A
tdr_dbg_ctrl_clock_stop_en_i	logic	Input	1	DEBUG_CONTROL CLA clock-stop enable.	DTP → DFD	1'b0
tdr_dbg_ctrl_clocks_stopped_by_cla_o	logic	Output	1	CLA halt-clock status under enable; also drives cpu_interrupts_o[NumExtInterrupts+64] (internal interrupt 0), bit 320.	DFD → DTP / interrupt	N/A
xtrigger_ss_i	smc_pkg::xtrigger_t	Input	-	Cross-trigger subsystem input. SMU drives from dtp_xtrig_ctm_src_req[1:0]. Internal SMU connection; not an SMU external port.	SMU internal: DTP xtrig_ctm_src_req_o[1:0] → SMC	N/A
xtrigger_ss_o	smc_pkg::xtrigger_t	Output	-	Cross-trigger subsystem output. SMU routes to dtp_xtrig_ctm_dst_req[1:0]. Internal SMU connection; not an SMU external port.	SMU internal: SMC → DTP xtrig_ctm_dst_req_i[1:0]	N/A
trace_mem_req_o	dfd_trace_mem_pkg::SinkMemPktIn_s	Output	[TRC_RAM_INSTANCES-1:0]	Trace memory interface requests.	DFD controller → Trace memory	N/A, leave floating if unused
trace_mem_resp_i	dfd_trace_mem_pkg::SinkMemPktOut_s	Input	[TRC_RAM_INSTANCES-1:0]	Trace memory interface responses.	Trace memory → DFD controller	N/A, tie to '0 if unused
ext_debug_bus_i	logic	Input	[511:0]	External debug bus input (512 bits, 16-bit aligned).	External debug sources → Debug bus aggregator	N/A, tie to '0 if unused
test_en_i	logic	Input	1	DFT/scan test-enable. Drives clock-gater test ports and AXI cell test inputs.	Test controller → All modules	1'b0
scan_rst_ni	logic	Input	1	Scan reset (active-low). Bypasses reset synchronizers during DFT.	Test controller → All modules	1'b1

8.13. GPIO Pin Requirements

The SMC provides GPIO functionality with multiplexed peripheral functions and test interfaces. The following table lists each pad's index, assigned function, recommended pull, and GPIO-specific notes. It does not define default direction or decode configuration straps; use `smc_padring_pkg::DefaultDirectionMap` and the `STRAPS_LO` and `STRAPS_HI` register descriptions (see Mandatory Region) for those.

Table 8. OCAH GPIO Requirements

Index	Function	Recommended Pull	GPIO Notes
0	SPI.DATA[0]	PD	
1	SPI.DATA[1]	PD	
2	SPI.DATA[2]	PD	
3	SPI.DATA[3]	PD	
4	SPI.DATA[4]	PD	
5	SPI.DATA[5]	PD	
6	SPI.DATA[6]	PD	
7	SPI.DATA[7]	PD	
8	SPI.CS	PU	
9	SPI.CLK	PD	
10	SPI.DQS	PD	
11	UART[0].RX	PU	
12	UART[0].TX	PU	
13	UART[0].RTS	PD	
14	UART[0].CTS	PD	
15	UART[1].RX	PU	
16	UART[1].TX	PU	
17	UART[1].RTS	PD	
18	UART[1].CTS	PD	
19	UART[2].RX	PU	
20	UART[2].TX	PU	
21	UART[2].RTS	PD	
22	UART[2].CTS	PD	
23	UART[3].RX	PU	
24	UART[3].TX	PU	
25	UART[3].RTS	PD	
26	UART[3].CTS	PD	
27	I3C[0] SCL	PU	
28	I3C[0] SDA	PU	
29	I3C[2] SCL	PU	
30	I3C[2] SDA	PU	
31	I3C[3] SCL	PU	
32	I3C[3] SDA	PU	

Index	Function	Recommended Pull	GPIO Notes
33	I3C[4] SCL	PU	
34	I3C[4] SDA	PU	
35	I3C[5] SCL	PU	
36	I3C[5] SDA	PU	
37	I2C[0] SCL	PU	Every I2C can be programmed to act as I2C, SMBus or PMBus
38	I2C[0] SDA	PU	
39	I2C[0] SMB_A	PU	SMBus Alert
40	I2C[0] SMB_D	PU	SMBus Suspend
41	I2C[1] SCL	PU	
42	I2C[1] SDA	PU	
43	I2C[1] SMB_A	PU	SMBus Alert
44	I2C[1] SMB_D	PU	SMBus Suspend
45	I2C[2] SCL	PU	
46	I2C[2] SDA	PU	
47	I2C[2] SMB_A	PU	SMBus Alert
48	I2C[2] SMB_D	PU	SMBus Suspend
49	AVS.CLOCK	PU	
50	AVS.MDATA	PU	
51	AVS.SDATA	PU	
52	Thermal Emergency N	PU	
53	Isolate Request	PD	
54	SPI DQS Loopback	PD	
55	System Timer Sync Load	PD	
56	System Timer Credit Count	PD	
57	Boot Stall	PD	Stall after fuse sensing to override shadow register values. Boot will resume when this GPIO is released. This value can be overridden by the value specified over JTAG using the boot_stall_jtag_ovrd and boot_stall_jtag_val signals from DTP.
58	OCCP Interface	PD	
59	Reserved	PD	
60	Reserved	PD	
61	Cool Reset In	PU	Not packaged out
62	Cool Reset Out	PU	Not packaged out
63	I3C[1] SCL	PU	Not packaged out
64	I3C[1] SDA	PU	Not packaged out
65	PTAP TCK		No dedicated interface in harness
66	PTAP TRST		No dedicated interface in harness
67	PTAP TDI		No dedicated interface in harness
68	PTAP TMS		No dedicated interface in harness
69	PTAP TDO		No dedicated interface in harness
70	STAP TCK		No dedicated interface in harness

Index	Function	Recommended Pull	GPIO Notes
71	STAP TRST		No dedicated interface in harness
72	STAP TDI		No dedicated interface in harness
73	STAP TMS		No dedicated interface in harness
74	STAP TDO		No dedicated interface in harness
75	BP_REFCLK_VDD_SYS		No dedicated interface in harness
76	BP_RESETN		No dedicated interface in harness
77	BP_POWERGOOD		No dedicated interface in harness

For multi-chiplet systems, the `PRIMARY_CHIPLET` strap (`STRAPS_LO[25]`) determines whether the chiplet operates as primary or secondary. The primary chiplet's System Timer OCTS (Open Chiplet Time Synchronization) generates synchronization pulses (`timer_sync_load_o`, `timer_cnt_credit_o`) that secondary chiplets receive to maintain coherent timestamps across the system. The `chiplet_is_primary_i` input carries this strap and should be held constant while the SMC is out of cold reset.

Chapter 9. Debug & Test Ports (DTP) Integration

9.1. Configuration

The SMU `smu_cfg_t` fields `XTRIG_NUM_CTP`, `XTRIG_NUM_INT_CT`, and `XTRIG_NUM_CLK_STOP_REQ` size the SMU-exposed cross-trigger ports and are passed through to the DTP (with the SMC-reserved lanes added to the internal-CT and clock-stop counts). The generated Cross Trigger Matrix register map must have one `CT_Src` and one `CT_Dst` for every DTP port (`XTRIG_NUM_CTP + XTRIG_NUM_INT_CT` at the DTP, which is the SMU-exposed internal-CT count plus `smu_pkg::XtrigSmcIntCtLanes`). Resizing is two steps: set the `CFG` counts, and regenerate the CTM with matching PeakRDL overrides:

```
make ocah-regen-regs TARGET=cross_trigger_matrix \
    OCAH_REG_RDL_PARAMS_cross_trigger_matrix="NUM_CT_SRC=<n> NUM_CT_DST=<n>"
```

`<n>` is the DTP total (`CFG.XTRIG_NUM_CTP + CFG.XTRIG_NUM_INT_CT` `smu_pkg::XtrigSmcIntCtLanes`). `cross_trigger_network` fails elaboration if the parameterized total disagrees with the generated map. Defaults remain 16 external CTPs, 10 internal CTs (8 exposed), and 9 clock-stop requests (8 exposed). `XTRIG_INT_CT_MODE` and every `JTAG_*` enable are set at the instantiation as shown below.

Include the DTP in your build using Bender target `dtp`, which pulls in the DTP RTL together with the JTAG and cross-trigger IPs it instantiates. The legacy filelist `hw/sys/dtp/rtl/config/dtp.f` covers only the JTAG portion of the hierarchy and is not a complete DTP recipe.

9.2. Instantiation

```
dtp #(
    // JTAG Configuration
    .JTAG_BSR_ENABLE          (1'b1),
    .JTAG_TMP_ENABLE          (1'b1),
    .JTAG_SMC_DBG_ENABLE      (1'b1),
    .JTAG_SEP_DBG_ENABLE      (1'b1),
    .JTAG_IDCODE_MFR_ID       (11'h15A),
    .JTAG_IDCODE_PART_NUM     (16'h1234),
    .JTAG_IDCODE_SI_REV       (4'h1),
    // Cross Trigger Configuration
    .XTRIG_NUM_CTP            (16),
    .XTRIG_NUM_INT_CT         (10),
    .XTRIG_NUM_CLK_STOP_REQ   (9),
    .XTRIG_INT_CT_MODE        (10'b0000000000)
) u_dtp (
    .clk_i                    (clk),
    .rst_n_i                  (rst_n),
    .pwr_on_rst_ni            (pwr_on_rst_n),
```

```
    // JTAG, Boundary Scan, Cross Trigger interfaces...
);
```

9.3. Clock and Reset

- **TCK Domain**: JTAG logic operates on external Test Clock
- **System Clock** (`clk_i`): Cross trigger logic and JTAG2AXI bridges
- **System Reset** (`rst_n_i`): Resets cross trigger and AXI; does not affect TAP controller
- **Power-On Reset** (`pwr_on_rst_ni`): Asynchronous reset for JTAG state machine

9.4. Constraints

- JTAG interface: 10-50 MHz TCK frequency
- Cross trigger signals synchronized to system clock
- DTP remains active during clock stop for debug access
- External CTP signals require pad ring integration with ESD protection

9.5. STAP Scan Chain Topology

The DTP secondary scan path chains multiple STAPs on a single PTAP-managed scan string. The fixed order on this string is: PTAP, I/O STAP (if `JTAG_STAP_IO_ENABLE=1`), SMC debug STAP (if `JTAG_SMC_DBG_ENABLE=1`), SEP debug STAP (if `CFG.SEP=1` in the SMU), extra STAP[N-1:0], and return to PTAP.

The string carries scans only while the PTAP 3DCR select bit is set. While the bit is clear, the scan-control strobes to every STAP stay low, so the STAP SIBs and 3DCRs hold their state through every PTAP scan, as does the downstream TAP of every deselected STAP. A STAP whose own select bit is set still forwards TCK and the live TMS, so its downstream TAP follows every PTAP scan; clear each STAP's select bit, or reset the chain, before clearing the PTAP select bit. The select bit updates at Update-DR, so it is set in a TAP_3DCR scan of its own before a scan that opens STAP SIBs or writes STAP 3DCRs.

Both the SMC and SEP debug STAP host interfaces are internal to the SMU and are not exported on the SMU boundary. The SMU wires the SMC debug STAP host outputs to the SMC CPU/system JTAG ports (`smc_cpu_jtag_*`) when `JTAG_SMC_DBG_ENABLE=1`; when disabled, the DTP bypasses this STAP in the secondary scan chain and drives the internal host outputs low. The SEP debug STAP connects to the SEP JTAG debug port when `CFG.SEP=1`; when `CFG.SEP=0`, the SMU ties `sep_stap_tdo_to_dtp = 1'b0` to bypass the SEP STAP return path.

Each STAP host interface is gated by a field of `dbg_disable_i`, synchronized to TCK inside the DTP. Each field is active-high (1 = path disabled).

STAP	Disable Field
I/O STAP	`dbg_disable_i.stap_io`
SMC debug STAP	`dbg_disable_i.stap_smc`
SEP debug STAP	`dbg_disable_i.stap_sep`
Extra STAPs	`dbg_disable_i.stap_extra`
External STAP return path	`dbg_disable_i.stap_host`

The full set of debug-disable-gated paths, including the iJTAG chains and the JTAG2AXI bridges, is tabulated in the DTP JTAG integration section of the TRM.

9.6. DTP Port Interface

The DTP exposes interfaces for JTAG TAP control, boundary scan, multiple STAP host ports, iJTAG scan chains, AXI debug bridges, cross trigger networks, and CTP GPIO pads. The following table defines all DTP top-level ports. Refer to the dedicated DTP documentation in `hw/sys/dtp/doc/` for internal design details.

Legend: Where no Default entry is shown in Description / Default, the original port reference specifies N/A — those ports have no integrator-supplied default; connect as directed by Description.

DTP Port Declaration lists the DTP boundary ports, their dimensions, and their connections.

Table 9. DTP Port Declaration

Interface Name	Type	Dir	Width	Description / Default	Connection
`clk_i`	`logic`	In	1	System clock for cross trigger logic and JTAG2AXI bridges.	System clock
`rst_n_i`	`logic`	In	1	System reset (active low). Resets cross trigger and AXI logic; does not affect the TAP controller.	System reset
`pwr_on_rst_ni`	`logic`	In	1	Power-on reset for JTAG state machine (active low, asynchronous).	Power-on reset
`dbg_disable_i`	`sep_lifecycle_ctrl_pkg ::dbg_disable_t`	In	-	Per-path lifecycle controls for disabling debug and test interfaces. Active-high (1 = path disabled). DTP synchronizes each field into the JTAG TCK domain.	SEP lifecycle controller or adopter-equivalent policy source
`jtag_ptap_client_tap_ctrl_i`	`jtag_tap_ctrl_t`	In	-	Primary JTAG TAP control (TCK, TMS, TRST_N).	External JTAG pads
`jtag_ptap_client_tdi_i`	`logic`	In	1	Primary JTAG TDI.	External JTAG pads
`jtag_ptap_client_tdo_o`	`logic`	Out	1	Primary JTAG TDO.	External JTAG pads
`jtag_ptap_client_tdo_oen_o`	`logic`	Out	1	Primary JTAG TDO output enable.	External JTAG pads
`jtag_bsr_host_scan_ctrl_o`	`jtag_scan_ctrl_t`	Out	-	Boundary scan control. Tie scan_in to scan_out if BSR is disabled.	Boundary scan chain
`jtag_bsr_host_scan_in_i`	`logic`	In	1	Boundary scan data in.	Boundary scan chain
`jtag_bsr_host_scan_out_o`	`logic`	Out	1	Boundary scan data out.	Boundary scan chain
`jtag_stap_io_host_tap_ctrl_o`	`jtag_tap_ctrl_t`	Out	-	I/O STAP TAP control for chiplet-to-chiplet connectivity. If `JTAG_STAP_IO_ENABLE=0`, only `.tck` is driven; `.tms` is 0 and `.trst_n` is 0, so TRST is asserted and a TAP connected to the port is held in Test-Logic-Reset.	I/O STAP host
`jtag_stap_io_host_tdi_i`	`logic`	In	1	I/O STAP TDI. Default: Tie to '0 if unused.	I/O STAP host
`jtag_stap_io_host_tdo_o`	`logic`	Out	1	I/O STAP TDO.	I/O STAP host
`jtag_stap_io_host_tdo_oen_o`	`logic`	Out	1	I/O STAP TDO output enable.	I/O STAP host

Interface Name	Type	Dir	Width	Description / Default	Connection
jtag_stap_smc_host_tap_ctrl_o	jtag_tap_ctrl_t	Out	-	SMC debug STAP TAP control. If JTAG_SMC_DBG_ENABLE=0, only .tck is driven; .tms is 0 and .trst_n is 0, so TRST is asserted and a TAP connected to the port is held in Test-Logic-Reset.	SMC JTAG port
jtag_stap_smc_host_tdi_i	logic	In	1	SMC debug STAP TDI (from SMC TDO). Default: Tie to '0 if unused.	SMC JTAG port
jtag_stap_smc_host_tdo_o	logic	Out	1	SMC debug STAP TDO (to SMC TDI).	SMC JTAG port
jtag_stap_smc_host_tdo_oen_o	logic	Out	1	SMC debug STAP TDO output enable.	SMC JTAG port
jtag_stap_sep_host_tap_ctrl_o	jtag_tap_ctrl_t	Out	-	SEP debug STAP TAP control. If JTAG_SEP_DBG_ENABLE=0, only .tck is driven; .tms is 0 and .trst_n is 0, so TRST is asserted and a TAP connected to the port is held in Test-Logic-Reset.	SEP JTAG port
jtag_stap_sep_host_tdi_i	logic	In	1	SEP debug STAP TDI (from SEP TDO). Default: Tie to '0 if unused.	SEP JTAG port
jtag_stap_sep_host_tdo_o	logic	Out	1	SEP debug STAP TDO (to SEP TDI).	SEP JTAG port
jtag_stap_sep_host_tdo_oen_o	logic	Out	1	SEP debug STAP TDO output enable.	SEP JTAG port
jtag_stap_extra_host_tap_ctrl_o	jtag_tap_ctrl_t	Out	[JtagNumExtraStapPorts-1:0]	Extra STAP TAP control array. If JTAG_NUM_EXTRA_STAPS=0, the single padded element drives only .tck; its .tms is 0 and its .trst_n is 0, so TRST is asserted and a TAP connected to it is held in Test-Logic-Reset.	Extra STAP hosts
jtag_stap_extra_host_tdi_i	logic	In	[JtagNumExtraStapPorts-1:0]	Extra STAP TDI array. Default: Tie to '0 if unused.	Extra STAP hosts
jtag_stap_extra_host_tdo_o	logic	Out	[JtagNumExtraStapPorts-1:0]	Extra STAP TDO array.	Extra STAP hosts
jtag_stap_extra_host_tdo_oen_o	logic	Out	[JtagNumExtraStapPorts-1:0]	Extra STAP TDO output enable array.	Extra STAP hosts
jtag_stap_host_scan_ctrl_o	jtag_scan_ctrl_t	Out	-	Extended STAP scan control. Tie scan_in to scan_out if unused.	STAP scan chain
jtag_stap_host_scan_in_i	logic	In	1	Extended STAP scan data in.	STAP scan chain
jtag_stap_host_scan_out_o	logic	Out	1	Extended STAP scan data out.	STAP scan chain
jtag_dfd_host_scan_ctrl_o	jtag_scan_ctrl_t	Out	-	DFD iJTAG scan control. Tie scan_in to scan_out if unused.	DFD iJTAG chain
jtag_dfd_host_scan_in_i	logic	In	1	DFD iJTAG scan data in.	DFD iJTAG chain
jtag_dfd_host_scan_out_o	logic	Out	1	DFD iJTAG scan data out.	DFD iJTAG chain
jtag_dft_secure_host_scan_ctrl_o	jtag_scan_ctrl_t	Out	-	Secure DFT iJTAG scan control. Tie scan_in to scan_out if unused.	Secure DFT iJTAG chain
jtag_dft_secure_host_scan_in_i	logic	In	1	Secure DFT iJTAG scan data in.	Secure DFT iJTAG chain
jtag_dft_secure_host_scan_out_o	logic	Out	1	Secure DFT iJTAG scan data out.	Secure DFT iJTAG chain
jtag_dft_host_scan_ctrl_o	jtag_scan_ctrl_t	Out	-	Non-secure DFT iJTAG scan control. Tie scan_in to scan_out if unused.	Non-secure DFT iJTAG chain
jtag_dft_host_scan_in_i	logic	In	1	Non-secure DFT iJTAG scan data in.	Non-secure DFT iJTAG chain
jtag_dft_host_scan_out_o	logic	Out	1	Non-secure DFT iJTAG scan data out.	Non-secure DFT iJTAG chain

Interface Name	Type	Dir	Width	Description / Default	Connection
axi_smc_dbg_req_o	smc_jtag_axi_req_t	Out	-	SMC fabric debug AXI request. Active when JTAG_SMC_DBG_ENABLE=1. Default: tie to '0 if disabled.	SMC jtag_axi_in_req_i
axi_smc_dbg_resp_i	smc_jtag_axi_resp_t	In	-	SMC fabric debug AXI response. Default: tie to '0 if disabled.	SMC jtag_axi_in_resp_o
axil_smc_otp_jtag_req_o	smc_otp_axil_req_t	Out	-	SMC OTP debug AXI-Lite request. Default: tie to '0 if disabled.	SMC axil_smc_otp_jtag_req_i
axil_smc_otp_jtag_resp_i	smc_otp_axil_resp_t	In	-	SMC OTP debug AXI-Lite response. Default: tie to '0 if disabled.	SMC axil_smc_otp_jtag_resp_o
axil_sep_otp_jtag_req_o	sep_otp_axil_req_t	Out	-	SEP OTP debug AXI-Lite request. Default: tie to '0 if disabled.	SEP axil_sep_otp_jtag_req_i
axil_sep_otp_jtag_resp_i	sep_otp_axil_resp_t	In	-	SEP OTP debug AXI-Lite response. Default: tie to '0 if disabled.	SEP axil_sep_otp_jtag_resp_o
stop_clks_o	logic	Out	1	Clock stop output from JTAG DEBUG_CONTROL or CLA clock-stop requests via CTN.	PLL clock gates
cla_clock_stop_en_o	logic	Out	1	CLA clock stop enable from DEBUG_CONTROL register.	Clock control logic
jtag_boot_stall_ovrd_o	logic	Out	1	JTAG boot stall override enable.	Boot control
jtag_boot_stall_o	logic	Out	1	JTAG boot stall value when override is asserted.	Boot control
jtag_ic_reset_smc_o	ic_reset_smc_t	Out	$bits(ic_reset_smc_t)	SMC IC_RESET override slice. Struct contains .ovrd (override enables) and .val (active-low override values), each sized to match the corresponding SMC reset control struct. Tied to '0 when JTAG_IC_RESET_SMC_ENABLE=0.	SMC jtag_smc_reset_ctrl_i
jtag_ic_reset_sep_o	ic_reset_sep_t	Out	$bits(ic_reset_sep_t)	SEP IC_RESET override slice. Struct contains .ovrd (override enables) and .val (active-low override values), each sized to match the corresponding SEP reset control struct. Tied to '0 when JTAG_IC_RESET_SEP_ENABLE=0.	SEP jtag_sep_reset_ctrl_i
jtag_ic_reset_ext_o	ic_reset_ext_t	Out	$bits(ic_reset_ext_t)	External IC_RESET override slice. Struct contains .ovrd (override enables) and .val (active-low override values), each sized per integrator-defined type. Tied to '0 when JTAG_IC_RESET_EXT_ENABLE=0.	Integrator-defined reset overrides for logic outside the SMU
jtag_ptap_state_o	tap_state_e	Out	-	Current PTAP TAP state.	Debug infrastructure
jtag_ptap_inst_decoded_o	jtag_instruction_decoded_e	Out	-	Current decoded PTAP instruction.	Debug infrastructure
axil_xtrig_req_i	xtrig_axil_req_t	In	-	Cross trigger CSR AXI-Lite request from SMC.	SMC axil_dtp_csr_req_o
axil_xtrig_resp_o	xtrig_axil_resp_t	Out	-	Cross trigger CSR AXI-Lite response to SMC.	SMC axil_dtp_csr_resp_i
xtrig_ctm_src_req_o	logic	Out	[XTRIG_NUM_INT_CT-1:0]	Cross trigger source requests from CTM to destination sinks.	Cross trigger sinks
xtrig_ctm_src_ack_i	logic	In	[XTRIG_NUM_INT_CT-1:0]	Cross trigger source acknowledges. Unused in pulse-sync mode (mode bit = 0). Default: tie to '0 in pulse-sync mode.	Cross trigger sinks

Interface Name	Type	Dir	Width	Description / Default	Connection
xtrig_ctm_dst_req_i	logic	In	[XTRIG_NUM_INT_CT-1:0]	Cross trigger destination requests from source generators. Default: tie to '0 if unused.	Cross trigger sources
xtrig_ctm_dst_ack_o	logic	Out	[XTRIG_NUM_INT_CT-1:0]	Cross trigger destination acknowledges. Unused in pulse-sync mode (mode bit = 0).	Cross trigger sources
xtrig_clk_stop_req_i	logic	In	[XTRIG_NUM_CLK_STOP_REQ-1:0]	Clock stop requests from CLAs or other devices in the chiplet. Default: Tie to '0 if unused.	Clock stop requesters
xtrig_ctp_req_out_dout_o	logic	Out	[XTRIG_NUM_CTP-1:0]	CTP request-out data output (to pad ring).	GPIO pad ring
xtrig_ctp_req_out_dout_en_o	logic	Out	[XTRIG_NUM_CTP-1:0]	CTP request-out output enable.	GPIO pad ring
xtrig_ctp_req_out_din_i	logic	In	[XTRIG_NUM_CTP-1:0]	CTP request-out loopback input. Default: tie to '0 if unused.	GPIO pad ring
xtrig_ctp_req_out_din_en_o	logic	Out	[XTRIG_NUM_CTP-1:0]	CTP request-out input enable.	GPIO pad ring
xtrig_ctp_req_in_dout_o	logic	Out	[XTRIG_NUM_CTP-1:0]	CTP request-in data output.	GPIO pad ring
xtrig_ctp_req_in_dout_en_o	logic	Out	[XTRIG_NUM_CTP-1:0]	CTP request-in output enable.	GPIO pad ring
xtrig_ctp_req_in_din_i	logic	In	[XTRIG_NUM_CTP-1:0]	CTP request-in data input. Default: tie to '0 if unused.	GPIO pad ring
xtrig_ctp_req_in_din_en_o	logic	Out	[XTRIG_NUM_CTP-1:0]	CTP request-in input enable.	GPIO pad ring
xtrig_ctp_ack_in_dout_o	logic	Out	[XTRIG_NUM_CTP-1:0]	CTP ack-in data output.	GPIO pad ring
xtrig_ctp_ack_in_dout_en_o	logic	Out	[XTRIG_NUM_CTP-1:0]	CTP ack-in output enable.	GPIO pad ring
xtrig_ctp_ack_in_din_i	logic	In	[XTRIG_NUM_CTP-1:0]	CTP ack-in data input. Default: tie to '0 if unused.	GPIO pad ring
xtrig_ctp_ack_in_din_en_o	logic	Out	[XTRIG_NUM_CTP-1:0]	CTP ack-in input enable.	GPIO pad ring
xtrig_ctp_ack_out_dout_o	logic	Out	[XTRIG_NUM_CTP-1:0]	CTP ack-out data output.	GPIO pad ring
xtrig_ctp_ack_out_dout_en_o	logic	Out	[XTRIG_NUM_CTP-1:0]	CTP ack-out output enable.	GPIO pad ring
xtrig_ctp_ack_out_din_i	logic	In	[XTRIG_NUM_CTP-1:0]	CTP ack-out data input. Default: tie to '0 if unused.	GPIO pad ring
xtrig_ctp_ack_out_din_en_o	logic	Out	[XTRIG_NUM_CTP-1:0]	CTP ack-out input enable.	GPIO pad ring
test_en_i	logic	In	1	DFT test-mode enable (active high) for the JTAG2AXI bridges and the cross trigger CSR crossbar. Default: 0 in functional mode.	DFT controller
scan_rst_ni	logic	In	1	DFT scan reset (active low). Unused: DTP has no reset synchronizer, so rst_n_i must arrive scan-controlled.	DFT controller, or tie high

Chapter 10. Security Processor (SEP) Integration

The Security Processor (SEP) is the Root of Trust for the chiplet, providing a physically isolated execution environment for security-critical services such as secure boot, lifecycle control, device attestation, cryptographic acceleration, and key management. This section provides practical integration procedures for adopters integrating the SEP as part of the SMU or as a standalone out-of-context IP.

For detailed architectural documentation about the SEP internal design, refer to the dedicated documentation located in the SEP reference manual. The SEP hardware overview and interconnect fabric are described in the SEP *Overview* and *Interconnect* sections.

10.1. Module Overview

The module in `sep.sv` provides the complete SEP functionality with logic-level interfaces. Technology-specific components — SRAM, ROM, and eFuse shims — are instantiated by the adopter outside SEP, following the same vendor-neutral philosophy used for the SMC.

Separately, SEP exposes interfaces for supplementary peripherals an adopter may add on rather than substitute: an external TRNG and an additional SPI controller. These are not technology-specific — SEP contains its own entropy source and SPI host — so the interfaces exist to supplement what SEP already provides, and may be left unused.

When integrated inside the SMU, the SEP's external interfaces are passed through the SMU boundary with a `sep_` prefix.

10.2. Clock and Reset Requirements

The SEP requires two clock inputs. Connect `clk_i` to the primary system clock driving all SEP internal logic. Connect `clk_wdt_i` to a 200 kHz clock source for the watchdog timer; this clock must be independent of the system clock so that the WDT can detect system clock failures.

The primary reset `rst_ni` follows the standard active-low, asynchronous assertion / synchronous deassertion convention. The `dbg_rstb_i` input provides a separate reset for the EL2 CPU debug module. The `wdt_rst_ni` input receives the watchdog reset generated by the SMC reset unit. The SEP outputs `wdt_timer_rst_req_o` when the WDT triggers a reset request.

10.3. AXI Interfaces

The SEP connects to the system through three AXI paths. The `smn_outbound_axi` master interface allows SEP firmware to access system resources outside the SEP domain, such as SMC registers or external memory. The `smn_inbound_axi` slave interface accepts requests from system masters requiring access to SEP-managed resources. System masters must access SEP register regions with single-beat transactions. The `sep_ext_to_smc_axi` output provides a dedicated AXI path for SEP-to-SMC transactions, connecting to the SMC's `sep_axi_in` port.

An additional AXI extension port (`axi_extension_axi`) enables adopters to attach custom peripherals to the SEP interconnect fabric without modifying the SEP core. When unused, tie the response input to return DECERR.

10.4. Memory Interfaces

All SEP memory macros are instantiated externally and connected through dedicated request/response interfaces. The `sep_cpu_tcm` interface provides tightly-coupled memory (ICCM and DCCM) for the VeeR EL2 CPU. The `sep_sram` interface connects scratch SRAM used for firmware data. The `sep_boot_rom` interface provides the BL0 ROM containing the first-stage bootloader.

The crypto subsystem requires additional memory interfaces. `sep_crypto_pka_imem_sram` and `sep_crypto_pka_dmem_sram` connect to the OTBN public key accelerator instruction and data memories respectively. `km_rom_mem` and `km_sram_mem` connect to the Key Manager's dedicated ROM and SRAM macros. The KM SRAM must assert `rvalid` at least one cycle after the cycle in which it grants a read; a registered-read macro meets this.

`abr_mem` connects the Adams Bridge working SRAM. On an idle cycle (`re` low), ABR expects the channel's read data to return 0 the following cycle. `SEP_ABR_SRAM_LATENCY` must equal the macros' native registered-read latency (default 1). Word-write channels are full-word `we` / `re` / address / data. `sig_z` and `pk` carry one-bit-per-byte `wstrobe`; map those to the macros' byte-write enables. `SEP_ABR_MASKING_EN` (default 1) selects 2-share DOM: thirteen 1R1W macros versus nine, plus a second NTT. This changes ABR internal behavior (sequencing, SCA, memory topology), not merely throughput. Give the same value to SEP and to the SRAM instances. When it is 0, do not instantiate the four masked coefficient banks; their response fields read as 0.

10.5. Cryptographic Subsystem Connections

The SEP includes an internal entropy source that requires a ring-oscillator sample clock connected to `entropy_rosc_sample_clk_i`. This clock must be asynchronous to `clk_i` and typically runs between 100 and 400 MHz.

10.5.1. External TRNG

For adopters providing an external TRNG, the `ext_trng_axil` AXI-Lite interface connects to the external TRNG's control registers, while the `ext_trng_axis` AXI-Stream interfaces carry entropy data. The `ext_trng_irq_i` input is routed to the PIC.

SEP decodes the 4 kB TRNG aperture (`0x1091_7000` - `0x1091_7FFF`) as a single window and forwards every access on `ext_trng_axil`. It does not decode offsets inside that window: the endpoint is an adopter-defined IP.

The adopter must perform the intra-aperture decode. An address that owns no register must return an error response and must not alias onto a live register. When no external TRNG is connected, terminate `ext_trng_axil` with a DECERR slave.

The TRNG path reaches `ext_trng_axil` through an AXI4-to-AXI-Lite conversion (64-to-32 downsizer and burst splitter). An errored read returns the slave's response verbatim. A single-beat write does too, so a DECERR slave answers both an errored read and an errored write with DECERR. A write the splitter breaks into more than one beat is reported as SLVERR when an earlier fragment erred and the last fragment is OKAY. An adopter whose endpoint returns DECERR for an unowned offset sees that code on a single-beat access.

10.6. eFuse and Lifecycle

SEP has its own eFuse bank. Instantiate a separate SHIM and physical eFuse bank; do not share the SMC's. The SHIM contract is the eFuse Interface. On `sep.sv`, connect `efuse_bank_ctrl_*` and `efuse_shim_command_*`. Through the SMU those ports appear as `sep_efuse_bank_ctrl_*` and `sep_efuse_shim_command_*`. The SEP SHIM control CSRs sit at the base of `sep_external`; see the SHIM control address window. SEP routes a burst on its first address, so a burst that begins in that

window carries its later beats to the SHIM. An address that owns no SHIM register must return SLVERR and must not alias onto a live register.

SEP senses this bank after its own reset. Watch `sep_fuse_sense_done_o`. On `sep.sv`, drive `smc_fuse_sense_done_i` from the SMC so firmware can see when the SMC fabric is out of reset; the SMU wires that input internally.

Lifecycle and test-mode ports on `sep.sv`, and what to do with them. Behaviour is in the Technical Reference Manual, not here.

- `secure_tm_req_i` signals a request to enter secure test mode. SEP latches it into `secure_tm_o` when fuse sensing completes, or at cold-reset release if SEC_DIS is already asserted. Secure test mode forces a no-secret, no-fuse-program configuration; it is not an authorization bit. See *Test Mode Entry* and *`secure_tm` and the Secure Scan Chain* in the TRM. `secure_tm_o` is also an SMU output.
- `security_disable_o` is the SEC_DIS token-match result (emergency A0 unlock). On `sep.sv`, route it to SMC `sep_security_disable_i`; the SMU does that internally. See *Security Disable* in the TRM.
- `lc_state_o` is the differentially encoded LC_STATE shadow. `lc_sigint_err_o` is the decode integrity error (fail-closed). Both are SMU outputs. See *Life Cycle Controller* in the TRM.
- `dbg_disable_o` is the typed per-path debug/test disable vector (active-high: 1 = path disabled). On `sep.sv`, route it to DTP `dbg_disable_i`; the SMU does that internally.
- `lcc_demote_state_1_o` / `lcc_demote_state_2_o` are differentially encoded in-field debug demotion. They leave the SMU for adopter lifecycle logic and are not connected to the SMC. See *in-field debug demotion* in the TRM.
- `sep_fuse_dft_disable_o` / `smc_fuse_dft_disable_o` gate DFT-inserted OTP access. Leave them open if no such path is inserted. They are SMU outputs.

OTP debug AXI-Lite (`axil_sep_otp_jtag`) is connected to the DTP inside the SMU; when `CFG.SEP=0` the SMU instantiates an AXI-Lite error slave on that path.

10.7. SPI Host Interface

The SEP provides a SPI host controller through the `sep_io_spi` interface, supporting standard, dual and quad modes. The controller's own interrupt leaves the module as the `irq` field of `sep_io_spi_req_o`, so no separate interrupt output is needed.

10.7.1. SEP SPI Host Pin Assignment

When the SMU is built with the SEP, the SMU drives the SMC SPI port (`spi_*_i` of `smc.sv`) from `sep_io_spi_req_o` and returns the data-pad inputs to the SEP in `sep_io_spi_rsp_i.sd`. The SMC places these signals on GPIO 0-10 and 54. Inside `smc.sv`, no register selects another SPI source for these pads.

GPIO	Function	SEP SPI host source
0-3	SPI.DATA[0-3]	Lane n drives `sd[n]` with output enable `sd_oe[n]`; the pad input returns as `sd[n]`.
4-7	SPI.DATA[4-7]	Not driven. Output disabled, input disabled.
8	SPI.CS	`cs_n`, output enable `cs_oe`.
9	SPI.CLK	`sck`, output enable `sck_oe`.
10	SPI.DQS	Not driven. Output disabled, input disabled.
54	SPI DQS Loopback	Not driven. Output disabled, input disabled.

In standard mode, SPI.DATA[0] is the host output (COPI) and SPI.DATA[1] is the host input (CIPO).

The SMC holds the SPI function select of these pads on whenever the SMU has the SEP. A pad follows the SEP SPI host once it is out of cold reset and while its GPIO `DATA_CTRL.lsio_disable` is clear; setting `lsio_disable` returns the pad to register control. A pad shim outside `smc.sv` can also hand a pad to a second function with its `CONTROL.hw2_ovrd`.

On SPI.CS, SPI.CLK and SPI.DATA[0-3], the pad input is enabled only while the host does not drive that pad. The host drives a data lane only while chip select is asserted, and drives no pad until software sets its `CONTROL.OUTPUT_EN`. Until then SPI.CS is undriven, so its level comes from the pad pull; the GPIO pin table recommends a pull-up.

10.8. Bring-Up Considerations

The SEP boot sequence begins after the SMC completes fuse sensing and releases the SEP reset. The SEP BL0 ROM FW verifies the BL1 software image using non-volatile public keys before transferring control. Feature controls derived from secure boot authentication and lifecycle state govern what operations are permitted after boot.

The `jtag_id_i` input must be tied to an appropriate constant in the top-level integration. A `rst_vec_i` port exists on `tb_top.sv` for DV reset vector injection; it is not present on `sep.sv` itself and requires no integration tie-off.

10.9. SEP Port Interface

The SEP exposes interfaces for clocks, resets, JTAG debug, CPU control, memory macros, AXI bus connections, cryptographic subsystem, eFuse, lifecycle, SPI host, and mailbox. The following table defines all SEP top-level ports.

Table 10. SEP Port Declaration

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
`clk_i`	`logic`	Input	1	Primary SEP clock. All SEP internal logic operates synchronously to this clock.	System clock	N/A
`clk_wdt_i`	`logic`	Input	1	200 kHz clock for watchdog timer.	Low-frequency clock source	N/A
`rst_ni`	`logic`	Input	1	Primary reset (active low). Asynchronous assertion, synchronous deassertion.	System reset	N/A
`dbg_rstb_i`	`logic`	Input	1	EL2 debugger reset (active low).	JTAG debug infrastructure	N/A
`wdt_rst_ni`	`logic`	Input	1	Watchdog timer reset from SMC (active low).	SMC reset unit	N/A
`wdt_timer_rst_req_o`	`logic`	Output	1	WDT reset request output. Triggers memory interface reset in `sep_wrapper`.	SEP wrapper	N/A
`jtag_tck_i`	`logic`	Input	1	JTAG test clock.	JTAG pads (via DTP STAP)	N/A
`jtag_tms_i`	`logic`	Input	1	JTAG test mode select.	JTAG pads (via DTP STAP)	N/A
`jtag_tdi_i`	`logic`	Input	1	JTAG test data in.	JTAG pads (via DTP STAP)	N/A
`jtag_trst_ni`	`logic`	Input	1	JTAG test reset (active low).	JTAG pads (via DTP STAP)	N/A
`jtag_tdo_o`	`logic`	Output	1	JTAG test data out.	JTAG pads (via DTP STAP)	N/A
`jtag_tdoEn_o`	`logic`	Output	1	JTAG test data out enable.	JTAG pads (via DTP STAP)	N/A

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
axil_sep_otp_jtag_req_i	sep_efuse_pkg::efuse_axil_req_t	Input	-	OTP debug AXI-Lite request from JTAG.	DTP axil_sep_otp_jtag_req_o	Tie to '0 if unused
axil_sep_otp_jtag_resp_o	sep_efuse_pkg::efuse_axil_resp_t	Output	-	OTP debug AXI-Lite response to JTAG.	DTP axil_sep_otp_jtag_resp_i	N/A
mpc_debug_halt_req_i	logic	Input	1	Async halt request from MPC.	Debug controller	Tie to '0 if unused
mpc_debug_run_req_i	logic	Input	1	Async run request from MPC.	Debug controller	Tie to '0 if unused
mpc_reset_run_req_i	logic	Input	1	Run/halt selection after reset.	Debug controller	1'b1
cpu_halt_req_i	logic	Input	1	Async halt request to CPU.	Debug controller	Tie to '0 if unused
cpu_run_req_i	logic	Input	1	Async restart request to CPU.	Debug controller	Tie to '0 if unused
test_en_i	logic	Input	1	DFT/scan test-enable. Drives clock-gater test ports and AXI cell test inputs.	Test controller	1'b0
scan_rst_ni	logic	Input	1	Scan reset (active-low). Bypasses reset synchronizers during DFT.	Test controller	1'b1
ext_boot_seq_done_i	logic	Input	1	Boot sequence completion qualified to clk_i (memory repair and shadow register override done); gates reset release. In SMU, driven from SMC-qualified logic. In standalone SEP, synchronize and qualify before this port.	SMC-qualified boot done or external synchronizer	1'b1
dmi_core_enable_i	logic	Input	1	Enables DMI accesses to the core debug module (addresses 0x00-0x4F). Tied high at SMU; TAP access is already lifecycle-gated.	Debug infrastructure	1'b1
dmi_uncore_enable_i	logic	Input	1	Enables DMI accesses to an external uncore aperture (addresses 0x50+). Unused: the core debug module AXI master already reaches the SEP fabric.	Debug infrastructure	Tie to '0 if unused
dmi_uncore_en_o	logic	Output	1	DMI uncore access enable.	Debug infrastructure	N/A
dmi_uncore_wr_en_o	logic	Output	1	DMI uncore write enable.	Debug infrastructure	N/A
dmi_uncore_addr_o	logic	Output	[6:0]	DMI uncore address.	Debug infrastructure	N/A
dmi_uncore_wdata_o	logic	Output	[31:0]	DMI uncore write data.	Debug infrastructure	N/A
dmi_uncore_rdata_i	logic	Input	[31:0]	DMI uncore read data.	Debug infrastructure	N/A
dmi_active_o	logic	Output	1	DMI active indicator.	Debug infrastructure	N/A
sep_cpu_trace_o	sep_cpu_trace_t	Output	-	CPU trace output for debug.	Trace infrastructure	N/A
lockstep_ctrl_i	sep_pkg::sep_lockstep_ctrl_t	Input	-	VeeR lockstep controls: disable corruption detection, and error-injection enable. Inert unless the core is built with RV_LOCKSTEP_ENABLE.	Chip-level RAS control	Tie to '0 if unused
lockstep_status_o	sep_pkg::sep_lockstep_status_t	Output	-	VeeR lockstep status: corruption detected. Reads 0 unless the core is built with RV_LOCKSTEP_ENABLE.	Chip-level RAS aggregation	N/A
jtag_id_i	logic	Input	[31:1]	JTAG ID value.	Configuration tie-off	N/A
timer_int_i	logic	Input	1	Timer interrupt.	Internal timer	Tie to '0 if unused

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
soft_int_i	logic	Input	1	Software interrupt.	Software IRQ source	Tie to '0 if unused
extintsrc_req_i	logic	Input	[NUM_EXTERNAL_IRQS-1:0]	External interrupt sources for PIC.	External IRQ sources	Tie to '0 if unused
sep_cpu_tcm_req_o	sep_cpu_tcm_req_t	Output	-	CPU tightly-coupled memory (TCM) request (ICCM + DCCM).	SRAM macros	N/A
sep_cpu_tcm_rsp_i	sep_cpu_tcm_rsp_t	Input	-	CPU TCM response.	SRAM macros	N/A
sep_sram_req_o	sep_sram_req_t	Output	-	Scratch SRAM request.	SRAM macros	N/A
sep_sram_rsp_i	sep_sram_rsp_t	Input	-	Scratch SRAM response.	SRAM macros	N/A
sep_boot_rom_req_o	sep_sram_req_t	Output	-	Boot ROM (BL0) request.	ROM macro	N/A
sep_boot_rom_rsp_i	sep_sram_rsp_t	Input	-	Boot ROM (BL0) response.	ROM macro	N/A
smn_outbound_axi_req_o	sep_system_peripherals_outbound_axi_req_t	Output	-	SMN outbound AXI request (system bus master).	SMN fabric	N/A
smn_outbound_axi_resp_i	sep_system_peripherals_outbound_axi_resp_t	Input	-	SMN outbound AXI response.	SMN fabric	N/A
smn_inbound_axi_req_i	sep_system_peripherals_internal_axi_req_t	Input	-	SMN inbound AXI request (system bus slave).	SMN fabric	Tie to '0 if unused
smn_inbound_axi_resp_o	sep_system_peripherals_internal_axi_resp_t	Output	-	SMN inbound AXI response.	SMN fabric	N/A
sep_ext_to_smc_axi_req_o	sep_system_peripherals_internal_axi_req_t	Output	-	SEP-to-SMC AXI request. Dedicated path for SEP to access SMC resources.	SMC sep_axi_in_req_i	N/A
sep_ext_to_smc_axi_resp_i	sep_system_peripherals_internal_axi_resp_t	Input	-	SEP-to-SMC AXI response.	SMC sep_axi_in_resp_o	N/A
entropy_rosc_sample_clk_i	logic	Input	1	Ring-oscillator sample clock for entropy source (async vs clk_i; ~100-400 MHz typical).	Ring oscillator	N/A
sep_crypto_pka_imem_sram_req_o	sep_crypto_pka_imem_sram_req_t	Output	-	OTBN PKA instruction memory SRAM request.	SRAM macro	N/A
sep_crypto_pka_imem_sram_rsp_i	sep_crypto_pka_imem_sram_rsp_t	Input	-	OTBN PKA instruction memory SRAM response.	SRAM macro	N/A
sep_crypto_pka_dmem_sram_req_o	sep_crypto_pka_dmem_sram_req_t	Output	-	OTBN PKA data memory SRAM request.	SRAM macro	N/A
sep_crypto_pka_dmem_sram_rsp_i	sep_crypto_pka_dmem_sram_rsp_t	Input	-	OTBN PKA data memory SRAM response.	SRAM macro	N/A
ext_trng_axil_req_o	sep_32_32_axil_req_t	Output	-	External TRNG AXI-Lite passthrough request (to sep_ip_integration).	External TRNG shim	N/A
ext_trng_axil_resp_i	sep_32_32_axil_resp_t	Input	-	External TRNG AXI-Lite passthrough response.	External TRNG shim	N/A
ext_trng_axis_req_i	ext_trng_axis_req_t	Input	[EXT_TRNG_NUM_AXIS-1:0]	External TRNG AXI-Stream request array (from sep_ip_integration).	External TRNG shim	N/A
ext_trng_axis_rsp_o	ext_trng_axis_rsp_t	Output	[EXT_TRNG_NUM_AXIS-1:0]	External TRNG AXI-Stream response array.	External TRNG shim	N/A

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
ext_trng_irq_i	logic	Input	1	External TRNG interrupt (routed to PIC).	External TRNG shim	Tie to '0 if unused
km_rom_mem_req_o	km_intf_pkg::km_rom_mem_req_t	Output	-	Key Manager ROM memory request.	ROM macro	N/A
km_rom_mem_rsp_i	km_intf_pkg::km_rom_mem_rsp_t	Input	-	Key Manager ROM memory response.	ROM macro	N/A
km_sram_mem_req_o	km_intf_pkg::km_sram_mem_req_t	Output	-	Key Manager SRAM memory request.	SRAM macro	N/A
km_sram_mem_rsp_i	km_intf_pkg::km_sram_mem_rsp_t	Input	-	Key Manager SRAM memory response.	SRAM macro	N/A
efuse_bank_ctrl_req_o	sep_efuse_pkg::efuse_axil_req_t	Output	-	eFuse bank control AXI-Lite request.	eFuse shim	N/A
efuse_bank_ctrl_resp_i	sep_efuse_pkg::efuse_axil_resp_t	Input	-	eFuse bank control AXI-Lite response.	eFuse shim	N/A
efuse_shim_command_req_o	sep_efuse_pkg::fuse_command_req_t	Output	-	eFuse command interface request.	eFuse shim	N/A
efuse_shim_command_resp_i	sep_efuse_pkg::fuse_command_resp_t	Input	-	eFuse command interface response.	eFuse shim	N/A
lcc_demote_state_1_o	logic	Output	[1:0]	Lifecycle demote state output 1. Exported at SMU boundary; not currently connected to SMC.	SMU external port	N/A
lcc_demote_state_2_o	logic	Output	[1:0]	Lifecycle demote state output 2. Exported at SMU boundary; not currently connected to SMC.	SMU external port	N/A
sep_io_spi_req_o	sep_io_spi_req_t	Output	-	SPI host controller request.	SPI pad interface	N/A
sep_io_spi_rsp_i	sep_io_spi_rsp_t	Input	-	SPI host controller response.	SPI pad interface	N/A
lc_state_o	logic	Output	[2*LcStateBitWidth-1:0]	Lifecycle state output (differentially encoded). Routed to SMC and the SMU external output; debug-disable control to DTP is via dbg_disable_o.	SMC, SMU external	N/A
dbg_disable_o	sep_lifecycle_ctrl_pkg::dbg_disable_t	Output	-	Typed debug-disable struct routed by SMU to DTP debug-disable control.	SMU → DTP	N/A
sep_fuse_dft_disable_o	logic	Output	1	Gate for the DFT-inserted access path into the SEP OTP, composed as Case 3 AND SEP_FUSE_DBG. Unconnected in the functional design; an adopter's DFT insertion connects it.	Adopter DFT insertion	Leave open if no path is inserted
smc_fuse_dft_disable_o	logic	Output	1	Gate for the DFT-inserted access path into the SMC OTP, composed as Case 2 AND SMC_FUSE_DBG. Unconnected in the functional design; an adopter's DFT insertion connects it.	Adopter DFT insertion	Leave open if no path is inserted
lc_sigint_err_o	logic	Output	1	Lifecycle signal integrity error.	SMU	N/A
security_disable_o	logic	Output	1	Security disable output.	SMU	N/A
secure_tm_o	logic	Output	1	Secure test mode output.	SMU	N/A
smc_mailbox_interrupt_o	logic	Output	[NumMailboxes-1:0]	Mailbox interrupt outputs to SMC.	SMC interrupt aggregator	N/A
smc_fuse_sense_done_i	logic	Input	1	SMC fuse sense done indication.	SMC fuse_sense_done_o	N/A
sep_fuse_sense_done_o	logic	Output	1	SEP fuse sense done output.	System integration	N/A
secure_tm_req_i	logic	Input	-	Secure test mode request.	GPIO padring / tie-off	N/A

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
axi_extension_axi_req_o	sep_32_64_6_12_axi_req_t	Output	-	AXI extension port request for adopter peripheral expansion.	Adopter peripherals	N/A
axi_extension_axi_resp_i	sep_32_64_6_12_axi_resp_t	Input	-	AXI extension port response.	Adopter peripherals	Tie to DECERR if unused
smc_global_base_addr_i	logic	Input	[55:0]	SMC global base address for outbound address translation.	System configuration	N/A
smc_region_size_i	logic	Input	[55:0]	SMC region size for outbound address translation.	SMC smc_region_size_o	N/A
km_unrecoverable_err_o	logic	Output	1	Key Manager unrecoverable error.	RAS / error handling	N/A
km_recoverable_err_o	logic	Output	1	Key Manager recoverable error.	RAS / error handling	N/A
ext_debug_bus_o	logic	Output	[511:0]	External debug observation bus.	Debug infrastructure	N/A

Chapter 11. SMU Composition and Internal Wiring

The System Management Unit (SMU) combines the SMC, DTP, and optionally the SEP into a unified subsystem. The interface descriptions in the preceding sections apply directly when integrating the SMU, since the SMU inherits all three subsystems' ports. This section covers the module variants available to adopters, how the SMU wires its subsystems together internally, and what additional configuration it exposes.

11.1. Module Variants and IP Integration

OCAH core modules (`smc.sv`, `sep.sv`, `dtp.sv`, and the composite `smu.sv`) expose logic-level interfaces only. All technology-specific components — PLLs, PVT sensors, eFuse shims, memory macros, GPIO padrings, SPI controllers, and external TRNG — are instantiated outside the core modules by the adopter. This separation keeps the OCAH deliverable technology-neutral.

The figure below shows that boundary. `smu` exposes a technology integration interface for each of these components, and a reference integration module holds the shims, models and macros behind them. An adopter keeps the interfaces and replaces what lies below the boundary with its own technology IP.

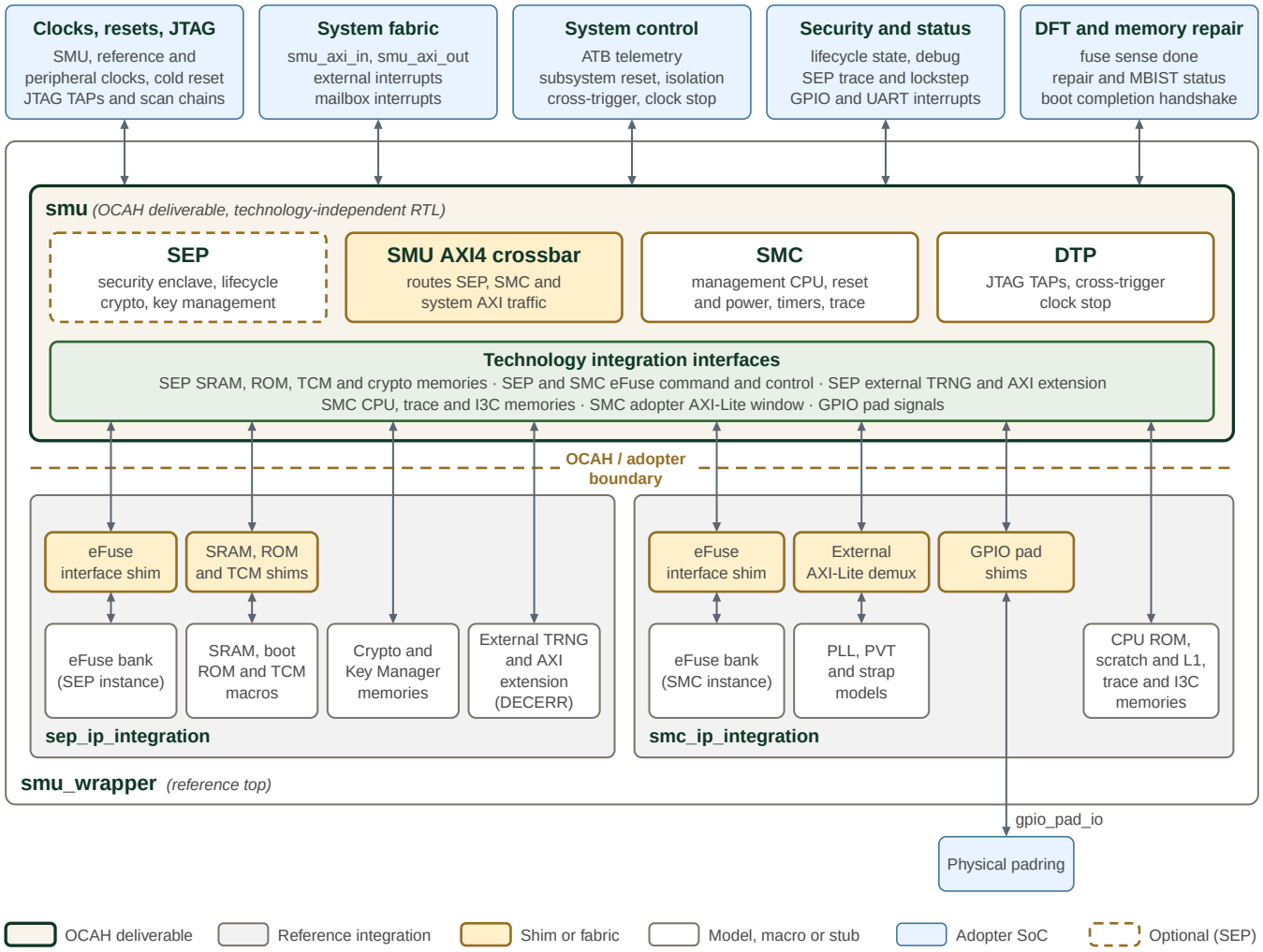


Figure 4. SMU Technology Integration

The repository provides reference wrappers that demonstrate how to bridge the gap between OCAH core interfaces and adopter-specific IP:

- `smu_wrapper.sv` wraps `smu.sv` with pad-level signal handling for clocks, resets, GPIO, eFuse, and JTAG interfaces. It internally instantiates `smc_ip_integration.sv` for SMC-related vendor IP and `sep_ip_integration.sv` for SEP-related vendor IP. Each integration module holds its own eFuse interface shim and eFuse bank.
- `smc_wrapper.sv` provides an equivalent reference wrapping `smc.sv` alone, useful when integrating the SMC without the DTP/SEP subsystems.

Neither the wrappers nor the IP integration modules are part of the OCAH deliverable; they exist as integration guides. Adopters must develop their own equivalents following the interface patterns demonstrated in these modules. The interface contracts at the module boundaries ensure technology substitution without modifying the verified OCAH logic.

The figure below illustrates the relationship between the OCAH core modules, the wrapper layer, and the IP integration modules.

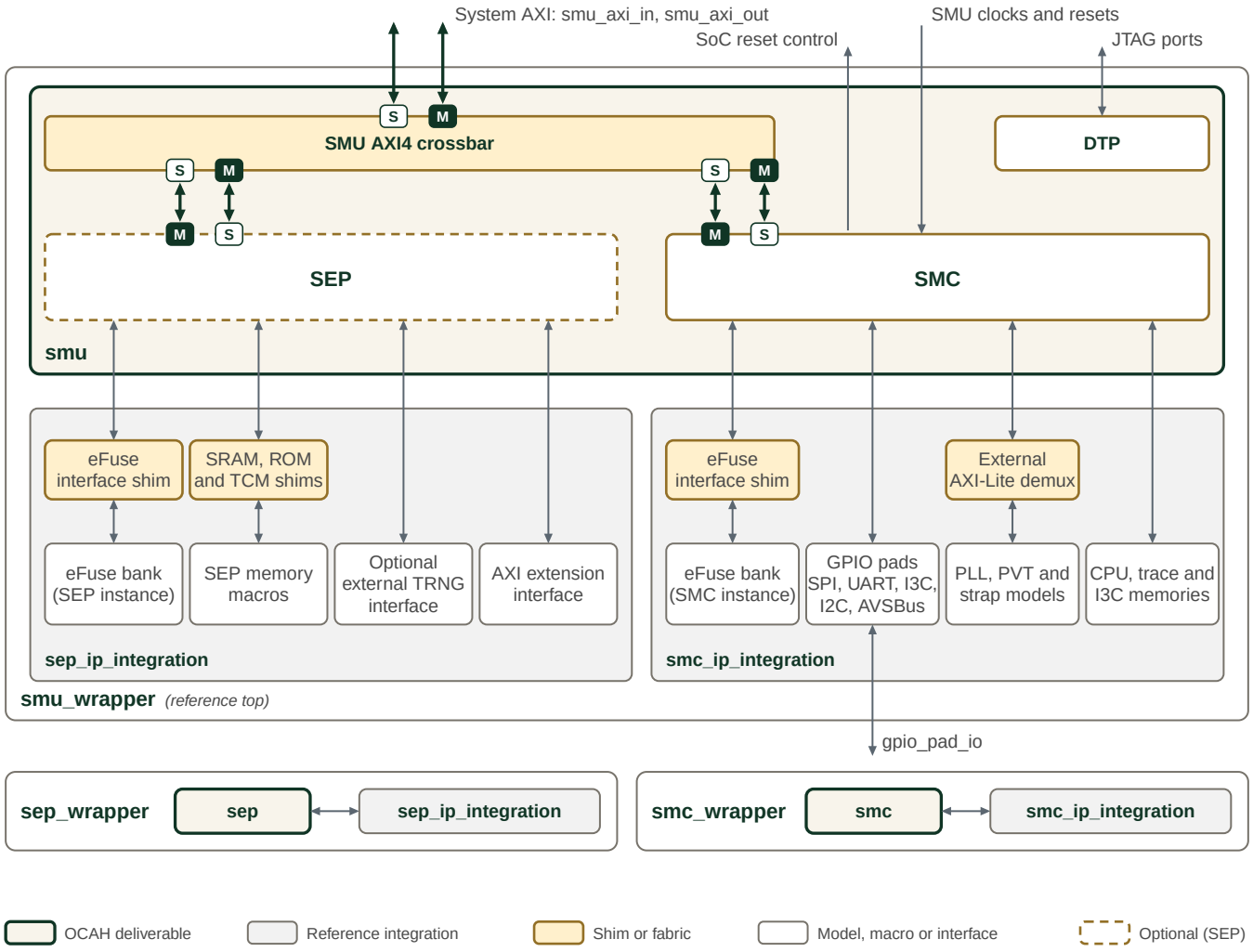


Figure 5. SMU Integration Components

11.1.1. IP Integration Modules

`smc_ip_integration.sv` demonstrates integration of SMC-side technology components: PLL wrappers (generating `pll_cgm_clk_o` and `pll_awm_clk_o`), PVT sensor interfaces, eFuse shims, GPIO padring logic, memory macros (ROM, scratchpad RAM, cache tag and data arrays), and trace memory instances.

`sep_ip_integration.sv` demonstrates integration of SEP-side technology components: SEP memory macros (TCM, scratch SRAM, boot ROM, OTBN IMEM/DMEM, Key Manager ROM/SRAM), eFuse shim, external TRNG, and SPI controller selection.

11.1.2. Wrapper Clock Architecture

The behavioral `pll_wrap` model is a DV/reference placeholder: it free-runs refclk, sys_clk, and periph_clk at 100 / 800 / 200 MHz (`+pll_sys_period_ns=10` runs sys_clk at 100 MHz; with `+pll_osc_bench` a bench toggles the oscillator nets itself), muxes sys and periph with ref through `prim_clock_mux2` then `prim_ag_clk_mux`, and terminates the AXI-Lite PLL control interface with OKAY. It is not a synthesis stand-in. An adopter who replaces `smc_ip_integration` supplies equivalent clocks internally; the bare cores still have the three clock inputs.

11.2. Clock and Reset Architecture

Connect clk_smu_i to the primary system clock and clk_ref_i to a stable reference source. The DTP operates on TCK for JTAG logic and on the system clock for cross-trigger and AXI bridges. Reset sequencing follows the SMC flow; the DTP JTAG state machine uses pwr_on_rst_ni, which the SMU connects to the SMC powergood_stable_o output. The SMU internally connects DTP debug paths to the SMC and optionally instantiates the SEP when CFG.SEP is set to 1. The full interconnection details are described in SMC-DTP Internal Interconnection and SEP-Conditional Wiring below.

11.3. SMU AXI Crossbar Address Map

Inside the SMU, the smu_axi_xbar module routes traffic between three AXI4 manager/subordinate pairs: the SEP outbound master (sep_out), the SMC outbound master (smc_out), and the external SMU AXI input (ext_in / smu_axi_in). The corresponding targets are the SEP and SMC inbound subordinates, and the external SMU AXI output (ext_out / smu_axi_out).

The SMU instantiates smu_axi_xbar only when CFG.SEP is 1. When CFG.SEP is 0, AXI ID-width converters connect the SMC system AXI ports to smu_axi_in / smu_axi_out instead: every SMC-initiated transaction leaves through smu_axi_out, every smu_axi_in transaction targets the SMC, and the aperture rules below do not apply. The smu_axi_in / smu_axi_out port types are the same in both configurations.

See the *SMU AXI Crossbar* and *OCAH Memory Map* sections of the Technical Reference Manual for connectivity and the responsibility split between hardware-fixed and firmware-programmable apertures.

The crossbar uses a **runtime** address map with two rules, the SEP and SMC apertures, both programmable at runtime. ext_out has no address rule. The apertures are driven as follows:

- sep_global_base_addr_i / sep_region_size_i come from the sep_cpu_ctrl CSRs SEP_GLOBAL_BASE_ADDR / SEP_REGION_SIZE, exposed at the sep.sv boundary as sep_global_base_addr_o / sep_region_size_o, and at the smu.sv boundary as sep_global_base_o / sep_region_size_o (both 56-bit). SEP_REGION_SIZE is a 32-bit CSR field, zero-extended to 56-bit at the SEP system CSR; the xbar input takes only the lower 32 bits. SEP_REGION_SIZE defines the SEP global/chiplet aperture for SMU routing and SEP inbound global-to-local remap. It is distinct from the SEP CPU's architectural local-alias span. SEP_REGION_SIZE currently resets to 0x0100_0000 (16 MiB). Tied to '0 at the smu.sv boundary when CFG.SEP is 0.
- smc_global_base_addr_i / smc_region_size_i come from the smc_base_config CSRs GLOBAL_BASE / REGION_SIZE, exposed at both the smc.sv and smu.sv boundaries as smc_global_base_o (56-bit) / smc_region_size_o (32-bit). REGION_SIZE resets to 0x0100_0000 (16 MiB).

SEP, SMC, and smu_axi_xbar all run on clk_smu_i, so these quasi-static CSR fields need no clock-domain crossing. The crossbar's address arithmetic is done on 57-bit widened sums. For where unmatched requests go, and which requests return DECERR, see the SMU AXI Crossbar section of the Technical Reference Manual.

11.3.1. Firmware programming order

Firmware must program SEP_GLOBAL_BASE_ADDR / SEP_REGION_SIZE and SMC GLOBAL_BASE / REGION_SIZE during early boot, **before any cross-subsystem AXI traffic is issued**, and must ensure the two apertures do not overlap and both fall within the chiplet-visible SMU envelope assigned by the integrator. The SMU crossbar does not itself constrain envelope size.

See the *OCAH Memory Map* section of the Technical Reference Manual for that responsibility split.

11.4. SMU Default Parameters

The DTP JTAG features, identification, cross trigger, and pipeline depths are fields of the `CFG` parameter, of type `smu_pkg::smu_cfg_t`. `CFG` defaults to the `smu_pkg::SmuConfigs` preset that the `CFG_IDX` parameter selects: `smu_pkg::DefaultCfg` at the default index 0, or `smu_pkg::NoSepCfg`, which differs only in `SEP`, at index 1. A `CFG` passed directly takes precedence over `CFG_IDX`. Flows that override parameters with `elaborate -param`, which cannot set a struct, select the preset through `CFG_IDX`.

The parameters of `smu` are listed in the HTML edition of the Technical Reference Manual, under RTL Modules Reference.

SEP-related DTP configuration has no `smu_cfg_t` field of its own. The SMU derives SEP debug/STAP enablement from `CFG.SEP` and wires the corresponding DTP interfaces internally to the SEP block when `CFG.SEP=1`.

11.5. IC_RESET TDR Structure

The DTP primary TAP implements the IEEE 1149.1-2013 §17 IC_RESET TDR as three independently enabled slices plus a single `reset_hold` bit nearest TDO. At the SMU the scan order from TDI to TDO is:

```
TDI → SMC slice → SEP slice → external slice → reset_hold → TDO
```

The SMU enables the SMC slice unconditionally, the SEP slice when `CFG.SEP=1` (otherwise that slice has width 0 and is omitted), and the external slice when `JTAG_IC_RESET_ENABLE` is set. Each slice is a packed struct with `.ovrd` and `.val` halves of equal width; `jtag_ptap` sizes the slice from `$bits(type)/2`. Each port has the reset-enable and reset-control pair IEEE §17 requires, with `reset_enable` nearer TDO. `.val` carries `reset_control`, and `.ovrd` carries the inverse of `reset_enable`. Declaration order within each half is the TDI-to-TDO scan order: the first field is nearest TDI.

Slice	Ports	Defined by	Consumed by
SMC	68	OCAH (`smc_pkg::jtag_smc_reset_ctrl_t`)	SMC reset unit; SMU-internal, no SMU port
SEP	8 when `CFG.SEP=1`, else 0	OCAH (`sep_pkg::jtag_sep_reset_ctrl_t`)	SEP reset controller; SMU-internal, no SMU port
External	`$bits(ic_reset_ext_t)/2`	Adopter (`ic_reset_ext_t`)	SMU output `jtag_ic_reset_ext_o`
`reset_hold`	1 bit	IEEE 1149.1 §17	Nearest TDO; not a reset port

The TDR length is `2 * (SMC ports + SEP ports + external ports) + 1`. Programming order and `reset_enable` polarity are in the DTP JTAG chapter of the Technical Reference Manual.

11.5.1. SMC slice (TDI to TDO)

The SMC slice covers the SMC reset domains and the 32 managed-subsystem (SS) cold and warm resets. Vector index `[31]` of each SS field is nearer TDI than `[0]`.

Port	Reset domain
`ss_warm_reset_n[31:0]`	Per-subsystem warm reset
`ss_cold_reset_n[31:0]`	Per-subsystem cold reset
`cold_reset_n`	SMC cold reset
`cool_reset_n`	SMC cool reset

Port	Reset domain
warm_reset_n	SMC warm reset
fuse_reset_n	Fuse reset (nearest the SEP slice)

11.5.2. SEP slice (TDI to TDO)

When CFG.SEP=1 the SEP slice covers the SEP main reset and the crypto, Key Manager, and TRNG resets consumed by sep_reset_ctrl. abr_jtag_rst_n is nearest TDI.

Port	Reset domain
abr_jtag_rst_n	Adams Bridge
trng_jtag_rst_n	TRNG
sep_reset_n	SEP main reset
kmac_jtag_rst_n	KMAC
hmac_jtag_rst_n	HMAC
aes_jtag_rst_n	AES
otbn_jtag_rst_n	OTBN
km_jtag_rst_n	Key Manager (nearest the external slice)

11.5.3. External slice (adopter-defined)

The external slice is reserved for reset overrides on logic instantiated outside the SMU. OCAH does not name its ports. Set the SMU type parameter ic_reset_ext_t to a packed struct with .ovrd and .val halves of equal width, and route jtag_ic_reset_ext_o to that logic. The parameter default (jtag_tap_pkg::jtag_ic_reset_default_t) is a single-override stub. Clear JTAG_IC_RESET_ENABLE when no external overrides are required.

Declare a new external override at each half's MSB so already-assigned TDR positions do not shift. Extra subsystem reset controls beyond the SMC slice use this typed port.

11.6. DTP Parameter Pass-Through

The SMU passes through most DTP parameters to allow integrator customization. Type parameters are set internally based on SMC package types.

11.6.1. JTAG Configuration Parameters

Parameter	SMU Default	Pass-through	Notes
JTAG_BSR_ENABLE	1	Yes	Boundary scan enabled
JTAG_EXTEST_TRAIN_ENABLE	1	Yes	
JTAG_EXTEST_PULSE_ENABLE	1	Yes	
JTAG_INTEST_ENABLE	1	Yes	
JTAG_CLAMP_ENABLE	1	Yes	
JTAG_HIGHZ_ENABLE	1	Yes	
JTAG_RUNBIST_ENABLE	1	Yes	
JTAG_TMP_ENABLE	1	Yes	

Parameter	SMU Default	Pass-through	Notes
JTAG_IC_RESET_ENABLE	1	No	SMU routes this to the DTP's JTAG_IC_RESET_EXT_ENABLE; SMC and SEP slices are enabled inside the SMU instantiation (SMC unconditionally, SEP when CFG.SEP=1)
JTAG_SMC_DBG_ENABLE	1	Yes	
JTAG_SEP_DBG_ENABLE	1 (CFG.SEP=1)	No	Derived internally from CFG.SEP
JTAG_STAP_IO_ENABLE	1	Yes	
JTAG_NUM_EXTRA_STAPS	1	Yes	Port array size equals parameter value; when set to 0, array size is 1 (tie-off required)

11.6.2. JTAG ID Parameters

Parameter	SMU Default	Pass-through	Notes
JTAG_IDCODE_MFR_ID	11'h000	Yes	Manufacturer ID
JTAG_IDCODE_PART_NUM	16'h0000	Yes	Part number
JTAG_IDCODE_SI_REV	4'h0	Yes	Silicon revision
JTAG_OCH_VER	8'h00	Yes	DTP IP major version

11.6.3. Cross Trigger Parameters

Parameter	SMU Default	DTP Internal	Pass-through	Notes
XTRIG_NUM_CTP	16	CFG	Yes	External CTP count
XTRIG_NUM_INT_CT	8	CFG + XtrigSmcIntCtLanes	Yes	SMU-exposed count; DTP adds the SMC-reserved lanes
XTRIG_NUM_CLK_STOP_REQ	8	CFG + XtrigSmcClkStopLanes	Yes	SMU-exposed count; DTP adds the SMC-reserved lane
XTRIG_INT_CT_MODE	32 bits, low 8 consumed	DTP width	Yes	SMC-reserved DTP mode bits are 0 (pulse-sync)

The SMU remaps DTP internal-CT lanes above the SMC reservation onto the SMU-exposed ports. Clock-stop request lane 0 is reserved for SMC; the SMU exposes the remaining DTP lanes.

11.6.4. Pipeline Depth Parameters

Parameter	SMU Default	Pass-through	Notes
SMC_OTP_RD_PL_DEPTH	2'h3	Yes	SMC OTP read pipeline depth
SMC_OTP_WR_PL_DEPTH	2'h3	Yes	SMC OTP write pipeline depth
SEP_OTP_RD_PL_DEPTH	2'h3	No	Fixed to 2'h3 in SMU; used when CFG.SEP=1, ignored when CFG.SEP=0
SEP_OTP_WR_PL_DEPTH	2'h3	No	Fixed to 2'h3 in SMU; used when CFG.SEP=1, ignored when CFG.SEP=0
SMC_RD_PL_DEPTH	2'h3	Yes	SMC fabric read pipeline depth
SMC_WR_PL_DEPTH	2'h3	Yes	SMC fabric write pipeline depth

11.6.5. Type Parameters

The following type parameters are set internally by the SMU and are not exposed for integrator configuration.

Parameter	Set To	Notes
jtag_tap_ctrl_t	prim_jtag_pkg::jtag_tap_ctrl_t	JTAG TAP control struct
jtag_scan_ctrl_t	prim_jtag_pkg::jtag_scan_ctrl_t	JTAG scan control struct
smc_jtag_axi_req_t	smc_pkg::smc_jtag_56_64_2_12_axi_req_t	SMC debug AXI request
smc_jtag_axi_resp_t	smc_pkg::smc_jtag_56_64_2_12_axi_resp_t	SMC debug AXI response
ic_reset_smc_t	smc_pkg::jtag_smc_reset_ctrl_t	SMC IC_RESET TDR slice
ic_reset_sep_t	sep_pkg::jtag_sep_reset_ctrl_t	SEP IC_RESET TDR slice
smc_otp_axil_req_t	smc_pkg::smc_axil_32_32_req_t	SMC OTP AXI-Lite request
smc_otp_axil_resp_t	smc_pkg::smc_axil_32_32_resp_t	SMC OTP AXI-Lite response
sep_otp_axil_req_t	smc_pkg::smc_axil_32_32_req_t	SEP OTP AXI-Lite request (error slave when CFG.SEP=0)
sep_otp_axil_resp_t	smc_pkg::smc_axil_32_32_resp_t	SEP OTP AXI-Lite response (error slave when CFG.SEP=0)
xtrig_axil_req_t	smc_pkg::smc_axil_32_32_req_t	Cross trigger CSR request
xtrig_axil_resp_t	smc_pkg::smc_axil_32_32_resp_t	Cross trigger CSR response

11.7. DTP Port Mapping

The following tables define the complete DTP port list as connected within the SMU. Ports marked "SMU external port" are exposed on the SMU boundary for integrator connection. Ports marked with a subsystem name (e.g. SMC, SEP) are wired internally within the SMU.

11.7.1. Clock and Reset

Port	Direction	Width	SMU Connection
clk_i	input	1	SMU clk_smu_i (shared with SMC clk_smc_i)
rst_n_i	input	1	SMC rst_primary_smc_clk_no
pwr_on_rst_ni	input	1	SMC powergood_stable_o (power-on reset for JTAG logic)

11.7.2. Primary JTAG TAP Interface

Port	Direction	Width	SMU Connection
jtag_ptap_client_tap_ctrl_i	input	jtag_tap_ctrl_t	SMU external port
jtag_ptap_client_tdi_i	input	1	SMU external port
jtag_ptap_client_tdo_o	output	1	SMU external port
jtag_ptap_client_tdo_oen_o	output	1	SMU external port

11.7.3. Boundary Scan Interface

Port	Direction	Width	SMU Connection
jtag_bsr_host_scan_ctrl_o	output	jtag_scan_ctrl_t	SMU external port
jtag_bsr_host_scan_in_i	input	1	SMU external port
jtag_bsr_host_scan_out_o	output	1	SMU external port

11.7.4. I/O STAP Interface

Port	Direction	Width	SMU Connection
jtag_stap_io_host_tap_ctrl_o	output	jtag_tap_ctrl_t	SMU external port
jtag_stap_io_host_tdi_i	input	1	SMU external port
jtag_stap_io_host_tdo_o	output	1	SMU external port
jtag_stap_io_host_tdo_oen_o	output	1	SMU external port

11.7.5. SMC Debug STAP Interface

Port	Direction	Width	SMU Connection
jtag_stap_smc_host_tap_ctrl_o	output	jtag_tap_ctrl_t	SMC smc_cpu_jtag_{TCK,TMS}_i; trst_n inverted drives SMC smc_cpu_jtag_reset_i when JTAG_SMC_DBG_ENABLE=1, else tie-off
jtag_stap_smc_host_tdi_i	input	1	SMC smc_cpu_jtag_TDO_data_o when JTAG_SMC_DBG_ENABLE=1, else 1'b0
jtag_stap_smc_host_tdo_o	output	1	SMC smc_cpu_jtag_TDI_i when JTAG_SMC_DBG_ENABLE=1, else unused
jtag_stap_smc_host_tdo_oen_o	output	1	Internal only; not consumed by SMC

11.7.6. SEP Debug STAP Interface

Port	Direction	Width	SMU Connection
jtag_stap_sep_host_tap_ctrl_o	output	jtag_tap_ctrl_t	SEP jtag_{tck_i,tms_i,trst_ni} when CFG.SEP=1, else unused
jtag_stap_sep_host_tdi_i	input	1	SEP jtag_tdo_o when CFG.SEP=1, else 1'b0
jtag_stap_sep_host_tdo_o	output	1	SEP jtag_tdi_i when CFG.SEP=1, else unused
jtag_stap_sep_host_tdo_oen_o	output	1	Internal only; not consumed by SEP at SMU level

11.7.7. Extra STAP Interfaces

Port	Direction	Width	SMU Connection
jtag_stap_extra_host_tap_ctrl_o	output	jtag_tap_ctrl_t[JtagNumExtraStapPorts-1:0]	SMU external port
jtag_stap_extra_host_tdi_i	input	[JtagNumExtraStapPorts-1:0]	SMU external port
jtag_stap_extra_host_tdo_o	output	[JtagNumExtraStapPorts-1:0]	SMU external port
jtag_stap_extra_host_tdo_oen_o	output	[JtagNumExtraStapPorts-1:0]	SMU external port

11.7.8. Extended STAP Scan Interface

Port	Direction	Width	SMU Connection
jtag_stap_host_scan_ctrl_o	output	jtag_scan_ctrl_t	SMU external port
jtag_stap_host_scan_in_i	input	1	SMU external port
jtag_stap_host_scan_out_o	output	1	SMU external port

11.7.9. DFD iJTAG Interface

Port	Direction	Width	SMU Connection
jtag_dfd_host_scan_ctrl_o	output	jtag_scan_ctrl_t	SMU external port
jtag_dfd_host_scan_in_i	input	1	SMU external port
jtag_dfd_host_scan_out_o	output	1	SMU external port

11.7.10. DFT iJTAG Interfaces

Port	Direction	Width	SMU Connection
jtag_dft_secure_host_scan_ctrl_o	output	jtag_scan_ctrl_t	Secure DFT iJTAG SMU external port
jtag_dft_secure_host_scan_in_i	input	1	Secure DFT iJTAG SMU external port
jtag_dft_secure_host_scan_out_o	output	1	Secure DFT iJTAG SMU external port
jtag_dft_host_scan_ctrl_o	output	jtag_scan_ctrl_t	Non-secure DFT iJTAG SMU external port
jtag_dft_host_scan_in_i	input	1	Non-secure DFT iJTAG SMU external port
jtag_dft_host_scan_out_o	output	1	Non-secure DFT iJTAG SMU external port

11.7.11. SMC Fabric Debug AXI Interface

Port	Direction	Width	SMU Connection
axi_smc_dbg_req_o	output	smc_jtag_axi_req_t	SMC jtag_axi_in_req_i
axi_smc_dbg_resp_i	input	smc_jtag_axi_resp_t	SMC jtag_axi_in_resp_o

11.7.12. SMC OTP Debug AXI-Lite Interface

Port	Direction	Width	SMU Connection
axil_smc_otp_jtag_req_o	output	smc_otp_axil_req_t	SMC axil_smc_otp_jtag_req_i
axil_smc_otp_jtag_resp_i	input	smc_otp_axil_resp_t	SMC axil_smc_otp_jtag_resp_o

11.7.13. SEP OTP Debug AXI-Lite Interface

When CFG.SEP=1, this interface connects to the SEP axil_sep_otp_jtag_* ports. When CFG.SEP=0, the SMU instantiates u_sep_otp_axil_err_slv, which responds with DECERR to all transactions.

Port	Direction	Width	SMU Connection
axil_sep_otp_jtag_req_o	output	sep_otp_axil_req_t	SEP axil_sep_otp_jtag_req_i when CFG.SEP=1, else AXI-Lite error slave (u_sep_otp_axil_err_slv)
axil_sep_otp_jtag_resp_i	input	sep_otp_axil_resp_t	SEP axil_sep_otp_jtag_resp_o when CFG.SEP=1, else AXI-Lite error slave (u_sep_otp_axil_err_slv)

11.7.14. Clock Control

Port	Direction	Width	SMU Connection
stop_clks_o	output	1	SMU external port: dtp_stop_clks_o (to PLL clock gates)

11.7.15. Boot Stall Control

Port	Direction	Width	SMU Connection
jtag_boot_stall_ovrd_o	output	1	SMC boot_stall_jtag_ovrd_i
jtag_boot_stall_o	output	1	SMC boot_stall_jtag_val_i

11.7.16. JTAG Reset Control

Port	Direction	Width	SMU Connection
jtag_ic_reset_smc_o	output	smc_pkg::jtag_smc_reset_ctrl_t	SMC jtag_smc_reset_ctrl_i (packed struct with .ovrd override enables and .val active-low override values)
jtag_ic_reset_sep_o	output	sep_pkg::jtag_sep_reset_ctrl_t	SEP jtag_sep_reset_ctrl_i (packed struct with .ovrd override enables and .val active-low override values); SMU-internal, no SMU port
jtag_ic_reset_ext_o	output	ic_reset_ext_t	SMU jtag_ic_reset_ext_o external port (integrator-defined packed struct; carries reset overrides for logic outside the SMU)

11.7.17. JTAG State Outputs

Port	Direction	Width	SMU Connection
jtag_ptap_state_o	output	tap_state_e	SMU external port
jtag_ptap_inst_decoded_o	output	jtag_instruction_decoded_e	SMU external port

11.7.18. Cross Trigger CSR AXI-Lite Interface

Port	Direction	Width	SMU Connection
axil_xtrig_req_i	input	xtrig_axil_req_t	SMC axil_dtp_csr_req_o
axil_xtrig_resp_o	output	xtrig_axil_resp_t	SMC axil_dtp_csr_resp_i

11.7.19. Cross Trigger Matrix Interface

Port	Direction	Width	SMU Connection
xtrig_ctm_src_req_o	output	[XtrigNumIntCt-1:0]	SMC-reserved DTP lanes stay inside the SMU; remaining lanes are the SMU external port
xtrig_ctm_src_ack_i	input	[XtrigNumIntCt-1:0]	SMC-reserved DTP lanes tied off; remaining lanes are the SMU external port
xtrig_ctm_dst_req_i	input	[XtrigNumIntCt-1:0]	SMC-reserved DTP lanes from SMC xtrigger_ss_o; remaining lanes are the SMU external port
xtrig_ctm_dst_ack_o	output	[XtrigNumIntCt-1:0]	SMC-reserved DTP lanes unused; remaining lanes are the SMU external port

The low smu_pkg::XtrigSmcIntCtLanes DTP internal-CT lanes are reserved for SMC (both bits of xtrigger_ss_o). The SMU exposes the remaining DTP lanes, so the SMU XTRIG_NUM_INT_CT field is that many less than DTP's. The SMC uses pulse-synchronization mode, so ack signals for the reserved lanes are unused: the DTP xtrig_ctm_src_ack_i reserved slice is tied to '0 and xtrig_ctm_dst_ack_o reserved bits are left unconnected. The usage of ack signals depends on the XTRIG_INT_CT_MODE parameter: when a mode bit is 0 (pulse synchronization), the corresponding ack is unused; when 1 (req/ack handshaking), the ack must be connected to the cross trigger endpoint.

11.7.20. Clock Stop Request Interface

Port	Direction	Width	SMU Connection
xtrig_clk_stop_req_i	input	[XtrigNumClkStopReq-1:0]	DTP lane 0: SMC tdr_dbg_ctrl_clocks_stopped_by_cla_o; remaining DTP lanes are the SMU external port

Clock-stop request lane 0 is reserved for SMC. The SMU exposes the remaining DTP lanes, so the SMU XTRIG_NUM_CLK_STOP_REQ field is smu_pkg::XtrigSmcClkStopLanes less than DTP's.

11.7.21. Cross Trigger Port GPIO Interface

CTP signals from DTP are ported through the SMU and are direct smu_wrapper external ports (connected through the .* SMU instantiation). The adopter must connect them to GPIO pad logic; no tie-off or dangling is present in the wrapper itself. Widths follow CFG.XTRIG_NUM_CTP (default 16).

Port	Direction	Width	SMU Connection
xtrig_ctp_req_out_dout_o	output	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_req_out_dout_en_o	output	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_req_out_din_i	input	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_req_out_din_en_o	output	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_req_in_dout_o	output	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_req_in_dout_en_o	output	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_req_in_din_i	input	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_req_in_din_en_o	output	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_ack_in_dout_o	output	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_ack_in_dout_en_o	output	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_ack_in_din_i	input	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_ack_in_din_en_o	output	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_ack_out_dout_o	output	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_ack_out_dout_en_o	output	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_ack_out_din_i	input	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic
xtrig_ctp_ack_out_din_en_o	output	[XtrigNumCtp-1:0]	SMU external port; connect to GPIO pad logic

11.8. SMC Port Mapping

The following tables define the complete SMC port list as connected within the SMU. Ports marked "SMU external port" are exposed on the SMU boundary. Ports marked with "DTP" or "SEP" are wired internally.

11.8.1. Clocks

Port	Direction	Width	SMU Connection
clk_smc_i	input	1	SMU clk_smu_i (shared with DTP clk_i)
clk_periph_i	input	1	SMU external port
clk_ref_i	input	1	SMU external port

11.8.2. Resets

Port	Direction	Width	SMU Connection
rst_cold_ni	input	1	SMU external port
rst_primary_ref_clk_no	output	1	SMU external port
rst_primary_smc_clk_no	output	1	SMU external port + DTP rst_n_i

11.8.3. System AXI Input

Port	Direction	Width	SMU Connection
sys_axi_in_req_i	input	smc_sys_in_56_64_6_12_axi_req_t	SMU external port
sys_axi_in_resp_o	output	smc_sys_in_56_64_6_12_axi_resp_t	SMU external port

11.8.4. JTAG AXI Input

Port	Direction	Width	SMU Connection
jtag_axi_in_req_i	input	smc_jtag_56_64_2_12_axi_req_t	DTP axi_smc_dbg_req_o
jtag_axi_in_resp_o	output	smc_jtag_56_64_2_12_axi_resp_t	DTP axi_smc_dbg_resp_i

11.8.5. SMC OTP JTAG AXI-Lite

Port	Direction	Width	SMU Connection
axil_smc_otp_jtag_req_i	input	efuse_axil_req_t	DTP axil_smc_otp_jtag_req_o
axil_smc_otp_jtag_resp_o	output	efuse_axil_resp_t	DTP axil_smc_otp_jtag_resp_i

11.8.6. SEP AXI Input

When CFG.SEP=0, the SEP AXI subordinate input is tied off to '0 and the response output is left unconnected. When CFG.SEP=1, the SMU drives this path from the SEP-to-SMC dedicated AXI connection.

Port	Direction	Width	SMU Connection
sep_axi_in_req_i	input	smc_sep_in_56_64_6_12_axi_req_t	Tied off to '0 (no incoming transactions)
sep_axi_in_resp_o	output	smc_sep_in_56_64_6_12_axi_resp_t	Unconnected (no transactions to respond to)

11.8.7. AXI Output

Port	Direction	Width	SMU Connection
output_axi_req_o	output	smc_sys_out_56_64_8_12_axi_req_t	SMU external port
output_axi_resp_i	input	smc_sys_out_56_64_8_12_axi_resp_t	SMU external port

11.8.8. DTP CSR Interface

Port	Direction	Width	SMU Connection
axil_dtp_csr_req_o	output	smc_axil_32_32_req_t	DTP axil_xtrig_req_i
axil_dtp_csr_resp_i	input	smc_axil_32_32_resp_t	DTP axil_xtrig_resp_o

11.8.9. Adopter Peripheral Window AXI-Lite

Port	Direction	Width	SMU Connection
smc_external_req_o	output	smc_axil_32_32_req_t	SMU external port (to adopter peripherals)
smc_external_resp_i	input	smc_axil_32_32_resp_t	SMU external port (from adopter peripherals)

11.8.10. eFuse Interface

Port	Direction	Width	SMU Connection
efuse_bank_ctrl_req_o	output	efuse_axil_req_t	SMU external port
efuse_bank_ctrl_resp_i	input	efuse_axil_resp_t	SMU external port
efuse_shim_command_req_o	output	fuse_command_req_t	SMU external port
efuse_shim_command_resp_i	input	fuse_command_resp_t	SMU external port
shadow_regs_o	output	efuse_map_t	SMU external port

11.8.11. GPIO Data Signals

Port	Direction	Width	SMU Connection
lsio_interface_select_o	output	[NumGpioWraps-1:0]	SMU external port
pad2core_i	input	[NumGpioWraps-1:0]	SMU external port
core2pad_o	output	[NumGpioWraps-1:0]	SMU external port
pad2core_en_o	output	[NumGpioWraps-1:0]	SMU external port
core2pad_en_o	output	[NumGpioWraps-1:0]	SMU external port

11.8.12. GPIO External Pins

Port	Direction	Width	SMU Connection
rst_cool_n_from_pin_i	input	1	SMU external port

11.8.13. ATB Telemetry

Port	Direction	Width	SMU Connection
clk_telemetry_i	input	1	SMU external port
rst_telemetry_ni	input	1	SMU external port
telemetry_atdata_i	input	telemetry_data_t[NumTelemetryReceivers-1:0]	SMU external port
telemetry_atid_i	input	atb_id_t[NumTelemetryReceivers-1:0]	SMU external port
telemetry_atready_o	output	[NumTelemetryReceivers-1:0]	SMU external port
telemetry_atvalid_i	input	[NumTelemetryReceivers-1:0]	SMU external port
telemetry_afvalid_o	output	[NumTelemetryReceivers-1:0]	SMU external port
telemetry_afready_i	input	[NumTelemetryReceivers-1:0]	SMU external port

11.8.14. DED/WDT

Port	Direction	Width	SMU Connection
smc_cluster_ded_o	output	1	SMU external port
smc_wdt_first_timeout_o	output	1	SMU external port
smc_wdt_second_timeout_o	output	1	SMU external port

11.8.15. Region Size

Port	Direction	Width	SMU Connection
smc_region_size_o	output	[31:0]	SMU external port

11.8.16. External Interrupts

Port	Direction	Width	SMU Connection
smc_ext_interrupts_i	input	[NUM_EXT_INTERRUPTS-1:0]	SMU external port
sep_mailbox_interrupts_i	input	[7:0]	Internal: from SEP smc_mailbox_interrupt_o when CFG.SEP=1, else tied to '0
sep_wdt_reset_n_i	input	1	Internal: ~sep_wdt_timer_rst_req when CFG.SEP=1, else tied to 1'b1

11.8.17. Fuse Signals

Port	Direction	Width	SMU Connection
smc_fuse_sense_done_o	output	1	SMU external port
smc_fuse_reset_n_delayed_o	output	1	SMU external port

11.8.18. DFT Signals

Port	Direction	Width	SMU Connection
skip_mem_repair_o	output	1	SMU external port
ext_boot_seq_done_i	input	1	SMU external port
ext_boot_seq_done_qual_o	output	1	Internal: drives SEP ext_boot_seq_done_i when CFG.SEP=1

11.8.19. Lifecycle State

Port	Direction	Width	SMU Connection
lc_state_o	output	[2*LcStateWidth-1:0]	SEP lifecycle state when CFG.SEP=1, else 8'hf0; ported through SMU
lc_sigint_err_o	output	1	Differential encoding error reported by the SEP (when CFG.SEP=1) or the SMC efuse interface.

11.8.20. Mailbox Interrupts

Port	Direction	Width	SMU Connection
smc_ext_mailbox_interrupts_o	output	[NumMailboxes-1:0]	SMU external port

11.8.21. Reset Unit Signals

Port	Direction	Width	SMU Connection
cfg_flr_pf_active_i	input	1	SMU external port
isolate_req_o	output	[31:0]	SMU external port
ss_reset_complete_i	input	[31:0]	SMU external port
ss_config_o	output	[31:0]	SMU external port
ss_reset_ctrl_o	output	reset_ctrl_t[31:0]	SMU external port
sync_irq_o	output	1	SMU external port

11.8.22. CPU Memory Signals

Port	Direction	Width	SMU Connection
smc_rom_intf_req_o	output	rom_req_t	SMU external port
smc_rom_intf_rsp_i	input	rom_rsp_t	SMU external port
smc_scratch_ram_intf_req_o	output	scratch_ram_req_t[31:0]	SMU external port
smc_scratch_ram_intf_rsp_i	input	scratch_ram_rsp_t[31:0]	SMU external port
smc_l1_icache_tag_intf_req_o	output	l1_icache_tag_req_t[3:0]	SMU external port
smc_l1_icache_tag_intf_rsp_i	input	l1_icache_tag_rsp_t[3:0]	SMU external port
smc_l1_icache_data_intf_req_o	output	l1_icache_data_req_t[7:0]	SMU external port
smc_l1_icache_data_intf_rsp_i	input	l1_icache_data_rsp_t[7:0]	SMU external port
smc_l1_dcache_tag_intf_req_o	output	l1_dcache_tag_req_t[3:0]	SMU external port
smc_l1_dcache_tag_intf_rsp_i	input	l1_dcache_tag_rsp_t[3:0]	SMU external port
smc_l1_dcache_data_intf_req_o	output	l1_dcache_data_req_t[3:0]	SMU external port
smc_l1_dcache_data_intf_rsp_i	input	l1_dcache_data_rsp_t[3:0]	SMU external port

11.8.23. Memory Init

Port	Direction	Width	SMU Connection
smc_disable_sram_auto_init_i	input	1	SMU external port
smc_init_mem_done_o	output	1	SMU external port

11.8.24. System Timer OCTS Interface

Port	Direction	Width	SMU Connection
timer_count_o	output	[63:0]	SMU external port

11.8.25. Reset Overrides (from DTP)

Port	Direction	Width	SMU Connection
jtag_reset_ctrl_i	input	smc_pkg::jtag_smc_reset_ctrl_t	DTP jtag_ic_reset_smc_o (packed struct with .ovrd override enables and .val active-low override values)

11.8.26. DFD Signals

Port	Direction	Width	SMU Connection
cla_gpio_enable_o	output	1	SMU external port
tdr_dbg_ctrl_clock_stop_en_i	input	1	DTP cla_clock_stop_en_o
tdr_dbg_ctrl_clocks_stopped_by_cla_o	output	1	DTP xtrig_clk_stop_req_i[0] (SMU connects SMC output to DTP input)

11.8.27. Cross Trigger Interface

Port	Direction	Width	SMU Connection
xtrigger_ss_i	input	smc_pkg::xtrigger_t	SMU internal: DTP xtrig_ctm_src_req_o[1:0] → SMC. Not an SMU external port.
xtrigger_ss_o	output	smc_pkg::xtrigger_t	SMU internal: SMC → DTP xtrig_ctm_dst_req_i[1:0]. Not an SMU external port.

11.8.28. Trace Memory

Port	Direction	Width	SMU Connection
trace_mem_req_o	output	SinkMemPktIn_s[TRC_RAM_INSTANCES-1:0]	SMU external port
trace_mem_resp_i	input	SinkMemPktOut_s[TRC_RAM_INSTANCES-1:0]	SMU external port

11.8.29. External Debug Bus

Port	Direction	Width	SMU Connection
ext_debug_bus_i	input	[127:0]	SMU external port

11.8.30. Test Mode

Port	Direction	Width	SMU Connection
test_en_i	input	1	SMU external port
scan_rst_ni	input	1	SMU external port

11.9. SMC-DTP Internal Interconnection

The following table summarizes the key internal connections between the DTP and SMC within the SMU. These connections are not visible on the SMU boundary.

Connection	DTP Port	SMC Port	Direction	Notes
AXI Debug Bus	axi_smc_dbg_req_o	jtag_axi_in_req_i	DTP → SMC	
	axi_smc_dbg_resp_i	jtag_axi_in_resp_o	SMC → DTP	
AXI OTP Bus	axil_smc_otp_jtag_req_o	axil_smc_otp_jtag_req_i	DTP → SMC	
	axil_smc_otp_jtag_resp_i	axil_smc_otp_jtag_resp_o	SMC → DTP	
DTP CSR Bus	axil_xtrig_req_i	axil_dtp_csr_req_o	SMC → DTP	

Connection	DTP Port	SMC Port	Direction	Notes
	axil_xtrig_resp_o	axil_dtp_csr_resp_i	DTP → SMC	
JTAG SMC Reset Overrides	jtag_ic_reset_smc_o	jtag_reset_ctrl_i	DTP → SMC	Packed smc_pkg::jtag_smc_reset_ctrl_t struct carrying all SMC reset override enables and active-low values
DTP Reset	rst_n_i	rst_primary_smc_clk_no	SMC → DTP	
Cross Trigger reserved lanes	xtrig_ctm_dst_req_i[XtrigSmcIntCtLanes-1:0]	xtrigger_ss_o	SMC → DTP	smu_pkg::XtrigSmcIntCtLanes DTP lanes
Clock Stop reserved lane	xtrig_clk_stop_req_i[0]	tdr_dbg_ctrl_clocks_stopped_by_cla_o	SMC → DTP	smu_pkg::XtrigSmcClkStopLanes DTP lane
Boot Stall	jtag_boot_stall_ovrd_o	boot_stall_jtag_ovrd_i	DTP → SMC	
	jtag_boot_stall_o	boot_stall_jtag_val_i	DTP → SMC	

11.10. SEP-Conditional Wiring

SEP-related interfaces are handled internally within the SMU. Their behavior depends on CFG.SEP.

Module/Signal	Implementation	Description
SMU AXI fabric	smu_axi_xbar when CFG.SEP=1, else u_iw_conv_smc_out / u_iw_conv_smc_in between the SMC system AXI ports and smu_axi_out / smu_axi_in	System-level AXI routing between SEP, SMC and the SMU boundary
SMC SEP AXI Input	sep_axi_in_req_i = SEP traffic when CFG.SEP=1, else '0	Dedicated SEP-to-SMC subordinate path
DTP SEP OTP JTAG	SEP axil_sep_otp_jtag_* when CFG.SEP=1, else u_sep_otp_axil_err_slv	SEP OTP debug AXI-Lite path
DTP SEP STAP return	SEP jtag_tdo_o when CFG.SEP=1, else sep_stap_tdo_to_dtp = 1'b0	SEP JTAG STAP return path
SMU	sep_mailbox_interrupts	Internal SEP-to-SMC mailbox interrupts when CFG.SEP=1, else '0
SMC	sep_wdt_reset_n_i = ~sep_wdt_timer_rst_req when CFG.SEP=1, else 1'b1	SEP watchdog reset input to SMC

11.10.1. Other Internal Tie-offs

Module	Port	Tie-off Value	Reason
DTP	xtrig_ctm_src_ack_i[XtrigSmcIntCtLanes-1:0]	'0	SMC uses pulse-sync mode

The remaining DTP xtrig_ctm_src_ack_i lanes (exposed as SMU xtrig_ctm_src_ack_i[XTRIG_NUM_INT_CT-1:0]) are ported out through the SMU. The integrator should tie off to '0 when using pulse-sync mode (XTRIG_INT_CT_MODE bit = 0), or connect to actual ack signals when using req/ack mode (bit = 1).

11.11. SMU Port Interface

The SMU exposes a single consolidated boundary that an adopter connects to at the chip or chiplet level. The following table defines all SMU top-level ports including clock, reset, JTAG, cross trigger, SMC-originated interfaces (AXI, GPIO, eFuse, PLL, PVT, memory), and SEP passthrough interfaces (memory macros, eFuse, crypto SRAM, SPI, AXI extension). Ports prefixed with sep_ are only active when CFG.SEP=1.

Table 11. SMU Port Declaration

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
clk_smu_i	logic	Input	1	Primary SMU clock domain. Shared by SMC and DTP system logic.	PLL output	N/A
clk_ref_i	logic	Input	1	Reference clock domain for timing-sensitive operations.	PLL reference clock	N/A
rst_cold_ni	logic	Input	1	Cold reset (active low). Assertion and deassertion are synchronous to clk_ref_i and do not propagate while that clock is stopped. Assertion is de-glitched for 32 cycles. A falling powergood_i asserts the downstream cold reset without that wait. Deassertion is held for 255 cycles.	System cold reset	N/A
rst_cold_stable_ref_clk_no	logic	Output	1	Stable cold reset synchronized to reference clock domain (active low).	Downstream subsystems	N/A
powergood_i	logic	Input	1	Power-good indication from supply supervisor.	Power supply supervisor	N/A
jtag_ptap_client_tap_ctrl_i	prim_jtag_pkg::jtag_tap_ctrl_t	Input	-	Primary JTAG TAP control (TCK, TMS, TRST_N).	External JTAG pads	N/A
jtag_ptap_client_tdi_i	logic	Input	1	Primary JTAG TDI.	External JTAG pads	N/A
jtag_ptap_client_tdo_o	logic	Output	1	Primary JTAG TDO.	External JTAG pads	N/A
jtag_ptap_client_tdo_oen_o	logic	Output	1	Primary JTAG TDO output enable.	External JTAG pads	N/A
jtag_bsr_host_scan_ctrl_o	prim_jtag_pkg::jtag_scan_ctrl_t	Output	-	Boundary scan control.	Boundary scan chain	N/A
jtag_bsr_host_scan_in_i	logic	Input	1	Boundary scan data in.	Boundary scan chain	N/A
jtag_bsr_host_scan_out_o	logic	Output	1	Boundary scan data out.	Boundary scan chain	N/A
jtag_stap_io_host_tap_ctrl_o	prim_jtag_pkg::jtag_tap_ctrl_t	Output	-	I/O STAP TAP control.	I/O STAP host	N/A
jtag_stap_io_host_tdi_i	logic	Input	1	I/O STAP TDI.	I/O STAP host	Tie to '0 if unused
jtag_stap_io_host_tdo_o	logic	Output	1	I/O STAP TDO.	I/O STAP host	N/A
jtag_stap_io_host_tdo_oen_o	logic	Output	1	I/O STAP TDO output enable.	I/O STAP host	N/A
jtag_stap_extra_host_tap_ctrl_o	prim_jtag_pkg::jtag_tap_ctrl_t	Output	[JtagNumExtraStapPorts-1:0]	Extra STAP TAP control array.	Extra STAP hosts	Tie to '0 if unused
jtag_stap_extra_host_tdi_i	logic	Input	[JtagNumExtraStapPorts-1:0]	Extra STAP TDI array.	Extra STAP hosts	Tie to '0 if unused
jtag_stap_extra_host_tdo_o	logic	Output	[JtagNumExtraStapPorts-1:0]	Extra STAP TDO array.	Extra STAP hosts	N/A
jtag_stap_extra_host_tdo_oen_o	logic	Output	[JtagNumExtraStapPorts-1:0]	Extra STAP TDO output enable array.	Extra STAP hosts	N/A
jtag_stap_host_scan_ctrl_o	prim_jtag_pkg::jtag_scan_ctrl_t	Output	-	Extended STAP scan control.	STAP scan chain	N/A

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
jtag_stap_host_scan_in_i	logic	Input	1	Extended STAP scan data in.	STAP scan chain	N/A
jtag_stap_host_scan_out_o	logic	Output	1	Extended STAP scan data out.	STAP scan chain	N/A
jtag_dfd_host_scan_ctrl_o	prim_jtag_pkg::jtag_scan_ctrl_t	Output	-	DFD iJTAG scan control.	DFD iJTAG chain	N/A
jtag_dfd_host_scan_in_i	logic	Input	1	DFD iJTAG scan data in.	DFD iJTAG chain	N/A
jtag_dfd_host_scan_out_o	logic	Output	1	DFD iJTAG scan data out.	DFD iJTAG chain	N/A
jtag_dft_secure_host_scan_ctrl_o	prim_jtag_pkg::jtag_scan_ctrl_t	Output	-	Secure DFT iJTAG scan control.	Secure DFT iJTAG chain	N/A
jtag_dft_secure_host_scan_in_i	logic	Input	1	Secure DFT iJTAG scan data in.	Secure DFT iJTAG chain	N/A
jtag_dft_secure_host_scan_out_o	logic	Output	1	Secure DFT iJTAG scan data out.	Secure DFT iJTAG chain	N/A
jtag_dft_host_scan_ctrl_o	prim_jtag_pkg::jtag_scan_ctrl_t	Output	-	Non-secure DFT iJTAG scan control.	Non-secure DFT iJTAG chain	N/A
jtag_dft_host_scan_in_i	logic	Input	1	Non-secure DFT iJTAG scan data in.	Non-secure DFT iJTAG chain	N/A
jtag_dft_host_scan_out_o	logic	Output	1	Non-secure DFT iJTAG scan data out.	Non-secure DFT iJTAG chain	N/A
dtp_stop_clks_o	logic	Output	1	Clock stop output from DTP (JTAG DEBUG_CONTROL or CLA clock-stop).	PLL clock gates	N/A
jtag_ptap_state_o	jtag_tap_pkg::tap_state_e	Output	-	Current PTAP TAP state.	Debug infrastructure	N/A
jtag_ptap_inst_decoded_o	jtag_inst_reg_pkg::jtag_instruction_decoded_e	Output	-	Current decoded PTAP instruction.	Debug infrastructure	N/A
jtag_ic_reset_ext_o	ic_reset_ext_t	Output	$bits(ic_reset_ext_t)	External JTAG IC_RESET override slice. Integrator-defined packed struct with .ovrd override enables and .val active-low override values. Driven by the DTP's jtag_ic_reset_ext_o when CFG.JTAG_IC_RESET_ENABLE=1, otherwise tied to '0. An integrator needing JTAG reset overrides for logic outside the SMU sets ic_reset_ext_t to their own override struct and routes this port to it; the SMU does not define that type.	Integrator-defined reset overrides	N/A
xtrig_ctm_src_req_o	logic	Output	[XtrigNumIntCt-1:0]	Cross trigger source requests. The SMU exposes DTP lanes above the SMC reservation (smu_pkg::XtrigSmcIntCtLanes).	Cross trigger network	N/A
xtrig_ctm_src_ack_i	logic	Input	[XtrigNumIntCt-1:0]	Cross trigger source acknowledges. Unused in pulse-sync mode (mode bit = 0).	Cross trigger network	Tie to '0 in pulse-sync mode
xtrig_ctm_dst_req_i	logic	Input	[XtrigNumIntCt-1:0]	Cross trigger destination requests. The SMU exposes DTP lanes above the SMC reservation.	Cross trigger network	Tie to '0 if unused
xtrig_ctm_dst_ack_o	logic	Output	[XtrigNumIntCt-1:0]	Cross trigger destination acknowledges. Unused in pulse-sync mode.	Cross trigger network	N/A
xtrig_clk_stop_req_i	logic	Input	[XtrigNumClkStopReq-1:0]	Clock stop requests. The SMU exposes DTP lanes above the SMC reservation (smu_pkg::XtrigSmcClkStopLanes).	Clock stop requesters	Tie to '0 if unused

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
xtrig_ctp_req_out_dout_o	logic	Output	[XtrigNumCtp-1:0]	CTP request-out data output.	GPIO pad ring	N/A
xtrig_ctp_req_out_dout_en_o	logic	Output	[XtrigNumCtp-1:0]	CTP request-out data output enable.	GPIO pad ring	N/A
xtrig_ctp_req_out_din_i	logic	Input	[XtrigNumCtp-1:0]	CTP request-out data input.	GPIO pad ring	Tie to '0 if unused
xtrig_ctp_req_out_din_en_o	logic	Output	[XtrigNumCtp-1:0]	CTP request-out data input enable.	GPIO pad ring	N/A
xtrig_ctp_req_in_dout_o	logic	Output	[XtrigNumCtp-1:0]	CTP request-in data output.	GPIO pad ring	N/A
xtrig_ctp_req_in_dout_en_o	logic	Output	[XtrigNumCtp-1:0]	CTP request-in data output enable.	GPIO pad ring	N/A
xtrig_ctp_req_in_din_i	logic	Input	[XtrigNumCtp-1:0]	CTP request-in data input.	GPIO pad ring	Tie to '0 if unused
xtrig_ctp_req_in_din_en_o	logic	Output	[XtrigNumCtp-1:0]	CTP request-in data input enable.	GPIO pad ring	N/A
xtrig_ctp_ack_in_dout_o	logic	Output	[XtrigNumCtp-1:0]	CTP ack-in data output.	GPIO pad ring	N/A
xtrig_ctp_ack_in_dout_en_o	logic	Output	[XtrigNumCtp-1:0]	CTP ack-in data output enable.	GPIO pad ring	N/A
xtrig_ctp_ack_in_din_i	logic	Input	[XtrigNumCtp-1:0]	CTP ack-in data input.	GPIO pad ring	Tie to '0 if unused
xtrig_ctp_ack_in_din_en_o	logic	Output	[XtrigNumCtp-1:0]	CTP ack-in data input enable.	GPIO pad ring	N/A
xtrig_ctp_ack_out_dout_o	logic	Output	[XtrigNumCtp-1:0]	CTP ack-out data output.	GPIO pad ring	N/A
xtrig_ctp_ack_out_dout_en_o	logic	Output	[XtrigNumCtp-1:0]	CTP ack-out data output enable.	GPIO pad ring	N/A
xtrig_ctp_ack_out_din_i	logic	Input	[XtrigNumCtp-1:0]	CTP ack-out data input.	GPIO pad ring	Tie to '0 if unused
xtrig_ctp_ack_out_din_en_o	logic	Output	[XtrigNumCtp-1:0]	CTP ack-out data input enable.	GPIO pad ring	N/A
rst_primary_ref_clk_no	logic	Output	1	Primary reset synchronized to reference clock domain (active low).	Downstream subsystems	N/A
rst_primary_smc_clk_no	logic	Output	1	Primary reset synchronized to SMC clock domain (active low).	Downstream subsystems	N/A
smu_axi_in_req_i	smu_axi_xbar_pkg::axi_56_64_req_t	Input	-	SMU AXI crossbar inbound request (from SMN).	SMN fabric	Tie to '0 if unused
smu_axi_in_resp_o	smu_axi_xbar_pkg::axi_56_64_resp_t	Output	-	SMU AXI crossbar inbound response.	SMN fabric	N/A
smu_axi_out_req_o	smu_axi_xbar_pkg::axi_out_req_t	Output	-	SMU AXI crossbar outbound request (to SMN).	SMN fabric	N/A
smu_axi_out_resp_i	smu_axi_xbar_pkg::axi_out_resp_t	Input	-	SMU AXI crossbar outbound response.	SMN fabric	Tie to '0 if unused
smc_external_req_o	smc_pkg::smc_axil_32_32_req_t	Output	-	SMC AXI-Lite external port request for adopter peripheral expansion.	Adopter peripherals	N/A
smc_external_resp_i	smc_pkg::smc_axil_32_32_resp_t	Input	-	SMC AXI-Lite external port response.	Adopter peripherals	Tie to DECERR if unused

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
smc_efuse_bank_ctrl_req_o	smc_pkg::smc_axil_32_32_req_t	Output	-	SMC eFuse bank control AXI-Lite request.	eFuse shim	N/A
smc_efuse_bank_ctrl_resp_i	smc_pkg::smc_axil_32_32_resp_t	Input	-	SMC eFuse bank control AXI-Lite response.	eFuse shim	N/A
smc_efuse_shim_command_req_o	smc_efuse_pkg::fuse_command_req_t	Output	-	SMC eFuse command interface request.	eFuse shim	N/A
smc_efuse_shim_command_resp_i	smc_efuse_pkg::fuse_command_resp_t	Input	-	SMC eFuse command interface response.	eFuse shim	N/A
smc_shadow_regs_o	smc_efuse_pkg::efuse_map_t	Output	-	SMC eFuse shadow register output.	External systems	N/A
pll_clk_obs_i	logic	Input	1	PLL clock observation input.	PLL shim	N/A
pll_clk_obs_en_i	logic	Input	1	PLL clock observation enable.	Control logic	N/A
pvt_process_clk_obs_i	logic	Input	1	PVT process monitor clock observation input.	PVT shim	N/A
pvt_process_clk_obs_en_i	logic	Input	1	PVT process monitor clock observation enable.	Control logic	N/A
cat_therm_i	logic	Input	1	CAT thermal sensor input.	CAT thermal sensor	Tie to '0 if unused
lsio_interface_select_o	logic	Output	[NumGpioWraps-1:0]	LSIO interface selection.	GPIO padring	N/A
pad2core_i	logic	Input	[NumGpioWraps-1:0]	GPIO input data from pad ring to core.	GPIO padring → SMU	N/A
core2pad_o	logic	Output	[NumGpioWraps-1:0]	GPIO output data from core to pad ring.	SMU → GPIO padring	N/A
pad2core_en_o	logic	Output	[NumGpioWraps-1:0]	GPIO pad-to-core path enable.	SMU → GPIO padring	N/A
core2pad_en_o	logic	Output	[NumGpioWraps-1:0]	GPIO core-to-pad path enable.	SMU → GPIO padring	N/A
rst_cool_n_from_pin_i	logic	Input	1	Cool reset input from GPIO pin.	GPIO pin	N/A
clk_telemetry_i	logic	Input	1	Telemetry clock domain for ATB data collection.	Telemetry clock source	N/A
rst_telemetry_ni	logic	Input	1	Telemetry reset (active low).	Telemetry reset source	N/A
telemetry_atdata_i	telemetry_receiver_pkg::telemetry_data_t	Input	[NumTelemetryReceivers-1:0]	ATB telemetry data input array.	ATB sources	N/A
telemetry_atid_i	telemetry_receiver_pkg::atb_id_t	Input	[NumTelemetryReceivers-1:0]	ATB telemetry ID input array.	ATB sources	N/A
telemetry_atready_o	logic	Output	[NumTelemetryReceivers-1:0]	ATB telemetry ready output.	ATB sources	N/A
telemetry_atvalid_i	logic	Input	[NumTelemetryReceivers-1:0]	ATB telemetry valid input.	ATB sources	N/A
telemetry_afvalid_o	logic	Output	[NumTelemetryReceivers-1:0]	ATB flush valid output.	ATB sources	N/A
telemetry_afready_i	logic	Input	[NumTelemetryReceivers-1:0]	ATB flush ready input.	ATB sources	N/A

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
smc_cluster_ded_o	logic	Output	1	Cluster double error detection output.	External systems	N/A
smc_wdt_first_timeout_o	logic	Output	1	First watchdog timeout signal.	Reset unit	N/A
smc_wdt_second_timeout_o	logic	Output	1	Second watchdog timeout signal.	External systems	N/A
smc_region_size_o	logic	Output	[31:0]	SMC address region size output.	External systems	N/A
smc_ext_interrupts_i	logic	Input	[CFG.NUM_INT_TO_SMC-1:0]	External interrupt inputs to SMC. Active-high level or a pulse long enough to be sampled on clk_smc.	External interrupt sources	Tie to '0 if unused
smc_fuse_sense_done_o	logic	Output	1	SMC fuse sense completion output.	Memory repair logic	N/A
smc_fuse_reset_n_delayed_o	logic	Output	1	Delayed fuse reset output.	External systems	N/A
skip_mem_repair_o	logic	Output	1	Skip memory repair output.	Memory repair logic	N/A
ext_boot_seq_done_i	logic	Input	1	Asynchronous external boot sequence completion (memory repair and shadow register override done). Connect only to SMC; qualified internally and fanned to SEP.	External boot/repair controller	N/A
lc_state_o	logic	Output	[2*LcStateWidth-1:0]	Lifecycle state output (driven by SEP when CFG.SEP=1, else 8'hf0).	External systems	N/A
lc_sigint_err_o	logic	Output	1	Lifecycle signal integrity error.	External systems	N/A
smc_ndmreset_request_i	logic	Input	[CpuClusterCount-1:0]	NDM reset request from external CPU cluster i on bit i.	External CPU clusters	Tie to '0 if unused
smc_ndmreset_process_o	logic	Output	[CpuClusterCount-1:0]	Firmware response to external CPU cluster i on bit i.	SMC → external CPU clusters	N/A
smc_ext_mailbox_interrupts_o	logic	Output	[NumMailboxes-1:0]	External mailbox interrupt outputs.	External targets	N/A
cfg_flr_pf_active_i	logic	Input	1	Function Level Reset (FLR) PF active signal.	PCIe / system	Tie to '0 if unused
isolate_req_o	logic	Output	[31:0]	Isolation request output for 32 subsystems.	Subsystems	N/A
ss_reset_complete_i	logic	Input	[31:0]	Subsystem reset completion status.	Subsystems	32'hFFFFFFFF
ss_config_o	logic	Output	[31:0]	Subsystem configuration output.	Subsystems	N/A
ss_reset_ctrl_o	smc_reset_unit_pkg::reset_ctrl_t	Output	[31:0]	Subsystem reset control output array.	Subsystems	N/A
sync_irq_o	logic	Output	1	Software-controlled global synchronisation bit (SYNC_REG.sync in the SMC reset unit). Not an interrupt aggregate output.	External systems	N/A
gpio_interrupt_o	logic	Output	[NumGpioWraps-1:0]	Raw GPIO interrupt vector from the padring, one bit per GPIO wrap (bonded + unbonded). Passed through from SMC. Software reads GPIO status registers to identify the interrupt source.	SMC → external consumers	N/A, leave floating if unused
uart_interrupt_o	logic	Output	[NumUart-1:0]	Raw UART interrupt outputs, one bit per UART instance, in the peripheral clock domain. Passed through from SMC.	SMC → external consumers	N/A, leave floating if unused

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
smc_rom_intf_req_o	rom_req_t	Output	-	SMC ROM memory interface request.	Boot ROM	N/A
smc_rom_intf_rsp_i	rom_rsp_t	Input	-	SMC ROM memory interface response.	Boot ROM	N/A
smc_scratch_ram_intf_req_o	scratch_ram_req_t	Output	[31:0]	SMC scratch RAM interface requests.	Scratch RAM banks	N/A
smc_scratch_ram_intf_rsp_i	scratch_ram_rsp_t	Input	[31:0]	SMC scratch RAM interface responses.	Scratch RAM banks	N/A
smc_l1_icache_tag_intf_req_o	l1_icache_tag_req_t	Output	[3:0]	SMC L1 I-cache tag interface requests.	I-cache tag banks	N/A
smc_l1_icache_tag_intf_rsp_i	l1_icache_tag_rsp_t	Input	[3:0]	SMC L1 I-cache tag interface responses.	I-cache tag banks	N/A
smc_l1_icache_data_intf_req_o	l1_icache_data_req_t	Output	[7:0]	SMC L1 I-cache data interface requests.	I-cache data banks	N/A
smc_l1_icache_data_intf_rsp_i	l1_icache_data_rsp_t	Input	[7:0]	SMC L1 I-cache data interface responses.	I-cache data banks	N/A
smc_l1_dcache_tag_intf_req_o	l1_dcache_tag_req_t	Output	[3:0]	SMC L1 D-cache tag interface requests.	D-cache tag banks	N/A
smc_l1_dcache_tag_intf_rsp_i	l1_dcache_tag_rsp_t	Input	[3:0]	SMC L1 D-cache tag interface responses.	D-cache tag banks	N/A
smc_l1_dcache_data_intf_req_o	l1_dcache_data_req_t	Output	[3:0]	SMC L1 D-cache data interface requests.	D-cache data banks	N/A
smc_l1_dcache_data_intf_rsp_i	l1_dcache_data_rsp_t	Input	[3:0]	SMC L1 D-cache data interface responses.	D-cache data banks	N/A
smc_disable_sram_auto_init_i	logic	Input	1	Disable SRAM automatic initialization.	Control logic	1’b0
smc_init_mem_done_o	logic	Output	1	Memory initialization completion signal.	Control logic	N/A
chiplet_is_primary_i	logic	Input	1	Chiplet is primary (for System Timer OCTS).	PRIMARY_CHIPLET configuration strap	N/A
timer_count_o	logic	Output	[63:0]	64-bit system timer count output (OCTS).	External systems	N/A
trace_mem_req_o	dfd_trace_mem_pkg::SinkMemPktIn_s	Output	[TRC_RAM_INSTANCES-1:0]	Trace memory interface requests.	Trace memory	N/A
trace_mem_resp_i	dfd_trace_mem_pkg::SinkMemPktOut_s	Input	[TRC_RAM_INSTANCES-1:0]	Trace memory interface responses.	Trace memory	N/A
test_en_i	logic	Input	1	DFT/scan test-enable. Drives clock-gater test ports and AXI cell test inputs.	Test controller	1’b0
scan_rst_ni	logic	Input	1	Scan reset (active-low). Bypasses reset synchronizers during DFT.	Test controller	1’b1
mem_repair_done_i	logic	Input	1	Memory repair done indicator.	DFT controller	1’b0
mem_repair_success_i	logic	Input	1	Memory repair success indicator.	DFT controller	1’b0
mem_repair_abort_i	logic	Input	1	Memory repair abort indicator.	DFT controller	1’b0
mbist_done_i	logic	Input	1	MBIST done indicator.	DFT controller	1’b0
mbist_pass_i	logic	Input	1	MBIST pass indicator.	DFT controller	1’b0
mbist_abort_i	logic	Input	1	MBIST abort indicator.	DFT controller	1’b0

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
sep_sram_req_o	sep_pkg::sep_sram_req_t	Output	-	SEP scratch SRAM request passthrough.	SEP SRAM macros	N/A
sep_sram_rsp_i	sep_pkg::sep_sram_rsp_t	Input	-	SEP scratch SRAM response passthrough.	SEP SRAM macros	N/A
sep_boot_rom_req_o	sep_pkg::sep_sram_req_t	Output	-	SEP boot ROM request passthrough.	SEP ROM macro	N/A
sep_boot_rom_rsp_i	sep_pkg::sep_sram_rsp_t	Input	-	SEP boot ROM response passthrough.	SEP ROM macro	N/A
sep_cpu_tcm_req_o	sep_pkg::sep_cpu_tcm_req_t	Output	-	SEP CPU TCM request passthrough (ICCM + DCCM).	SEP SRAM macros	N/A
sep_cpu_tcm_rsp_i	sep_pkg::sep_cpu_tcm_rsp_t	Input	-	SEP CPU TCM response passthrough.	SEP SRAM macros	N/A
sep_efuse_bank_ctrl_req_o	sep_efuse_pkg::efuse_axil_req_t	Output	-	SEP eFuse bank control AXI-Lite request passthrough.	SEP eFuse shim	N/A
sep_efuse_bank_ctrl_resp_i	sep_efuse_pkg::efuse_axil_resp_t	Input	-	SEP eFuse bank control AXI-Lite response passthrough.	SEP eFuse shim	N/A
sep_efuse_shim_command_req_o	sep_efuse_pkg::fuse_command_req_t	Output	-	SEP eFuse command interface request passthrough.	SEP eFuse shim	N/A
sep_efuse_shim_command_resp_i	sep_efuse_pkg::fuse_command_resp_t	Input	-	SEP eFuse command interface response passthrough.	SEP eFuse shim	N/A
sep_crypto_pka_imem_sram_req_o	sep_crypto_pkg::sep_crypto_pka_imem_sram_req_t	Output	-	SEP OTBN PKA instruction memory SRAM request passthrough.	SRAM macro	N/A
sep_crypto_pka_imem_sram_rsp_i	sep_crypto_pkg::sep_crypto_pka_imem_sram_rsp_t	Input	-	SEP OTBN PKA instruction memory SRAM response passthrough.	SRAM macro	N/A
sep_crypto_pka_dmem_sram_req_o	sep_crypto_pkg::sep_crypto_pka_dmem_sram_req_t	Output	-	SEP OTBN PKA data memory SRAM request passthrough.	SRAM macro	N/A
sep_crypto_pka_dmem_sram_rsp_i	sep_crypto_pkg::sep_crypto_pka_dmem_sram_rsp_t	Input	-	SEP OTBN PKA data memory SRAM response passthrough.	SRAM macro	N/A
sep_km_rom_mem_req_o	km_intf_pkg::km_rom_mem_req_t	Output	-	SEP Key Manager ROM memory request passthrough.	ROM macro	N/A
sep_km_rom_mem_rsp_i	km_intf_pkg::km_rom_mem_rsp_t	Input	-	SEP Key Manager ROM memory response passthrough.	ROM macro	N/A
sep_km_sram_mem_req_o	km_intf_pkg::km_sram_mem_req_t	Output	-	SEP Key Manager SRAM memory request passthrough.	SRAM macro	N/A
sep_km_sram_mem_rsp_i	km_intf_pkg::km_sram_mem_rsp_t	Input	-	SEP Key Manager SRAM memory response passthrough.	SRAM macro	N/A
sep_external_req_o	sep_pkg::sep_32_64_6_12_axi_req_t	Output	-	SEP AXI extension port request passthrough.	Adopter peripherals	N/A
sep_external_resp_i	sep_pkg::sep_32_64_6_12_axi_resp_t	Input	-	SEP AXI extension port response passthrough.	Adopter peripherals	Tie to DECERR if unused
sep_cpu_trace_o	sep_pkg::sep_cpu_trace_t	Output	-	SEP CPU trace output.	Trace infrastructure	N/A
sep_lockstep_ctrl_i	sep_pkg::sep_lockstep_ctrl_t	Input	-	SEP CPU lockstep controls, passed through to the SEP. Inert unless the SEP core is built with RV_LOCKSTEP_ENABLE.	Chip-level RAS control	Tie to '0 if unused
sep_lockstep_status_o	sep_pkg::sep_lockstep_status_t	Output	-	SEP CPU lockstep status. Reads 0 when the SEP is absent (CFG.SEP=0) or the core is built without RV_LOCKSTEP_ENABLE.	Chip-level RAS aggregation	N/A

Interface Name	Type	Direction	Width	Description	Connection	Default value (if applicable)
lcc_demote_state_1_o	logic	Output	[1:0]	SEP lifecycle demote state 1.	External lifecycle logic	N/A
lcc_demote_state_2_o	logic	Output	[1:0]	SEP lifecycle demote state 2.	External lifecycle logic	N/A
sep_fuse_sense_done_o	logic	Output	1	SEP fuse sense done output.	System integration	N/A
clk_sep_wdt_i	logic	Input	1	SEP watchdog timer clock passthrough.	Low-frequency clock source	N/A
secure_tm_req_i	logic	Input	-	Secure test mode request, passthrough to SEP.	GPIO padring / tie-off	N/A

System Management Network (SMN) Integration

Chapter 12. Status

WARNING This section is currently under development. Content will be added to describe SMN integration procedures, network topology, and management interfaces.

AXI-over-UCIe Bridge (AoU) Integration

Parameters, clocks, resets, and bring-up for the vendored AXI-over-UCIe bridge.

Chapter 13. Integration guide

References

1. AXI over UCIe (AoU) Protocol Specification, v0.7 (https://cdn.sanity.io/files/jpb4ed5r/production/728b2f8cfc09023466cf350db53764890b4f2343.pdf)
2. AXI over UCIe (AoU) Protocol Specification, v0.5
3. Universal Chiplet Interconnect Express (UCIe) Specification, Revision 3.0 (https://www.uciexpress.org/)
4. Arm AMBA AXI and ACE Protocol Specification (AXI4) (https://developer.arm.com)
5. Arm AMBA APB Protocol Specification (https://developer.arm.com)

Arm, AMBA, AXI, APB, and ACE are registered trademarks or trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. Universal Chiplet Interconnect Express (UCIe) is a trademark of the UCIe Consortium. All other trademarks are the property of their respective owners.

Integration Options

The AOU IP can be integrated at two levels. Choose the option that matches your system's FDI bringup strategy:

Option	Module	When to use	FDI bringup	Additional ports
A (recommended)	AOU_TOP	Turn-key FDI bringup control is desired	Handled internally by AOU_FDI_BRINGUP_CTRL	FDI bringup control, SW bringup control, bringup status
B	AOU_CORE_TOP	You have an external UCIe controller or custom FDI state machine	Must be handled externally	I_INT_FSM_IN_ACTIVE, O_AOU_ACTIVATE_ST_*, O_AOU_REQ_LINKRESET

> **Note:** All AXI, APB, FDI datapath, interrupt, and DFT interfaces are identical between AOU_TOP and AOU_CORE_TOP. The sections below that document these interfaces apply to both modules unless otherwise noted.

For detailed micro-architecture information — including datapath internals, error handling, debugging features, and credit management — refer to the *AOU_CORE Micro-Architecture Specification* (DOC/MAS/index.adoc).

Module Overview

AOU_TOP

AOU_TOP is the recommended top-level integration module. It instantiates two submodules:

- **AOU_FDI_BRINGUP_CTRL** — A UCIe 3.0 FDI state machine that manages the four FDI handshake pairs (wake, clock, state negotiation, RX activation) required to bring the FDI link from RESET to ACTIVE and handle teardown, LinkError recovery, Retrain, and L1 entry/exit.
- **AOU_CORE_TOP** — The AXI-over-UCIe protocol engine described in AOU_CORE_TOP.

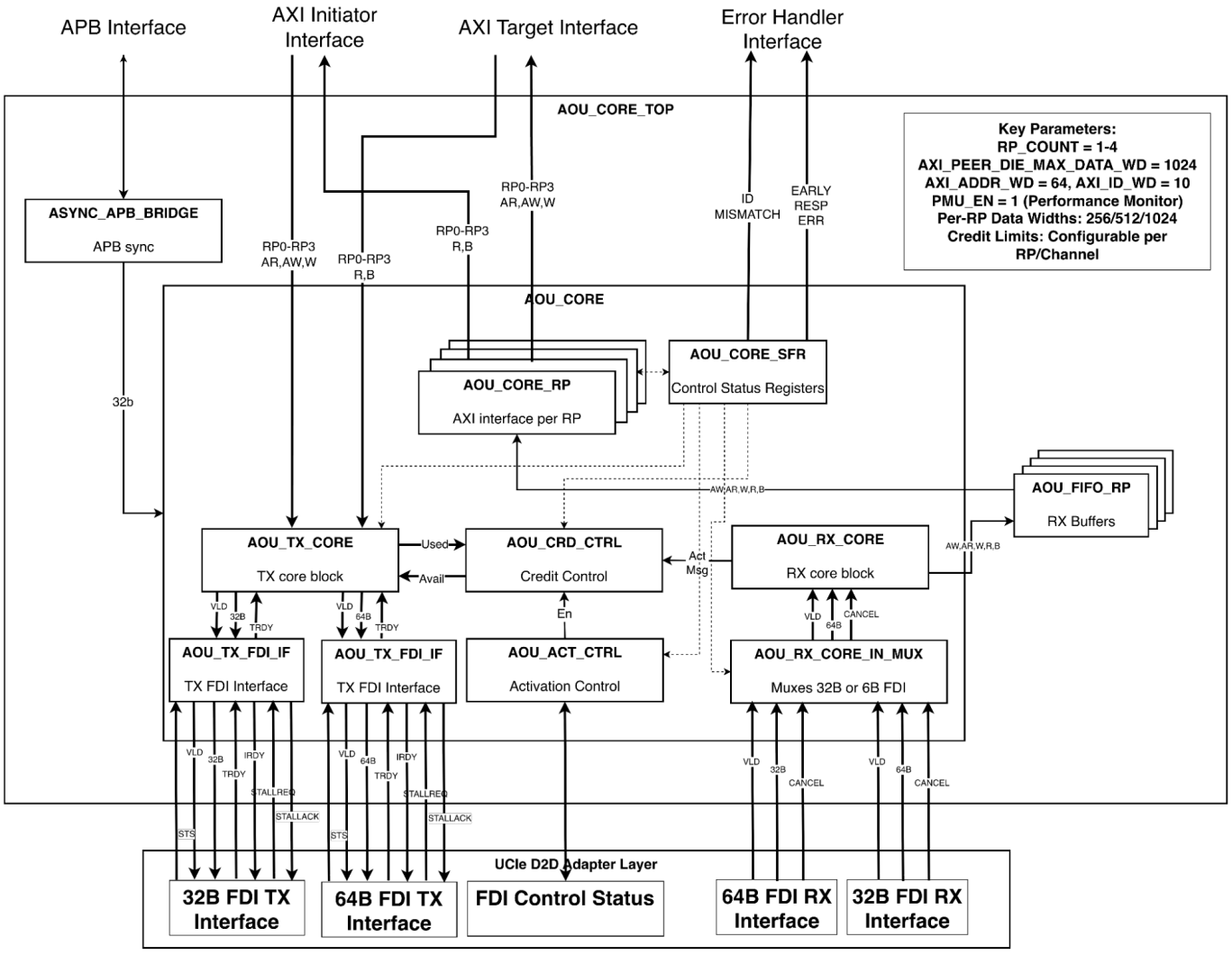
The bringup controller drives I_INT_FSM_IN_ACTIVE into the protocol engine internally, so this signal is not exposed as a top-level port on AOU_TOP. Instead, AOU_TOP exposes:

- FDI bringup control ports for the physical layer handshakes (FDI Bringup Control Interface (AOU_TOP only))
- Software bringup control pins (FDI Bringup Software Control (AOU_TOP only))
- Bringup status outputs (FDI Bringup Status (AOU_TOP only))

The FDI bringup flow is described in FDI Bringup Flow (AOU_TOP only).

AOU_CORE_TOP

Figure 1 shows high level block diagram of AOU CORE TOP.



Purpose and Function

The AOU_CORE_TOP is a bridge between AXI interface and the UCIe FDI interface, defined in AoU standard, AXI over UCIe Protocol Specification v0.7.

To achieve low latency, AOU_CORE directly handles signals related to the UCIe FDI interface data flow control and processes data in FDI-width chunks (32, 64, or 128 bytes, as selected by the FDI_CONFIG parameter) in a cut-through

manner, without converting them to 256-byte flit data. It receives AXI messages from its own AXI slave interface, packs them into FDI-width chunks, and transmits these chunks to the remote AoU device via UCIe. The remote AoU receives these FDI-width chunks from the UCIe FDI interface, unpacks them into AXI messages, and delivers the data through its AXI master interface. It supports flow control for all exceptional cases defined in the UCIe standard. First, it handles the chunk valid/cancel signal, including the alternative implementation method described in the UCIe specification. Second, it handles the stall request signal received from the D2D Adapter while maintaining the Flit-aligned boundary.

Interoperability is also supported when the remote device uses a different AXI data width. AOU_CORE supports a configurable number of Resource Plane(RPs), with both the number of RPs and each RP's AXI data width fully parameterizable. A dedicated field in the SFR allows software to configure the target RP, enabling flexible RP mapping. For each RP, the AW, W and AR channel support three arbitration modes: Round Robin, AXI QoS scheme, and Port QoS scheme. Additionally, a starvation-prevention mechanism is implemented to ensure fair arbitration across all RP channels.

AOU_CORE_TOP implements the subset of FDI signals required specifically for transporting AXI across UCIe. FDI bringup/teardown and FDI sideband flows are expected to be handled by other components.

The block consists of the following sub blocks

- AOU_CORE
  - AOU_CORE_RP Manages a AXI interface for write request, read request, write data, read data and write response busses
  - AOU_CORE_SFR This implements control and status register of the AOU core. It has a 32bit APB interface for programming. It configures all the blocks in the core.
  - AOU_TX_CORE This block packs the arbitrated AXI channels, credits, and activation messages into FDI-width chunks (32, 64, or 128 bytes per FDI_CONFIG). It follows AOU specifications to pack the messages into 256 byte flit data. It then passes it onto UCIe controller over the FDI interface for transmission.
  - AOU_CRD_CTRL This block manages credits available for TX. It gets remote credits available from AOU_RX_CORE, and used credits from AOU_TX_CORE and creates final credits that can be used for TX.
  - AOU_RX_CORE This block processes FDI-width chunks (32, 64, or 128 bytes per FDI_CONFIG) received by UCIe controller and passed to the core over the FDI interface. It decodes different message types which encapsulate different AXI channel transactions. These messages are buffered and then passed onto AOU_CORE_RP for presenting the transaction to its corresponding AXI interface.
  - AOU_TX_FDI_IF This block receives FDI-width data chunks from AOU_TX_CORE and buffers them. It does handshake with UCIe controller FDI RX interface and passes the data chunks downstream to be sent to remote chiplet.
  - AOU_ACT_CTRL This block manages activation and deactivation control of the AOU bridge. It initiates activate and deactivate messages to remote AOU bridge when configured via APB.
- AOU_FIFO_RP This block buffers decoded messages from AOU_RX_CORE. It separates the messages based on resource plane (RP) indicator and writes them into dedicated FIFOs. These FIFOs feed the corresponding AOU_CORE_RP which manage a single AXI interface.
- ASYNC_APB_BRIDGE This block synchronizes APB transactions to the local core clock.

When using AOU_CORE_TOP directly, the integrator must provide an external FDI state machine that drives I_INT_FSM_IN_ACTIVE and the associated control/status signals listed in UCIe / External Control and Status.

Key Features

- Single APB interface Configuration/control/status register access
- Multiple AXI master & slave interface
  - Configurable RP port & RP remapping supported
  - Supports variable data widths and burst length requests. Configurable FIFO Depth
- UCIe 256B latency-optimized flit(Format 6) support only
- Message packing and unpacking are handled in variable-sized units of 32, 64, or 128 bytes, corresponding to the FDI data width selected by FDI_CONFIG.
- Selectable Early response by setting SFR8
- Selectable 32B / 64B / 128B single-PHY and 32B+64B / 64B+128B two-PHY FDI configurations via the FDI_CONFIG parameter (see Parameter List). Two-PHY variants additionally require the +define+TWO_PHY build flag so the second-PHY ports physically exist.
- Parameterized number of Resource Plane and each RP's AXI data width
- RP Remapping
- QoS scheme between RP with AW, W, AR channel
- Interoperability with different data width remote die
- Handling of flit cancel and stall request as specified in the UCIe specification
- Selectable AXI Aggregator added for BUS efficiency

Parameter List

Parameter	Type	Default / Note	Description
RP_COUNT	parameter	1	Number of receive ports (AXI M/S pairs). Parameterizable up to 4.
FDI_CONFIG	parameter (int)	FDI_CFG_SP_32B	FDI width selector. See *FDI_CONFIG values* below. Two-PHY values additionally require +define+TWO_PHY at compile time.
RP0_RX_AW_FIFO_DEPTH	parameter	44	RX AW FIFO depth for RP0. The depth of each RP*_RX_*_FIFO_DEPTH parameter should be determined by considering the Turn-Around-Time (TAT) between the local die and the remote die, so that no performance drop occurs due to AoU — TAT being the time, in a UCIe-integrated environment, from when the local die sends a message until the credit is received back from the remote die.
RP0_RX_AR_FIFO_DEPTH	parameter	44	RX AR FIFO depth for RP0.
RP0_RX_W_FIFO_DEPTH	parameter	88 / 140	RX W FIFO depth for RP0. Default is 140 when FDI_CONFIG selects a 1024b interface (FDI_CFG_SP_128B / FDI_CFG_TP_64B_128B); 88 otherwise.

Parameter	Type	Default / Note	Description
RP0_RX_R_FIFO_DEPTH	parameter	88 / 140	RX R FIFO depth for RP0. Same FDI-aware default as RP0_RX_W_FIFO_DEPTH.
RP0_RX_B_FIFO_DEPTH	parameter	44	RX B FIFO depth for RP0.
RP1_RX_AW_FIFO_DEPTH	parameter	44	RX AW FIFO depth for RP1.
RP1_RX_AR_FIFO_DEPTH	parameter	44	RX AR FIFO depth for RP1.
RP1_RX_W_FIFO_DEPTH	parameter	88 / 140	RX W FIFO depth for RP1 (FDI-aware default; see RP0).
RP1_RX_R_FIFO_DEPTH	parameter	88 / 140	RX R FIFO depth for RP1 (FDI-aware default; see RP0).
RP1_RX_B_FIFO_DEPTH	parameter	44	RX B FIFO depth for RP1.
RP2_RX_AW_FIFO_DEPTH	parameter	44	RX AW FIFO depth for RP2.
RP2_RX_AR_FIFO_DEPTH	parameter	44	RX AR FIFO depth for RP2.
RP2_RX_W_FIFO_DEPTH	parameter	88 / 140	RX W FIFO depth for RP2 (FDI-aware default; see RP0).
RP2_RX_R_FIFO_DEPTH	parameter	88 / 140	RX R FIFO depth for RP2 (FDI-aware default; see RP0).
RP2_RX_B_FIFO_DEPTH	parameter	44	RX B FIFO depth for RP2.
RP3_RX_AW_FIFO_DEPTH	parameter	44	RX AW FIFO depth for RP3.
RP3_RX_AR_FIFO_DEPTH	parameter	44	RX AR FIFO depth for RP3.
RP3_RX_W_FIFO_DEPTH	parameter	88 / 140	RX W FIFO depth for RP3 (FDI-aware default; see RP0).
RP3_RX_R_FIFO_DEPTH	parameter	88 / 140	RX R FIFO depth for RP3 (FDI-aware default; see RP0).
RP3_RX_B_FIFO_DEPTH	parameter	44	RX B FIFO depth for RP3.
RX_AW_FIFO_RS_EN	parameter	1	When 1, inserts an output register-slice on the RX AW FIFO read port. Trades latency (+1 cycle) for timing closure.
RX_AR_FIFO_RS_EN	parameter	1	RX AR FIFO output register-slice enable (see RX_AW_FIFO_RS_EN).
RX_W_FIFO_RS_EN	parameter	1	RX W FIFO output register-slice enable (see RX_AW_FIFO_RS_EN).
RX_R_FIFO_RS_EN	parameter	1	RX R FIFO output register-slice enable (see RX_AW_FIFO_RS_EN).
RX_B_FIFO_RS_EN	parameter	1	RX B FIFO output register-slice enable (see RX_AW_FIFO_RS_EN).
RP0_AXI_DATA_WD	parameter	512	AXI data width for RP0 (bits).
RP1_AXI_DATA_WD	parameter	512	AXI data width for RP1 (bits).
RP2_AXI_DATA_WD	parameter	512	AXI data width for RP2 (bits).
RP3_AXI_DATA_WD	parameter	512	AXI data width for RP3 (bits).
AXI_PEER_DIE_MAX_DATA_WD	parameter	1024	Maximum peer data width (bits).
APB_ADDR_WD	parameter	32	APB address width.
APB_DATA_WD	parameter	32	APB data width.
S_RD_MO_CNT	parameter	32	Slave read outstanding count.
S_WR_MO_CNT	parameter	32	Slave write outstanding count.
M_RD_MO_CNT	parameter	32	Master read outstanding count.

Parameter	Type	Default / Note	Description
M_WR_MO_CNT	parameter	32	Master write outstanding count.

FDI_CONFIG values

FDI_CONFIG is a single integer parameter that selects the FDI data width pair forwarded to the internal hierarchy. Named constants are declared in RTL/packet_def_pkg.sv and should be used at the instantiation site:

FDI_CONFIG value	PHY0 width (FDI_IF_WD0)	PHY1 width (FDI_IF_WD1)	PHY count	Build flag
FDI_CFG_SP_32B (default)	256	512 (internal decode)	Single PHY	none
FDI_CFG_SP_64B	512	512 (internal decode)	Single PHY	none
FDI_CFG_SP_128B	1024	1024 (internal decode)	Single PHY	none
FDI_CFG_TP_32B_64B	256	512	Two PHY	+define+TWO_PHY
FDI_CFG_TP_64B_128B	512	1024	Two PHY	+define+TWO_PHY

Notes:

- For single-PHY values, only the PHY0 ports (I_FDI_PL_0_* / O_FDI_LP_0_*) carry traffic. The PHY1 internal width is used only as the RX-side decode width.
- The two-PHY values (FDI_CFG_TP_*) are valid **only** when the design is built with +define+TWO_PHY. The build flag gates the physical presence of the PHY1 ports (I_FDI_PL_1_*, O_FDI_LP_1_*) and the I_PHY_TYPE selector on both AOU_TOP and AOU_CORE_TOP. Mixing a FDI_CFG_TP_* value with a build that omits +define+TWO_PHY (or vice-versa) is an integration error; the design does **not** enforce this consistency at elaboration time.
- The FDI_CFG_TP_32B_128B (256b + 1024b) combination is **not** supported by the design: AOU_RX_CORE_IN_MUX requires FDI_IF_WD0 == FDI_IF_WD1 / 2 for two-PHY accumulation.

Build flags

Flag	Effect
+define+TWO_PHY	Compile-time switch that adds the second-PHY ports (I_FDI_PL_1_*, O_FDI_LP_1_*) and the I_PHY_TYPE selector to both AOU_TOP and AOU_CORE_TOP. Required for FDI_CFG_TP_* values of FDI_CONFIG.

Interface Signals

Clock and Reset

Signal	Direction	Width	Description
I_CLK	input	1	Core system clock (AOU_CORE and AOU_FIFO_RP).
I_RESETN	input	1	Active-low asynchronous reset.
I_PCLK	input	1	APB clock (slave side of async bridge).
I_PRESETN	input	1	APB domain reset.

APB Slave Interface

Signal	Direction	Width	Description
I_AOU_APB_SI0_PSEL	input	1	APB select.
I_AOU_APB_SI0_PENABLE	input	1	APB enable.
I_AOU_APB_SI0_PADDR	input	APB_ADDR_WD	APB address.
I_AOU_APB_SI0_PWRITE	input	1	APB write.
I_AOU_APB_SI0_PWDATA	input	APB_DATA_WD	APB write data.
O_AOU_APB_SI0_PRDATA	output	APB_DATA_WD	APB read data.
O_AOU_APB_SI0_PREADY	output	1	APB ready.
O_AOU_APB_SI0_PSLVERR	output	1	APB slave error.

AXI Master (RX) – AR Channel

Signal	Direction	Width	Description
O_AOU_RX_AXI_M_ARID	output	[RP_COUNT-1:0][AXI_ID_WD-1:0]	Read address ID.
O_AOU_RX_AXI_M_ARADDR	output	[RP_COUNT-1:0][AXI_ADDR_WD-1:0]	Read address.
O_AOU_RX_AXI_M_ARLEN	output	[RP_COUNT-1:0][AXI_LEN_WD-1:0]	Burst length.
O_AOU_RX_AXI_M_ARSIZE	output	[RP_COUNT-1:0][2:0]	Transfer size.
O_AOU_RX_AXI_M_ARBURST	output	[RP_COUNT-1:0][1:0]	Burst type.
O_AOU_RX_AXI_M_ARLOCK	output	[RP_COUNT-1:0]	Lock.
O_AOU_RX_AXI_M_ARCACHE	output	[RP_COUNT-1:0][3:0]	Cache attributes.
O_AOU_RX_AXI_M_ARPROT	output	[RP_COUNT-1:0][2:0]	Protection.
O_AOU_RX_AXI_M_ARQOS	output	[RP_COUNT-1:0][3:0]	QoS.
O_AOU_RX_AXI_M_ARVALID	output	[RP_COUNT-1:0]	AR valid.
I_AOU_RX_AXI_M_ARREADY	input	[RP_COUNT-1:0]	AR ready.

AXI Master (RX) – R Channel

Signal	Direction	Width	Description
I_AOU_TX_AXI_M_RID	input	[RP_COUNT-1:0][AXI_ID_WD-1:0]	Read data ID.
I_AOU_TX_AXI_M_RDATA	input	[RP_COUNT-1:0][RP_AXI_DATA_WD_MAX-1:0]	Read data.
I_AOU_TX_AXI_M_RRESP	input	[RP_COUNT-1:0][1:0]	Read response.
I_AOU_TX_AXI_M_RLAST	input	[RP_COUNT-1:0]	Read last.
I_AOU_TX_AXI_M_RVALID	input	[RP_COUNT-1:0]	R valid.
O_AOU_TX_AXI_M_RREADY	output	[RP_COUNT-1:0]	R ready.

AXI Master (RX) – AW Channel

Signal	Direction	Width	Description
O_AOU_RX_AXI_M_AWID	output	[RP_COUNT-1:0][AXI_ID_WD-1:0]	Write address ID.
O_AOU_RX_AXI_M_AWADDR	output	[RP_COUNT-1:0][AXI_ADDR_WD-1:0]	Write address.
O_AOU_RX_AXI_M_AWLEN	output	[RP_COUNT-1:0][AXI_LEN_WD-1:0]	Burst length.

Signal	Direction	Width	Description
O_AOU_RX_AXI_M_AWSIZE	output	[RP_COUNT-1:0][2:0]	Transfer size.
O_AOU_RX_AXI_M_AWBURST	output	[RP_COUNT-1:0][1:0]	Burst type.
O_AOU_RX_AXI_M_AWLOCK	output	[RP_COUNT-1:0]	Lock.
O_AOU_RX_AXI_M_AWCACHE	output	[RP_COUNT-1:0][3:0]	Cache attributes.
O_AOU_RX_AXI_M_AWPROT	output	[RP_COUNT-1:0][2:0]	Protection.
O_AOU_RX_AXI_M_AWQOS	output	[RP_COUNT-1:0][3:0]	QoS.
O_AOU_RX_AXI_M_AWVALID	output	[RP_COUNT-1:0]	AW valid.
I_AOU_RX_AXI_M_AWREADY	input	[RP_COUNT-1:0]	AW ready.

AXI Master (RX) – W and B Channels

Signal	Direction	Width	Description
O_AOU_RX_AXI_M_WDATA	output	[RP_COUNT-1:0][RP_AXI_DATA_WD_MAX-1:0]	Write data.
O_AOU_RX_AXI_M_WSTRB	output	[RP_COUNT-1:0][RP_AXI_STRB_WD_MAX-1:0]	Write strobe.
O_AOU_RX_AXI_M_WLAST	output	[RP_COUNT-1:0]	Write last.
O_AOU_RX_AXI_M_WVALID	output	[RP_COUNT-1:0]	W valid.
I_AOU_RX_AXI_M_WREADY	input	[RP_COUNT-1:0]	W ready.
I_AOU_TX_AXI_M_BID	input	[RP_COUNT-1:0][AXI_ID_WD-1:0]	Write response ID.
I_AOU_TX_AXI_M_BRESP	input	[RP_COUNT-1:0][1:0]	Write response.
I_AOU_TX_AXI_M_BVALID	input	[RP_COUNT-1:0]	B valid.
O_AOU_TX_AXI_M_BREADY	output	[RP_COUNT-1:0]	B ready.

AXI Slave (TX) – AR Channel

Signal	Direction	Width	Description
I_AOU_TX_AXI_S_ARID	input	[RP_COUNT-1:0][AXI_ID_WD-1:0]	Read address ID.
I_AOU_TX_AXI_S_ARADDR	input	[RP_COUNT-1:0][AXI_ADDR_WD-1:0]	Read address.
I_AOU_TX_AXI_S_ARLEN	input	[RP_COUNT-1:0][AXI_LEN_WD-1:0]	Burst length.
I_AOU_TX_AXI_S_ARSIZE	input	[RP_COUNT-1:0][2:0]	Transfer size.
I_AOU_TX_AXI_S_ARBURST	input	[RP_COUNT-1:0][1:0]	Burst type.
I_AOU_TX_AXI_S_ARLOCK	input	[RP_COUNT-1:0]	Lock.
I_AOU_TX_AXI_S_ARCACHE	input	[RP_COUNT-1:0][3:0]	Cache attributes.
I_AOU_TX_AXI_S_ARPROT	input	[RP_COUNT-1:0][2:0]	Protection.
I_AOU_TX_AXI_S_ARQOS	input	[RP_COUNT-1:0][3:0]	QoS.
I_AOU_TX_AXI_S_ARVALID	input	[RP_COUNT-1:0]	AR valid.
O_AOU_TX_AXI_S_ARREADY	output	[RP_COUNT-1:0]	AR ready.

AXI Slave (TX) – R Channel

Signal	Direction	Width	Description
O_AOU_RX_AXI_S_RID	output	[RP_COUNT-1:0][AXI_ID_WD-1:0]	Read data ID.

Signal	Direction	Width	Description
O_AOU_RX_AXI_S_RDATA	output	[RP_COUNT-1:0][RP_AXI_DATA_WD_MAX-1:0]	Read data.
O_AOU_RX_AXI_S_RRESP	output	[RP_COUNT-1:0][1:0]	Read response.
O_AOU_RX_AXI_S_RLAST	output	[RP_COUNT-1:0]	Read last.
O_AOU_RX_AXI_S_RVALID	output	[RP_COUNT-1:0]	R valid.
I_AOU_RX_AXI_S_RREADY	input	[RP_COUNT-1:0]	R ready.

AXI Slave (TX) – AW, W, B Channels

Signal	Direction	Width	Description
I_AOU_TX_AXI_S_AWID	input	[RP_COUNT-1:0][AXI_ID_WD-1:0]	Write address ID.
I_AOU_TX_AXI_S_AWADDR	input	[RP_COUNT-1:0][AXI_ADDR_WD-1:0]	Write address.
I_AOU_TX_AXI_S_AWLEN	input	[RP_COUNT-1:0][AXI_LEN_WD-1:0]	Burst length.
I_AOU_TX_AXI_S_AWSIZE	input	[RP_COUNT-1:0][2:0]	Transfer size.
I_AOU_TX_AXI_S_AWBURST	input	[RP_COUNT-1:0][1:0]	Burst type.
I_AOU_TX_AXI_S_AWLOCK	input	[RP_COUNT-1:0]	Lock.
I_AOU_TX_AXI_S_AWCACHE	input	[RP_COUNT-1:0][3:0]	Cache attributes.
I_AOU_TX_AXI_S_AWPROT	input	[RP_COUNT-1:0][2:0]	Protection.
I_AOU_TX_AXI_S_AWQOS	input	[RP_COUNT-1:0][3:0]	QoS.
I_AOU_TX_AXI_S_AWVALID	input	[RP_COUNT-1:0]	AW valid.
O_AOU_TX_AXI_S_AWREADY	output	[RP_COUNT-1:0]	AW ready.
I_AOU_TX_AXI_S_WDATA	input	[RP_COUNT-1:0][RP_AXI_DATA_WD_MAX-1:0]	Write data.
I_AOU_TX_AXI_S_WSTRB	input	[RP_COUNT-1:0][RP_AXI_STRB_WD_MAX-1:0]	Write strobe.
I_AOU_TX_AXI_S_WLAST	input	[RP_COUNT-1:0]	Write last.
I_AOU_TX_AXI_S_WVALID	input	[RP_COUNT-1:0]	W valid.
O_AOU_TX_AXI_S_WREADY	output	[RP_COUNT-1:0]	W ready.
O_AOU_RX_AXI_S_BID	output	[RP_COUNT-1:0][AXI_ID_WD-1:0]	Write response ID.
O_AOU_RX_AXI_S_BRESP	output	[RP_COUNT-1:0][1:0]	Write response.
O_AOU_RX_AXI_S_BVALID	output	[RP_COUNT-1:0]	B valid.
I_AOU_RX_AXI_S_BREADY	input	[RP_COUNT-1:0]	B ready.

PHY and FDI (Flit Data Interface)

The FDI port set is split into two banks: PHY0 (always present) and PHY1 (present only when the design is built with +define+TWO_PHY). Both banks use the parameterized widths derived from FDI_CONFIG (see FDI_CONFIG values).

FDI_CONFIG	FDI_IF_WD0 (PHY0 width)	FDI_IF_WD1 (PHY1 width)	PHY1 ports present
FDI_CFG_SP_32B	256	512 (internal decode)	no
FDI_CFG_SP_64B	512	512 (internal decode)	no
FDI_CFG_SP_128B	1024	1024 (internal decode)	no
FDI_CFG_TP_32B_64B	256	512	yes (+define+TWO_PHY)
FDI_CFG_TP_64B_128B	512	1024	yes (+define+TWO_PHY)

PHY0 (always present)

Signal	Direction	Width	Description
I_FDI_PL_0_VALID	input	1	PHY0 flit valid from PHY.
I_FDI_PL_0_DATA	input	FDI_IF_WD0	PHY0 flit data.
I_FDI_PL_0_FLIT_CANCEL	input	1	PHY0 flit cancel.
I_FDI_PL_0_TRDY	input	1	PHY0 ready.
I_FDI_PL_0_STALLREQ	input	1	PHY0 stall request.
I_FDI_PL_0_STATE_STS	input	[3:0]	PHY0 state status.
O_FDI_LP_0_DATA	output	FDI_IF_WD0	PHY0 flit data to PHY.
O_FDI_LP_0_VALID	output	1	PHY0 flit valid.
O_FDI_LP_0_IRDY	output	1	PHY0 link ready.
O_FDI_LP_0_STALLACK	output	1	PHY0 stall acknowledge.

PHY1 (present only when +define+TWO_PHY)

Signal	Direction	Width	Description
I_FDI_PL_1_VALID	input	1	PHY1 flit valid from PHY.
I_FDI_PL_1_DATA	input	FDI_IF_WD1	PHY1 flit data.
I_FDI_PL_1_FLIT_CANCEL	input	1	PHY1 flit cancel.
I_FDI_PL_1_TRDY	input	1	PHY1 ready.
I_FDI_PL_1_STALLREQ	input	1	PHY1 stall request.
I_FDI_PL_1_STATE_STS	input	[3:0]	PHY1 state status.
O_FDI_LP_1_DATA	output	FDI_IF_WD1	PHY1 flit data to PHY.
O_FDI_LP_1_VALID	output	1	PHY1 flit valid.
O_FDI_LP_1_IRDY	output	1	PHY1 link ready.
O_FDI_LP_1_STALLACK	output	1	PHY1 stall acknowledge.

PHY type select (present only when +define+TWO_PHY)

Signal	Direction	Width	Description
I_PHY_TYPE	input	1	PHY selector for state-status / stall-request multiplexing: 0 = PHY0, 1 = PHY1. See PHY Type Mux.

Interrupt and Error (to Error Handler)

Signal	Direction	Width	Description
INT_REQ_LINKRESET	output	1	Link reset request.
INT_SI0_ID_MISMATCH	output	1	Slave interface ID mismatch.
INT_MI0_ID_MISMATCH	output	1	Master interface ID mismatch.
INT_EARLY_RESP_ERR	output	1	Early response error.
INT_ACTIVATE_START	output	1	Activate start.
INT_DEACTIVATE_START	output	1	Deactivate start.

UCIe / External Control and Status

> **AOU_TOP users:** The signals in this table are present on AOU_CORE_TOP only. When integrating via AOU_TOP, these signals are consumed/generated internally by AOU_FDI_BRINGUP_CTRL and are not exposed as top-level ports. See AOU_CORE_TOP-only Ports (not on AOU_TOP).

Signal	Direction	Width	Description
I_INT_FSM_IN_ACTIVE	input	1	Initialization done / FSM in active state indicator.
I_MST_BUS_CLEANY_COMPLETE	input	1	Master bus cleany complete.
I_SLV_BUS_CLEANY_COMPLETE	input	1	Slave bus cleany complete.
O_AOU_ACTIVATE_ST_DISABLED	output	1	Activation state disabled.
O_AOU_ACTIVATE_ST_ENABLED	output	1	Activation state enabled.
O_AOU_REQ_LINKRESET	output	1	AOU link reset request.

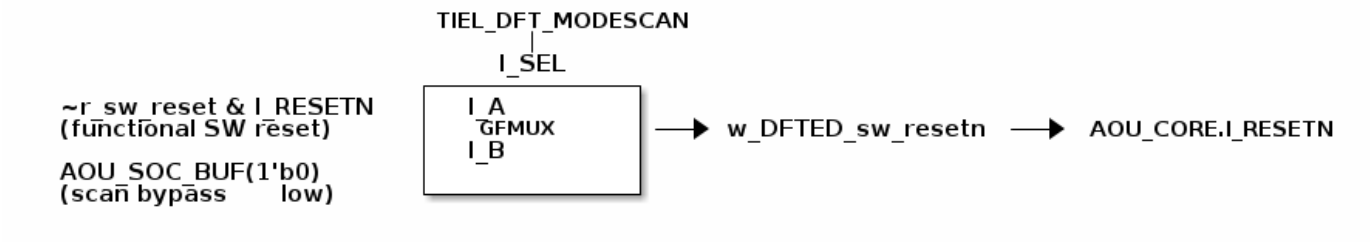
- I_INT_FSM_IN_ACTIVE is an input signal that indicates that the link up is complete. When starting the test, I_INT_FSM_IN_ACTIVE should be 0, and when you want to activate AOU_CORE, I_INT_FSM_IN_ACTIVE should be 1. When testing with IP standalone, you must force I_INT_FSM_IN_ACTIVE.
- I_MST_BUS_CLEANY_COMPLETE is asserted when all responses to the remote die's requests have been completed. The signal originates from the BUS_CLEANY. If the BUS_CLEANY feature is not used, the corresponding port must be tied to 1.
- I_SLV_BUS_CLEANY_COMPLETE is asserted when all responses from the remote die have been received for the requests sent by the local die. The signal originates from the BUS_CLEANY. If the BUS_CLEANY feature is not used, the corresponding port must be tied to 1.
- After AOU_CORE deactivation, the activation status transitions to DISABLED, allowing the UCIe link to go down. Once deactivation completes, O_AOU_ACTIVATE_ST_DISABLED is asserted to notify the UCIE_CORE.
- When the AOU_CORE is in the ENABLED state, O_AOU_ACTIVATE_ST_ENABLED is asserted to notify the UCIE_CORE.
- O_AOU_REQ_LINKRESET is asserted together with INT_REQ_LINKRESET when the AOU_CORE detects an AoU protocol violation, to notify the UCIE_CORE.

DFT

Signal	Direction	Width	Description
TIEL_DFT_MODESCAN	input	1	Scan-mode indicator. Tie low in functional operation; assert high from the SoC scan controller during scan shift and capture.

TIEL_DFT_MODESCAN Usage

TIEL_DFT_MODESCAN controls a glitch-free mux (u_sw_reset_mux, instance of AOU_SOC_GFMUX_LVT) that selects the reset source for AOU_CORE:



- **Functional mode** (TIEL_DFT_MODESCAN = 0): The mux selects I_A, the functional software-controlled reset path (~r_sw_reset & I_RESETN). This allows the CSR-writable SW reset to combine with the hardware reset.
- **Scan mode** (TIEL_DFT_MODESCAN = 1): The mux selects I_B, a constant 1'b0 driven through AOU_SOC_BUF (dft_persistent_buf_scan_resetn_sw). This value de-asserts the active-low SW reset, neutralizing it so that scan chains can shift freely without the software reset logic interfering with controllability or observability.

The AOU_SOC_BUF instance exists solely to prevent the synthesis tool from optimizing away the tied-low constant; it must be preserved as a persistent buffer (see Library Cell Replacement).

The SDC file (INTEG/constraints/aou_core_top.sdc) constrains this pin with set_case_analysis 0 in functional timing mode.

Integrator DFT Responsibilities

The design provides internal DFT muxing only for the software-controlled reset path (w_DFTED_sw_resetn). The two primary hardware resets and all scan infrastructure are the integrator's responsibility:

Responsibility	Detail
HW reset DFT muxes	I_RESETN and I_PRESETN have no internal scan-mode bypass. The integrator must add external DFT muxes (or equivalent) on both resets so they can be held inactive (high) during scan shift and capture. Without this, all flip-flops using these resets will be uncontrollable during ATPG.
Scan chain insertion	The design has no SCAN_EN, SCAN_IN, or SCAN_OUT ports. The integrator must insert scan chains via the synthesis tool's DFT insertion flow or by adding scan ports manually.
Multi-clock scan definition	I_CLK (1 GHz core) and I_PCLK (100 MHz APB) must be defined as separate scan clocks. The ATPG tool must not shift both domains simultaneously, or lockup latches must be inserted at domain boundaries.
I/O constraints during scan	AXI and FDI interface ports should be constrained to safe values during scan (via wrapper cells or top-level scan constraints) to prevent unintended activity on external buses.
CDC synchronizer handling	The AOU_SOC_SYNCHSR instances in the ASYNC_APB_BRIDGE require scan-safe treatment. See Library Cell Replacement for replacement cell requirements.

FDI Bringup Control Interface (AOU_TOP only)

These signals connect AOU_TOP to the D2D adapter's physical layer for FDI bringup control. They are not present on AOU_CORE_TOP.

Signal	Direction	Width	Description
I_PL_INBAND_PRES	input	1	Inband presence detect from PL. The bringup FSM waits for this before initiating the wake handshake.

Signal	Direction	Width	Description
I_PL_CLK_REQ	input	1	PL clock request. LP responds with O_LP_CLK_ACK.
I_PL_WAKE_ACK	input	1	PL acknowledgement of LP wake request.
I_PL_RX_ACTIVE_REQ	input	1	PL requests RX activation. LP responds with O_LP_RX_ACTIVE_STS.
O_LP_STATE_REQ	output	[3:0]	LP state request (RESET=0, ACTIVE=1, LinkError=2, Retrain=3, L1=4).
O_LP_WAKE_REQ	output	1	LP-initiated wake request.
O_LP_CLK_ACK	output	1	LP acknowledgement of PL clock request.
O_LP_RX_ACTIVE_STS	output	1	LP RX active status in response to I_PL_RX_ACTIVE_REQ.

The four handshake pairs correspond to UCIe 3.0 FDI bringup control signals:

Handshake	LP signal	PL signal	Purpose
Wake	O_LP_WAKE_REQ	I_PL_WAKE_ACK	LP-initiated link wake from low-power
Clock	O_LP_CLK_ACK	I_PL_CLK_REQ	PL-initiated clock request
State	O_LP_STATE_REQ	(via I_FDI_PL_*_STATE_STS)	State negotiation
RX Active	O_LP_RX_ACTIVE_STS	I_PL_RX_ACTIVE_REQ	RX path activation

FDI Bringup Software Control (AOU_TOP only)

These top-level ports provide direct software control over the FDI bringup FSM. They are **not SFR-mapped**; the integrator must drive them from external logic or tie them as needed.

Signal	Direction	Width	Description
I_SW_ACTIVATE_START	input	1	Trigger FDI bringup (RESET to ACTIVE). ORed internally with the core's INT_ACTIVATE_START.
I_SW_DEACTIVATE_START	input	1	Trigger FDI teardown. ORed internally with the core's INT_DEACTIVATE_START.
I_SW_RETRAIN_REQ	input	1	Request link retrain (LP-initiated).
I_SW_LINKERROR_INJECT	input	1	Force the bringup FSM into LinkError state for debug/test.

FDI Bringup Status (AOU_TOP only)

Signal	Direction	Width	Description
O_FDI_FSM_STATE	output	[3:0]	Current FDI state. Encoding matches UCIe 3.0: RESET=0, ACTIVE=1, LinkError=2, Retrain=3, L1=4, L2=5, Disabled=6.
O_FDI_LINK_UP	output	1	Asserted when the FDI bringup FSM is in the ACTIVE state.

AOU_CORE_TOP-only Ports (not on AOU_TOP)

The following signals are present on AOU_CORE_TOP but are **not** exposed on AOU_TOP. In AOU_TOP, they are consumed or generated internally by AOU_FDI_BRINGUP_CTRL.

Signal	Direction (on AOU_CORE_TOP)	Description
I_INT_FSM_IN_ACTIVE	input	FDI link active indicator. Driven by the bringup FSM's O_INT_FSM_IN_ACTIVE inside AOU_TOP.
O_AOU_ACTIVATE_ST_DISABLED	output	Protocol activation state: disabled. Fed back to bringup FSM.
O_AOU_ACTIVATE_ST_ENABLED	output	Protocol activation state: enabled. Fed back to bringup FSM.
O_AOU_REQ_LINKRESET	output	Protocol-level LinkReset request. Fed back to bringup FSM.

Software operation Guide

This section gives information about operation guide for the AOU_CORE.

Register Map

The complete register map — address offsets, field definitions, access types, and reset values — is generated from the SystemRDL source and maintained separately:

- **AOU Core CSR Reference** (generated from csr/aou-core.rdl)

To regenerate after editing the RDL source, see DOC/csr/README.md.

Interrupts

- **INT_ACTIVATE_START** An interrupt that occurs when activation of the AoU Protocol layer is required. The interrupt can be cleared by writing '1' to the AOU_CORE.AOU_INIT.INT_ACTIVATE_START SFR.
- **INT_DEACTIVATE_START** An interrupt that occurs when deactivation of the AoU Protocol layer is required. The interrupt can be cleared by writing '1' to the AOU_CORE.AOU_INIT.INT_DEACTIVATE_START SFR.
- **INT_SI0_ID_MISMATCH** An interrupt that occurs on the AXI Slave Interface when a B or R channel response is received with an ID that was not previously issued as a request. SW can check the mismatched AXI ID by reading AOU_CORE.AXI_SLV_ID_MISMATCH_ERR SFR. The interrupt can be cleared by writing '1' to the AOU_CORE.AXI_SLV_ID_MISMATCH_ERR.SLV_B/RRESP_ERR SFR.
- **INT_MI0_ID_MISMATCH** An interrupt that occurs on the AXI Master Interface when a B or R channel response is received with an ID that was not previously issued as a request. SW can check the mismatched AXI ID by reading AOU_CORE.ERROR_INFO SFR. The interrupt can be cleared by writing '1' to the AOU_CORE.ERROR_INFO.SPLIT_B/RID_MISMATCH_ERR SFR.
- **INT_EARLY_RESP_ERR** An interrupt that occurs when, due to the Write Early Response feature, a B response has already been sent through the AXI Slave Interface, and a subsequent actual B response arrives with an Error. SW can check the ID and error type of the transaction in which the error occurred by reading AOU_CORE.WRITE_EARLY_RESPONSE SFR. The interrupt can be cleared by writing '1' to the AOU_CORE.WRITE_EARLY_RESPONSE.WRITE_RESP_ERR SFR.
- **INT_REQ_LINKRESET** An interrupt that occurs when AOU_CORE receives a protocol violation. Refer to the

AOU_CORE Micro-Architecture Specification (DOC/MAS/interrupts-errors.adoc) for debugging details. When AOU_CORE receives a protocol violation, SW needs to do SW reset AOU_CORE and re-enter the activation sequence.

Activation Flow

> **AOU_TOP users:** When using AOU_TOP, FDI bringup (RESET to ACTIVE) is handled by AOU_FDI_BRINGUP_CTRL before protocol-level activation begins. The protocol activation described in this section occurs after O_FDI_LINK_UP is asserted. See FDI Bringup Flow (AOU_TOP only) for the FDI bringup sequence.

The Activation of AOU_CORE begins after the UCIe Link-up process has been successfully completed. The completion of UCIe link-up is indicated by the **I_INT_FSM_IN_ACTIVE** signal of the **AOU_CORE_TOP**. There is no dependency between the activation of the local die and the remote die. Each die may initiate the activation sequence independently, which means the timing of sending **ActivateReq** can differ. Both dies must exchange **ActivateReq** and **ActivateAck** messages to transition to the **ENABLED** state, at which point credited messages can be transmitted.

Regardless of its own activation state, a die must send an ActivateAck in response to an ActivateReq from the other die, to acknowledge receipt of the request.

Credit Management Type

There are two types of Credit Management Type. Since current AOU_SPEC cannot resolve pending AXI response without Activate again. When Credit Management type is set to 1, during Deactivated state, AOU_CORE can resolve pending AXI response itself.

It can be configured through AOU_CON0.CREDIT_MANAGE, and the default value is 1.

Credit Management Type	Specification	Description
0	Based on AoU v0.5	After deactivation, if a new request is received from the remote die, no response message can be sent. To deliver the corresponding response, the system must go through activation again after deactivation. Credit management and transmission availability for both Request-related messages and Response-related messages are controlled together. Credited messages must not be sent, after the DeactivateReq message is sent. Credits must not be sent after the DeactivateAck message is sent. The Activate Interrupt and Deactivate Interrupt that occur in the process of resuming the exchange of response messages impose mandatory requirements to set Activate and Deactivate SFR.
1 (default)	Proposal by BOS	When the AoU Activity state is DEACTIVATE, manage Request-related messages (WREQ, RREQ,WDATA) and Response-related messages (RDATA, WRESP) separately. RDATA, WRESP Credited messages can be sent, after the DeactivateReq message is sent. Credits for RDATA, WRESP messages must be sent after the DeactivateAck message is sent. The Deactivate interrupt only provides a hint indicating that the remote die has started to activate / deactivate.

Activation Start

Activation can be initiated by setting ACTIVATE_START SFR: In the current implementation, credits are granted based on the depth of the RX FIFO, so it is required to ensure that all messages in the RX FIFO are popped before sending the ActivateReq message.

Activation START (via SFR AOU_INIT.ACTIVATE_START SFR)

- Activation can be triggered by setting ACTIVATE_START.
- Activation does not proceed until I_INT_FSM_IN_ACTIVE is asserted, since no flits can be transmitted beforehand. Therefore, it is allowed to set the ACTIVATE_START SFR before I_INT_FSM_IN_ACTIVE is asserted. If it is set before the assertion, the activation process will automatically proceed after I_INT_FSM_IN_ACTIVE becomes asserted.
- AOU_INIT.ACTIVATE_START SFR is automatically cleared to 0 once the activation is completed and AOU_ACTIVATE_STATE transitions to ENABLED.

Deactivation Start

Deactivation is initiated by setting the AOU_INIT.DEACTIVATE_START register. Setting this register does not immediately trigger sending a DeactivateReq message. The detailed conditions and sequence for deactivation can be found in the **Deactivation Sequence Flow** section. This section will only describe AOU_INIT.DEACTIVATE_TIME_OUT_VALUE SFR.

- When software initiates a DeactivateReq by setting the SFR, there may still be outstanding requests that have not yet left AOU_CORE FDI outputs.
- To handle this safely, a timeout mechanism is implemented to ensure that no valid packets remain in the AOU_TX_CORE.
- If software guarantees that all responses to its issued requests have been received before setting the deactivation start SFR, the deactivate TIME_OUT_VALUE (AOU_INIT.DEACTIVATE_TIME_OUT_VALUE SFR) can be safely configured to a shorter duration.

The ACTIVATION_OP field encodes deactivation messages as follows:

- 2 = DeactivateReq
- 3= DeactivateAck

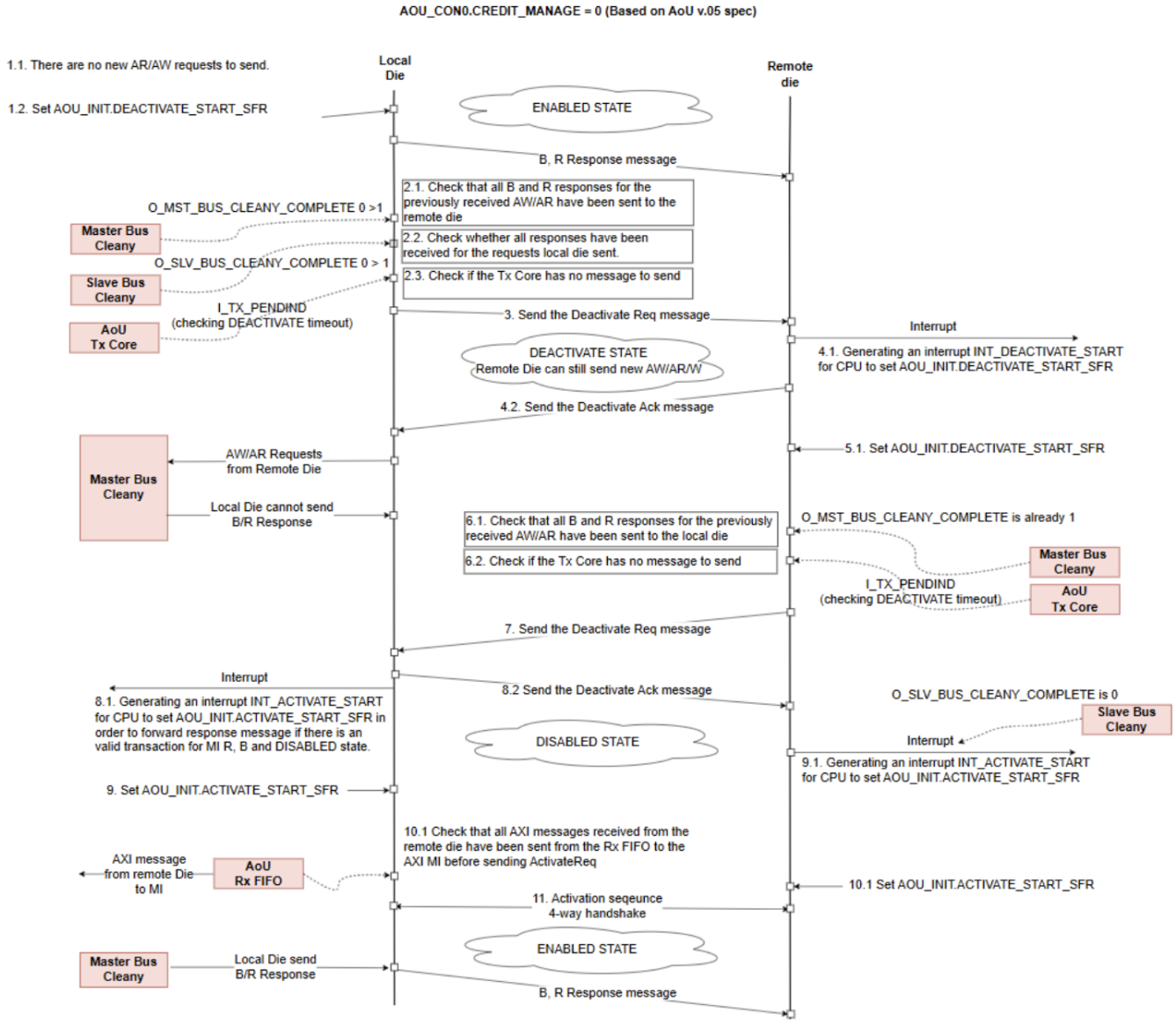
Deactivation Sequence Flow

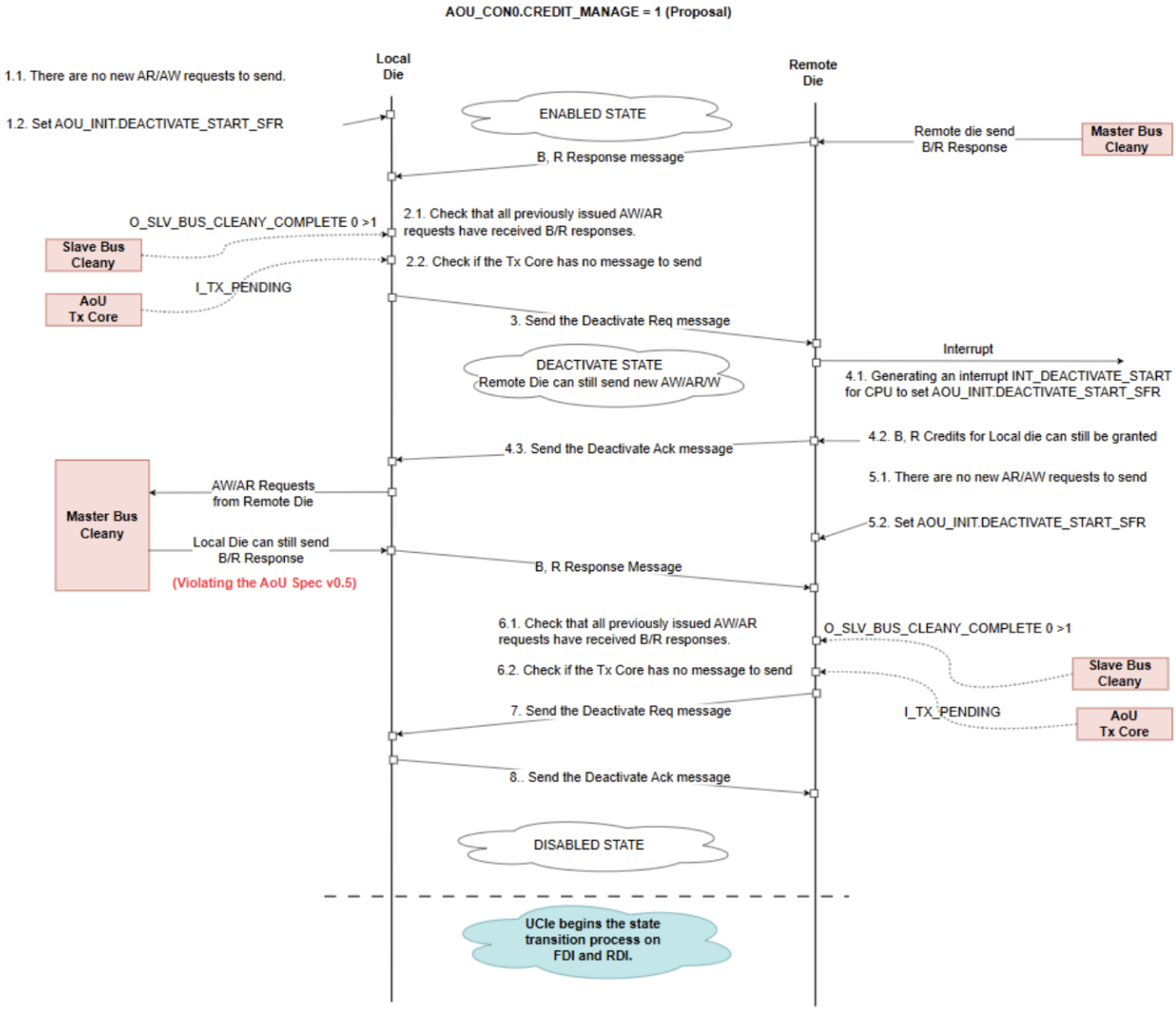
Once the local die issues a DeactivateReq, it can no longer provide responses to transactions initiated by the remote die. If the remote die continues to send requests or waits for responses without being notified, it may enter a hang state.

To manage this safely, the remote die must take explicit action upon receiving a DeactivateReq:

1. Immediately generate an interrupt to the CPU to inform the system software that a DeactivateReq has been received and that it must set the DEACTIVATE_START SFR. Although deactivation of the local die and the remote die operate independently, it is essential at the system level to communicate the deactivation & activation state through interrupts. This ensures that system software is explicitly informed of deactivation events and prevents the remote die from continuing to expect a response that will never arrive, thereby avoiding hang conditions.
2. This approach guarantees that once deactivation is initiated, both dies can coordinate the transition into safe and consistent DISABLED state.
3. If the software on both dies can explicitly coordinate to guarantee that all outstanding transactions have been

completed and that no new transactions will be issued, then such a complicated implementation would not be necessary.





This system provides a bus cleanly-completion mechanism:

- Slave bus cleanly indicates whether the local die has received all responses to the requests it sent to the remote die.
- Master bus cleanly indicates whether the remote die has received all responses to the requests it sent to the local die.

After the local die sends a DeactivateReq, if the remote die issues a new request and expects a response, the system must re-enter the activation sequence before any response can be provided. Before sending a new ActivateReq, the Rx FIFO must be completely emptied – that is, all messages must be popped.

After deactivation, the credit count is reset, and during the subsequent activation process, credits are advertised based on the RX FIFO depth.

Therefore, before sending an ActivateReq, the local die must confirm, as described in the local die's sequence 10.1, that all messages in the RX FIFO have been popped.

To meet satisfy the condition in the local die's sequence 10.1, no backpressure must occur on the local die's MI AW/AR/W channels.

Activation/Deactivation Interrupt

Interrupt	Description
INT_ACTIVATE_START	When this interrupt is detected, either the AOU_INIT.ACTIVATE_START SFR must be set to 1. When AOU_INIT.ACTIVATE_STATE_ENABLED becomes 1, you must write 1 to the AOU_INIT.INT_ACTIVATE_START W1C SFR must be set to 1. An interrupt occurs when the following conditions are met while both AOU_INIT.ACTIVATE_START SFR is not set. There is a message to send When AOU_CON0.CREDIT_MANAGE is set to 0, if a new request arrives from the remote die after the local die has sent a DeactivateReq, this interrupt is asserted because the local die is required to return a response to the remote die. When AOU_CON0.CREDIT_MANAGE is set to 1, this interrupt can also be asserted if a new request after the remote die has sent a DeactivateReq. There is a response to be received (I_SLV_BUS_CLEANY_COMPLETE == 0) When AOU_CON0.CREDIT_MANAGE is set to 0, this interrupt can be asserted if the local die issues new AXI requests after the remote die has sent a DeactivateReq. An ActivateReq message is received from the remote die.
INT_DEACTIVATE_START	When AOU_CON0.CREDIT_MANAGE is set to 0, if the interrupt is detected, the master IP must stop sending new request messages and the AOU_INIT.DEACTIVATE_START SFR must be set to 1. When AOU_CON0.CREDIT_MANAGE is set to 1, if the interrupt is detected, this interrupt serves only as a hint that the remote die intends to deactivate. The local die can continue sending request messages. Once all messages have been sent, the AOU_INIT.DEACTIVATE_START SFR must be set. Otherwise, the remote die may end up in a state where it can never send requests again. When AOU_INIT.ACTIVATE_STATE_DISABLED becomes 1, SW must write 1 to the AOU_INIT.INT_DEACTIVATE_START W1C SFR must be set to 1. An interrupt can be asserted when AOU_INIT.DEACTIVATE_START SFR is not set and the local die receives a DeactivateReq message from the remote die.

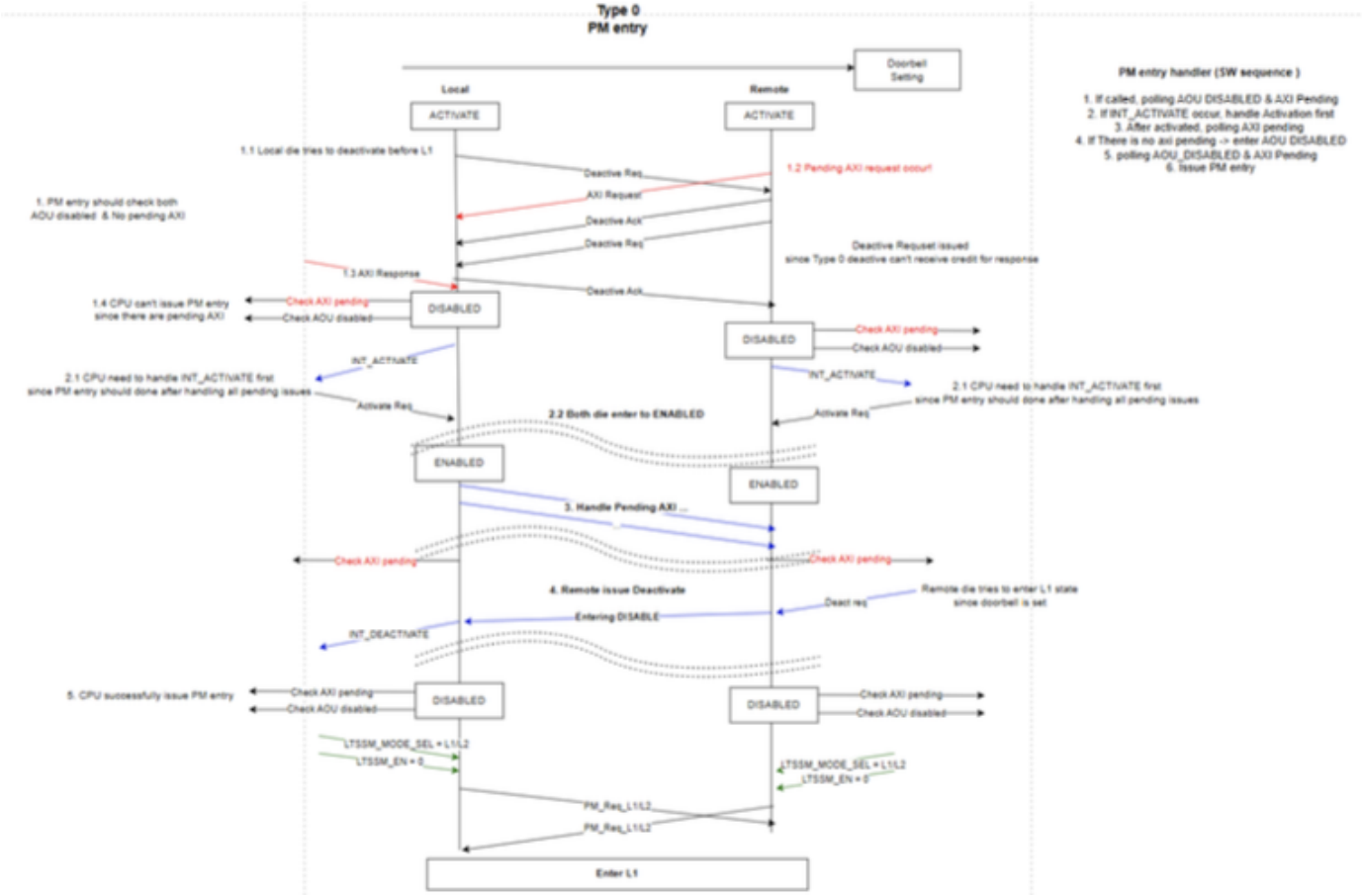
PM Entry / LinkReset / LinkDisabled Sequence

For PM entry / LinkReset / LinkDisable entry sequence, UCIE_CORE should check AOU_CORE state and try to change state. For this sequence, resolving pending AXI transactions is necessary.

Since current AOU SPEC has no way to send AXI responses after sending DeactiveReq. CREDIT_MANAGE = 0 (Type 0) is matched with current AOU_SPEC. For this case, SW needs to check whether there is pending AXI transaction.

PM Entry SW Sequence

1. Write AOU_INIT.DEACTIVATE_START to 1.
2. Polling AOU_INIT.ACTIVATE_STATE_DISABLED and AOU_INIT.SLV_TR_COMPLETE & AOU_INIT.MST_TR_COMPLETE. 2.1 Although AOU_CORE state becomes DISABLED, there may be pending AXI transactions. 2.2 AOU_CORE issues Interrupt for resolving pending AXI transactions. 2.3 Activate AOU_CORE and resolve pending AXI transactions. 2.4 Deactivate AOU_CORE
3. Polling AOU_INIT.ACTIVATE_STATE_DISABLED and AOU_INIT.SLV_TR_COMPLETE & AOU_INIT.MST_TR_COMPLETE.
4. Do PM Entry Sequence on UCIE_CORE.



Link Disable SW Sequence

Same as PM entry, before doing UCIe state transition, AOU_CORE needs to be Disabled properly.

1. Write AOU_INIT.DEACTIVATE_START to 1.
2. Polling AOU_INIT.ACTIVATE_STATE_DISABLED and AOU_INIT.SLV_TR_COMPLETE & AOU_INIT.MST_TR_COMPLETE. 2.1 Although AOU_CORE state becomes DISABLED, there may be pending AXI transactions. 2.2 AOU_CORE issues Interrupt for resolving pending AXI transactions. 2.3 Activate AOU_CORE and resolve pending AXI transactions. 2.4 Deactivate AOU_CORE
3. Polling AOU_INIT.ACTIVATE_STATE_DISABLED and AOU_INIT.SLV_TR_COMPLETE & AOU_INIT.MST_TR_COMPLETE.
4. Do LinkReset / LinkDisable Sequence on UCIE_CORE.

Link Reset SW Sequence

If AOU_CORE faces an uncorrectable error (ex. AOU SPEC violation), AOU_CORE sends LinkReset to CPU and D2D adapter. LinkReset indicates that an error has occurred which requires the Link to go down.

While handling LinkReset, SW needs to do AOU_CORE SW reset by setting AOU_CON0.AOU_SW_RESET.

FDI Bringup Flow (AOU_TOP only)

> This section applies **only** when integrating via AOU_TOP. If you are using

> AOU_CORE_TOP directly, you must implement an equivalent FDI state machine externally.

Overview

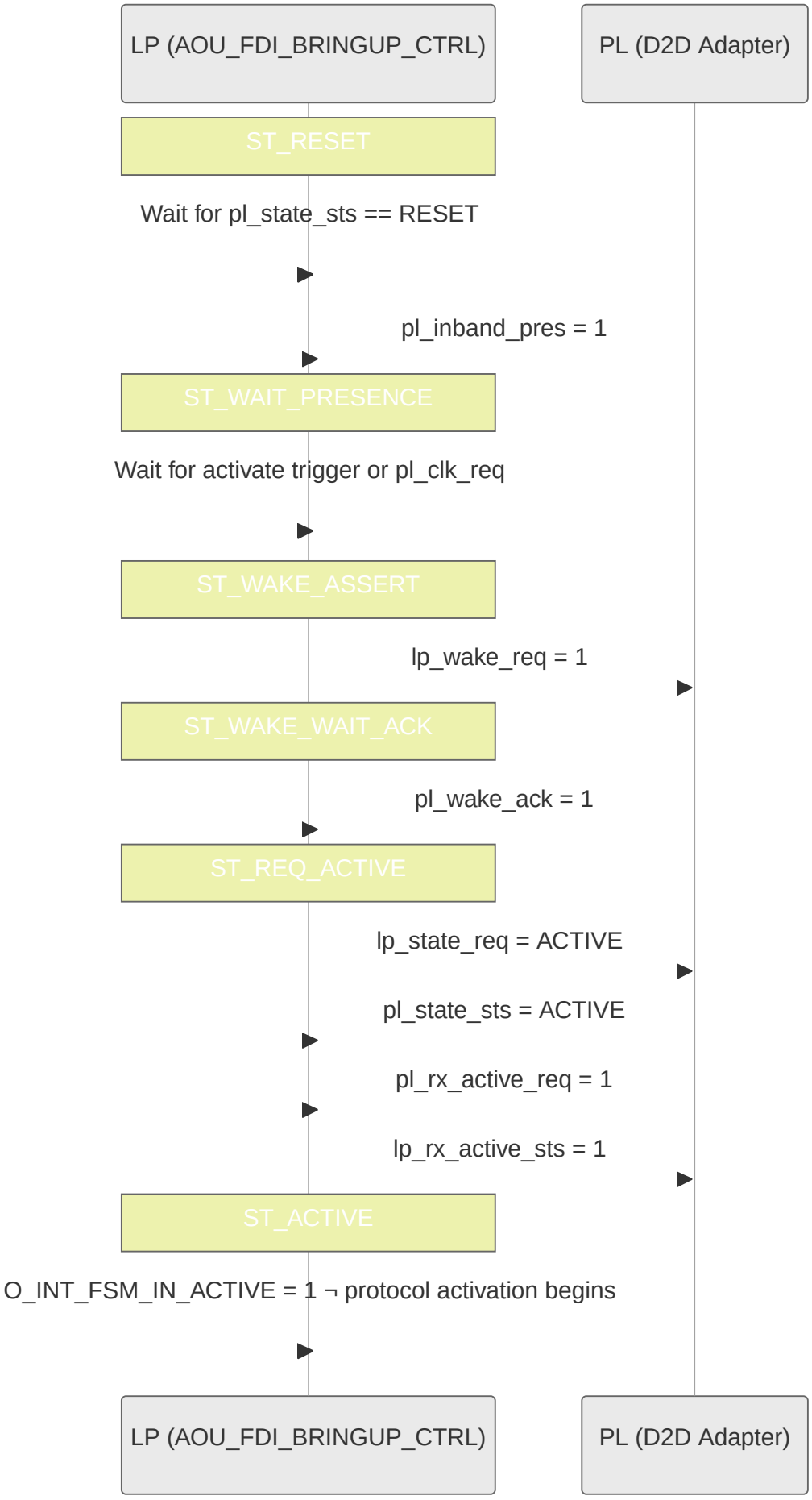
AOU_FDI_BRINGUP_CTRL manages the UCIe 3.0 FDI state machine, handling the four handshake pairs required to bring the FDI link from RESET to ACTIVE and to manage teardown, LinkError recovery, Retrain, and L1 entry/exit.

The bringup FSM operates in the I_CLK domain and uses 12 internal states. The status output O_FDI_FSM_STATE maps these to the standard UCIe 3.0 FDI state encoding.

Once the bringup FSM reaches ACTIVE, it asserts O_INT_FSM_IN_ACTIVE into AOU_CORE_TOP, which enables protocol-level activation as described in Activation Flow.

Bringup Sequence

The normal (happy-path) bringup from cold reset proceeds as follows:


LP (AOU_FDI_BRINGUP_CTRL)
PL (D2D Adapter)
ST_RESET
Wait for pl_state_sts == RESET
pl_inband_pres = 1
ST_WAIT_PRESENCE
Wait for activate trigger or pl_clk_req
ST_WAKE_ASSERT
lp_wake_req = 1
ST_WAKE_WAIT_ACK
pl_wake_ack = 1
ST_REQ_ACTIVE
lp_state_req = ACTIVE
pl_state_sts = ACTIVE
pl_rx_active_req = 1
lp_rx_active_sts = 1
ST_ACTIVE
O_INT_FSM_IN_ACTIVE = 1 ¬ protocol activation begins
LP (AOU_FDI_BRINGUP_CTRL)
PL (D2D Adapter)

The steps in detail:

1. **RESET**: After hardware reset, the FSM waits for PL to report pl_state_sts = RESET and assert pl_inband_pres (remote die detected).
2. **WAIT_PRESENCE**: Remote die is present. The FSM asserts lp_rx_active_sts (if PL requests it) and waits for an activation trigger. The trigger can be:
   - I_SW_ACTIVATE_START (top-level port)
   - I_INT_ACTIVATE_START (from the core's activation controller)
   - I_PL_CLK_REQ (PL-initiated clock request)
3. **WAKE_ASSERT** / **WAKE_WAIT_ACK**: The LP-initiated wake handshake. lp_wake_req is asserted and the FSM waits for pl_wake_ack.
4. **REQ_ACTIVE**: Wake handshake complete. The FSM drives lp_state_req = ACTIVE and waits for both pl_state_sts = ACTIVE and pl_rx_active_req = 1.
5. **ACTIVE**: The FDI link is up. O_FDI_LINK_UP and O_INT_FSM_IN_ACTIVE are asserted. Protocol-level activation (Activation Flow) can now proceed.

Supported State Transitions

From	To	Trigger
ACTIVE	LINKERROR	pl_state_sts = LinkError, I_AOU_REQ_LINKRESET, or I_SW_LINKERROR_INJECT
ACTIVE	RETRAIN	pl_state_sts = Retrain or I_SW_RETRAIN_REQ
ACTIVE	DEACTIVATE	Deactivate request AND AOU_ACTIVATE_ST_DISABLED
ACTIVE	L1	pl_state_sts = L1 (PL-initiated only)
LINKERROR	RESET	pl_state_sts = RESET
RETRAIN	ACTIVE	pl_state_sts = ACTIVE
RETRAIN	LINKERROR	pl_state_sts = LinkError
DEACTIVATE	UNWAKE_WAIT	Unconditional (1 cycle)
UNWAKE_WAIT	RESET	pl_wake_ack = 0 AND pl_state_sts = RESET
L1	L1_CLK_WAKE	pl_clk_req = 1
L1	WAKE_ASSERT	Activate request
L1_CLK_WAKE	WAKE_ASSERT	Unconditional (1 cycle, asserts lp_clk_ack)

PHY Type Mux

> **Note:** This PHY-type multiplexer is only present in builds compiled with +define+TWO_PHY. In single-PHY builds, the pl_state_sts and pl_stallreq signals routed to the bringup controller come from PHY0 unconditionally; the I_PHY_TYPE port and the PHY1-side ports do not exist on the module boundary.

When built with +define+TWO_PHY, AOU_TOP includes a PHY-type multiplexer controlled by I_PHY_TYPE:

I_PHY_TYPE	Selected PHY	pl_state_sts source	pl_stallreq source
0	PHY0 (FDI_IF_WD0)	I_FDI_PL_0_STATE_STS	I_FDI_PL_0_STALLREQ
1	PHY1 (FDI_IF_WD1)	I_FDI_PL_1_STATE_STS	I_FDI_PL_1_STALLREQ

The muxed pl_state_sts and pl_stallreq are routed to the bringup controller. FDI datapath signals (data, valid, trdy, etc.) pass through to AOU_CORE_TOP unchanged — the core's AOU_TX_FDI_IF and AOU_RX_FDI_IF handle per-PHY data-path switching internally.

Software Bringup Control

The I_SW_ACTIVATE_START and I_SW_DEACTIVATE_START ports on AOU_TOP are **not SFR-mapped**. They are top-level pins that must be driven by external logic (e.g. a system controller or GPIO). The bringup FSM ORs I_SW_ACTIVATE_START with the core's internal INT_ACTIVATE_START to form the combined activation trigger.

> **Note (GAP-15):** The SFR register at address 0x8 bit[0] (activate_start) feeds the protocol-level activation controller, not the FDI bringup FSM. Writing this bit has no effect until the FDI link is already in the ACTIVE state. To trigger cold-start FDI bringup via software, assert I_SW_ACTIVATE_START on the AOU_TOP port or arrange for PL to drive pl_clk_req.

Known Limitations

The current implementation covers the core FDI bringup happy path. The following limitations apply (see RTL/fdi-bringup-todo.md for the full gap analysis):

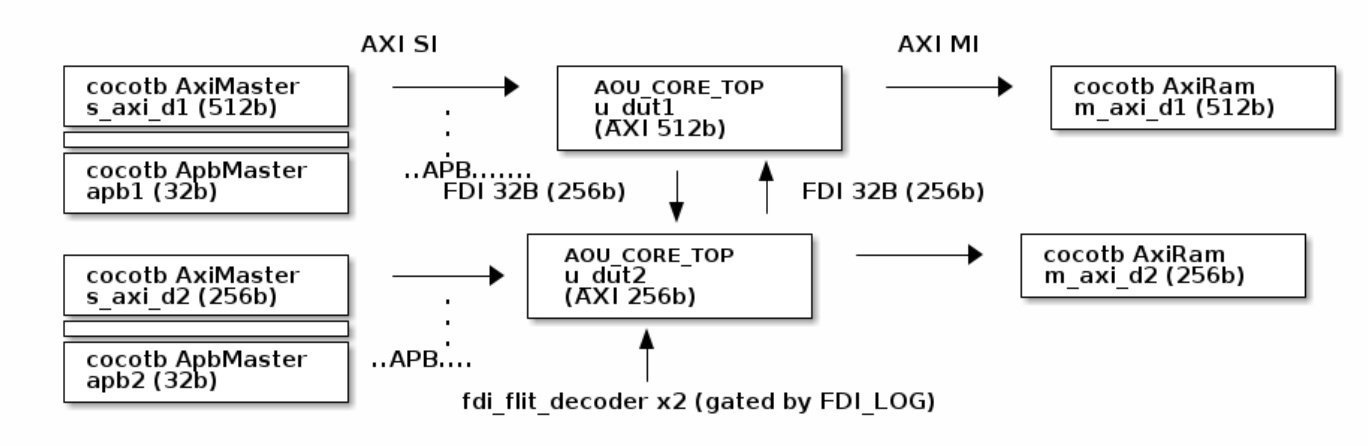
ID	Description
GAP-1	No wake handshake timeout. If pl_wake_ack never arrives, the FSM hangs in ST_WAKE_WAIT_ACK indefinitely.
GAP-3	L2 (deep low-power) state is not implemented.
GAP-4	Disabled state is not implemented.
GAP-9	LP-initiated L1 entry is not supported (L1 entry is PL-initiated only).
GAP-15	SFR activate_start (0x8 bit[0]) is disconnected from the FDI bringup FSM.

Verification Testbench

The verification flow is built on cocotb and cocotbext-axi. All verification IP is fetched from PyPI at install time — no commercial VIP is required and no AXI VIP source is vendored into this repository. The default simulator is Verilator (open source); Synopsys VCS is supported transparently for users with a commercial license.

The harness (VERIF/aou_cocotb_top.sv) instantiates two AOU_CORE_TOP cores back-to-back via their 32B (256b) single-PHY FDI interfaces. Each DUT is overridden with .FDI_CONFIG(packet_def_pkg::FDI_CFG_SP_32B) and the build does not define +define+TWO_PHY, so only the PHY0 ports are connected. The two DUT instances use different AXI data widths (512b and 256b) to exercise the width-interoperability path. The four AXI buses and two APB slave buses are exposed as flat top-level ports using cocotbext-axi's canonical signal naming (s_axi_d1_*, m_axi_d1_*, s_axi_d2_*, m_axi_d2_*, apb1_*, apb2_*) so the Python tests can use AxiBus.from_prefix(dut, "<prefix>") and ApbBus.from_prefix(dut, "<prefix>") directly. Refer to VERIF/README.md for setup, watchdogs, and the test list.

Architecture



The data flow for a write transaction initiated on u_dut1's AXI slave interface is:

1. The cocotb AxiMaster on s_axi_d1 issues a 512b write.
2. u_dut1 packs the AXI transaction into AOU flit(s) and transmits via the 32B (256b) PHY0 FDI port.
3. u_dut2 receives the FDI data, unpacks it, and presents the transaction on its AXI MI.
4. The cocotb AxiRam on m_axi_d2 accepts the write into its internal memory model.
5. A subsequent read to the same address returns the stored data back through the reverse FDI path.
6. The Python test compares the read data against the originally written payload byte-for-byte.

The reverse direction (initiated from s_axi_d2 to m_axi_d1) operates symmetrically.

A separate test (test_csr_reset_values) reads every register defined in csr/aou-core.rdl over APB on both DUTs immediately after reset and *before* APB activation, verifying that every sw-readable, reset-bearing field matches its RDL-defined reset value. The expected values are built at runtime by systemrdl-compiler, so the check self-updates whenever the RDL changes and surfaces RDL RTL drift on every run.

Components

Component	Instance / role	Description
AOU_CORE_TOP	u_dut1	DUT instance 1. AXI data width = 512b (default parameters). FDI_CONFIG = FDI_CFG_SP_32B (256b PHY0; no TWO_PHY).
AOU_CORE_TOP	u_dut2	DUT instance 2. RP0-RP3 AXI data widths overridden to 256b. FDI_CONFIG = FDI_CFG_SP_32B.
cocotbext.axi.AxiMaster	s_axi_d1, s_axi_d2	Drives AXI writes/reads on each DUT's slave interface.
cocotbext.axi.AxiRam	m_axi_d1, m_axi_d2	Memory-backed AXI slave attached to each DUT's master interface. Stores writes and serves reads.
cocotbext.axi.ApbMaster	apb1, apb2	Drives the AOU activate write (paddr=0x8, pwdata=0x1) on each DUT after reset, and reads CSRs for the reset-value test.
fdi_flit_decoder	u_fdi_dec1, u_fdi_dec2	Passive FDI decoders. Log decoded flits to dut1_fdi.log / dut2_fdi.log. Built only when FDI_LOG=1 is passed to make.

Key testbench parameters:

Parameter	Value	Description
CORE_CLK_PERIOD_NS	1 ns	Core clock period (AXI/FDI domain).
APB_CLK_PERIOD_NS	10 ns	APB clock period.
D1_AXI_DATA_WIDTH	512	u_dut1 AXI data width (bits).
D2_AXI_DATA_WIDTH	256	u_dut2 AXI data width (bits).
AXI_ADDR_WIDTH	64	AXI address width.
AXI_ID_WIDTH	10	AXI ID width.
APB_ADDR_WIDTH / APB_DATA_WIDTH	32 / 32	APB widths on both DUT slaves.
OVERALL_TIMEOUT_US	500	Per-test sim-time backstop.

Running the Testbench

From the repository root, set up the Python environment once and then invoke the testbench:

```
cd VERIF
bash setup_cocotb_env.sh        # creates VERIF/venv/, installs cocotb + deps
source venv/bin/activate
make                            # Verilator (default), 8 parallel C++ jobs
make SIM=vcs                    # Synopsys VCS
make WAVES=1                    # waveform dump (FST/VCD or FSDB/VPD)
make FDI_LOG=1                  # enable fdi_flit_decoder logs
make clean                      # remove sim_build/ and generated artefacts
```

A successful run prints *** SIMULATION PASSED *** and writes results.xml (junit). Per-phase watchdogs (reset, APB activate, APB read, AXI write, AXI read) name the stuck phase explicitly on stalls and dump a snapshot of the relevant signal values so the simulator never spins silently.

You can also drive the flow via pytest, which calls cocotb's runner API and parametrises across every simulator on PATH:

```
pytest -v                       # all available simulators
pytest -v -k verilator          # only Verilator
SIM=vcs pytest -v               # force Synopsys VCS
```

IP Integration Collateral

Machine-readable integration collateral is generated from the SystemRDL register source (csr/aou-core.rdl) using PeakRDL. This section describes the available outputs, the single-source workflow, and regeneration instructions.

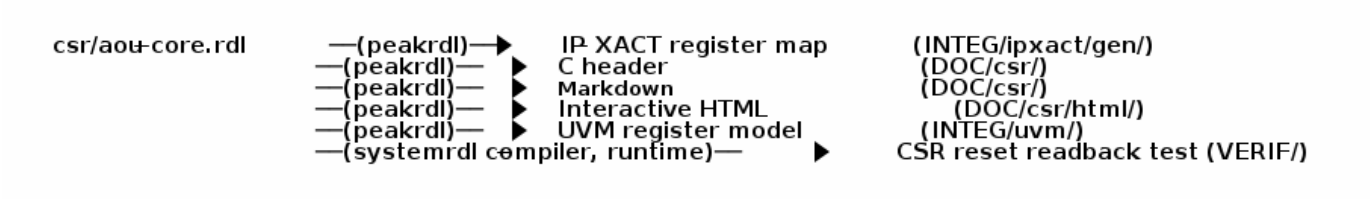
Generated Outputs (Phase 1)

Output	Path	Audience
IP-XACT register map (IEEE 1685-2014)	INTEG/ipxact/gen/aou_core_regmap.xml	SoC integration tools (register only — memoryMaps element)
C header	DOC/csr/aou_core_csr.h	Firmware / driver developers (#define macros for offsets and field masks)
Interactive HTML docs	DOC/csr/html/	Browsable register reference (any team)
UVM register model	INTEG/uvm/aou_core_csr_uvm_pkg.sv	Verification (UVM uvm_reg_block package) — drop into your UVM env's RAL tier.
Markdown register docs	DOC/csr/aou-core-csrs.md	Quick reference (also linked from Register Map)

Note: the in-repo cocotb testbench does not consume the generated UVM package — it parses csr/aou-core.rdl directly via systemrdl-compiler at runtime. The UVM package is shipped as collateral for integrators whose own verification environments are UVM-based.

Single Source of Truth

All register information lives in the SystemRDL source file csr/aou-core.rdl. Every output listed above is derived from this single file — there is no separate register spreadsheet or hand-maintained IP-XACT register map.



To modify register definitions (addresses, field widths, access types, reset values, descriptions), edit csr/aou-core.rdl and regenerate. IMPORTANT: RTL is not generated from RDL so any modification to RDL files will not be reflected in the design.

Regeneration

From the repository root, activate the Python virtual environment and run the generation script:

```
source venv/bin/activate
bash scripts/gen_collateral.sh
```

The generation script automatically validates the IP-XACT output against the official Accellera IEEE 1685-2014 XSD schema after generation. The schema files are fetched from accellera.org on first run and cached locally (in INTEG/ipxact/schema/, gitignored).

To validate IP-XACT files independently:

```
python3 scripts/validate_ipxact.py INTEG/ipxact/gen/aou_core_regmap.xml
```

See DOC/csr/README.md for a description of each output file and the top-level README.md for virtual-environment setup.

Planned: Full IP-XACT Component (Phase 2)

The Phase 1 IP-XACT output contains the <memoryMaps> element only (registers, fields, access types, reset values). A full IP-XACT component — including <busInterfaces> (ARM AMBA AXI4 and APB3), <ports>, and <model> parameters — is planned for a later phase.

This is deferred because AOU_CORE_TOP.sv uses SystemVerilog 2D packed arrays for its per-RP AXI ports (e.g., [RP_COUNT-1:0][AXI_ID_WD-1:0]), which have no clean single-dimension representation in IP-XACT. Once a wrapper module (AOU_CORE_TOP_WRAP) is created that flattens these into individual per-RP port signals, the full IP-XACT component will be generated by merging the PeakRDL register map into a hand-authored component template via scripts/merge_ipxact.py.

Timing Constraints (SDC)

Constraint files are located in INTEG/constraints/.

File	Purpose
aou_core_top.sdc	Clock definitions, I/O delays, CDC, false paths
aou_core_top.upf	Power domain and supply network

The SDC file defines timing constraints for synthesis and STA. All delay budget variables are declared at the top of the file and can be overridden by the integrating project.

Clock Definitions

Clock	Port	Frequency	Period
clk_core	I_CLK	1 GHz	1.000 ns
clk_apb	I_PCLK	100 MHz	10.000 ns

Both resets (I_RESETN, I_PRESETN) are constrained with set_false_path and should be synchronised externally or by the integrating design. The DFT scan-mode pin (TIEL_DFT_MODESCAN) is held at 0 via set_case_analysis in functional mode.

I/O Delay Budgeting Strategy

Two budgeting categories are used, based on whether the path between the I/O port and the nearest internal flop is combinational or registered:

Path type	Internal budget	External budget	Rationale
Combinational	50% of clock period	50% of clock period	Mux/decode logic inside RTL takes some portion of cycle
Registered	30% of clock period	70% of clock period	Short wire path to first flop; external block gets more margin

Interface classification summary:

- **APB Slave**: Registered (via ASYNC_APB_BRIDGE 2-flop synchroniser on both sides). Constrained to clk_apb.
- **AXI Slave inputs** (address, data, handshake to FIFOs): Registered — input data is captured directly into FIFO register arrays.
- **AXI Slave outputs** (READY signals, read/bresp data): Combinational — passes through arbitration muxes.

- **AXI Master outputs** (AR/AW/W channels): Combinational — data passes through FIFO read mux, arbitration mux (AXI4MUX_3X1), and REV_RS mux.
- **AXI Master inputs** (R/B data, READY feedback): Combinational — feeds control logic and FIFO write.
- **FDI TX outputs** (DATA, VALID, IRDY): Combinational. STALLACK outputs are registered.
- **FDI RX inputs**: Combinational — data/valid pass through an input mux. FLIT_CANCEL drives both sequential and combinational logic.
- **Interrupt/control signals**: Combinational.

Clock Domain Crossings

The only CDC in AOU_CORE_TOP is between clk_core and clk_apb, handled by the ASYNC_APB_BRIDGE module which uses AOU_SOC_SYNCHSR (2-flop) synchronisers. The SDC uses set_clock_groups -asynchronous to exclude cross-domain paths from STA.

UPF Power Intent

The current UPF defines a single always-on power domain (PD_AOU_TOP) covering the integration scope (the AOU_TOP instance, which includes AOU_CORE_TOP and AOU_FDI_BRINGUP_CTRL, when integrating via AOU_TOP; or the AOU_CORE_TOP instance when integrating via AOU_CORE_TOP directly) with a VDD/VSS supply pair. No isolation, retention, or level-shifting strategies are required at this time. When power domains are introduced in the future, the UPF should be extended with create_power_domain for each switchable domain, along with appropriate isolation and retention strategies.

Library Cell Replacement

The RTL/LIB/ directory contains three behavioral reference models that are used by the design for reset infrastructure and clock-domain crossing. These models are functionally correct for simulation but **must be replaced** with process-appropriate implementations before synthesis or tapeout.

Module	File	Parameters	Ports	Behavioral function	Where instantiated
AOU_SOC_SYNCHSR	RTL/LIB/AOU_SOC_SYNCHSR.v	DW (width), RST_VAL (reset value), DEPTH (2/3/4)	I_CLK, I_RESETN, I_D[DW-1:0] → O_Q[DW-1:0]	Multi-stage synchroniser: per-bit shift register of depth DEPTH with async active-low reset to RST_VAL	ASYNC_APB_BRIDGE_SI, ASYNC_APB_BRIDGE_MI (CDC synchronisation)
AOU_SOC_GFMUX_LVT	RTL/LIB/AOU_SOC_GFMUX_LVT.v	DW (width)	I_SEL, I_A[DW-1:0], I_B[DW-1:0] → O_Y[DW-1:0]	Glitch-free 2:1 mux (I_SEL=0 selects I_A, I_SEL=1 selects I_B)	AOU_CORE_TOP as u_sw_reset_mux (SW reset selection)
AOU_SOC_BUF	RTL/LIB/AOU_SOC_BUF.v	DW (width)	I_A[DW-1:0] → O_Y[DW-1:0]	Non-inverting buffer (identity: O_Y = I_A)	AOU_CORE_TOP as dft_persistent_buf_scan_resetn_sw (DFT persistent buffer on scan reset)

Replacement approaches:

1. **Drop-in replacement** — Create new module definitions with the same module names and identical port/parameter interfaces, backed by standard cells or hard macros from the target process library. Replace the corresponding files in the filelist (RTL/LIB/filelist.f) with the new implementations.
2. **In-place wrapper** — Edit the provided module shells directly, replacing the behavioral assign/always bodies with

instantiations of target-library primitives while keeping the module name and port interface unchanged.

Important considerations:

- AOU_SOC_SYNCHSR is on the CDC path and must be replaced with a foundry-qualified synchroniser cell (or equivalent hardened multi-flop structure) to meet metastability MTBF requirements. The DEPTH parameter must be honoured.
- AOU_SOC_GFMUX_LVT is on the reset mux path and must be replaced with a glitch-free clock/reset mux cell to avoid glitches during reset switching.
- AOU_SOC_BUF is on the DFT scan-reset path. Replace with a buffer cell that will be preserved through synthesis optimisation (typically via a dont_touch or equivalent attribute).
- The behavioral models can serve as a golden reference for verifying functional equivalence of the replacements.

Scan-Friendly Cell Requirements

Beyond functional correctness, each replacement cell must be compatible with ATPG scan insertion and pattern generation. The behavioral models have no scan awareness; the replacements must address this.

AOU_SOC_SYNCHSR — Scan-safe synchronizer

Standard multi-flop synchronizers cause X-propagation during scan shift because the asynchronous input crossing from the other clock domain is not controllable by the ATPG tool. The replacement synchronizer must be **scan-safe** via one of the following approaches:

1. **Built-in scan bypass mux** (preferred): The synchronizer cell includes a scan-mode mux that bypasses the asynchronous first-stage flop during shift. The ATPG tool holds the bypass active during shift and restores normal operation for capture. This is the standard approach used by foundry-provided synchronizer cells (e.g., SDFF-based sync cells with a scan-enable input).
2. **ATPG tool constraint**: If the replacement cell does not have a built-in bypass, the integrator must declare it as a synchronizer in the ATPG tool so the tool forces the input to a known value during shift (e.g., Synopsys set_dft_signal -type constant on the synchronizer input).

Foundry-provided synchronizer cells are strongly preferred because they are pre-characterized for both metastability MTBF and scan testability.

AOU_SOC_GFMUX_LVT — Clock/reset mux recognition

This cell sits on the scan-mode reset mux path (see TIEL_DFT_MODESCAN Usage). The replacement must be recognized by the ATPG tool as a **clock or reset mux cell**, not a generic data mux. If the tool treats it as a regular mux, it may toggle I_SEL during scan patterns, causing reset glitches that corrupt shift-register state and invalidate test patterns.

The integrator should:

- Use a process library cell explicitly categorized as a clock/reset mux (e.g., a cell from the library's ICG or clock-mux family).
- Apply the appropriate DFT tool attribute to prevent I_SEL toggling during scan. For example, in Synopsys DFT Compiler: set_dft_signal -view existing_dft -type ScanMux -port u_sw_reset_mux/I_SEL.
- Verify in the ATPG tool's DFT rule check that the mux select is classified as a test-mode signal, not a scannable data input.

AOU_SOC_BUF — Optimization protection

This buffer drives the tied-low scan-reset bypass value into u_sw_reset_mux. Without protection, the synthesis tool will optimize it away (replacing it with a direct tie-off), which may cause the downstream reset mux to be optimized or restructured in ways that break the intended DFT architecture.

The replacement buffer must carry a dont_touch or size_only attribute (the specific attribute name depends on the synthesis tool) to ensure it survives all optimization passes, including:

- Logic optimization and constant propagation
- Incremental optimization during place-and-route
- Post-route optimization

Project Defines

Compile-time defines for OCAH RTL: the names that apply across OCAH, then the defines specific to one subsystem. Set the OCAH names in this chapter. Do not set vendor preprocessor names; `hw/common/defs/ocah_vendor_defines.svh` derives the aliases those names imply. OCAH lint and Yosys force-include that header. Native DV expands the same aliases onto the command line. An adopter compile must do one of those two. Block-selection defines ride the Bender filelist as `define` lines (Synthesis filelist defines); do not restate them on the command line.

A `define` in a header is visible only in compilation units that include it. Command-line `` `define `` / `-D` is global. Force-include (or the flow's alias expansion) is what makes the mapping apply to vendored files that never include an OCAH header.

Chapter 14. SIMULATION

SIMULATION selects the verification view of OCAH RTL. Set it on every simulation compile and on no other kind of build:

```
+define+SIMULATION   # VCS, Xcelium
-DSIMULATION         # Verilator, Icarus
```

OCAH's own DV flows set it; a compile driven by an adopter's testbench does not.

WARNING Do not let SIMULATION reach synthesis, formal, or a gate-level flow. It enables initial blocks, $readmemh, $random and DPI exports; a tool that skips those rather than rejecting them yields a netlist that does not correspond to the RTL you verified.

Without the define, these testbench hooks are not compiled in and the plusargs are silently ignored:

Plusarg	Effect
+skip_fuse_sense	efuse_shadow_regs reports fuse sense complete immediately rather than sensing the bank through the adopter's SHIM.
+smc_shadow_reg_preload=<file>, +sep_shadow_reg_preload=<file>	Loads the shadow registers from a hex image. Takes effect only alongside +skip_fuse_sense.
+rom_bin64=<file>, +rom_hex=<file>	Preloads OCAH4CORECluster_rom_ext, the SMC CPU ROM simulation model. Applies only to integrations that use the reference memory swaps.
+cdc_instrumentation_enabled=1	Arms every prim_cdc_rand_delay and abr_prim_cdc_rand_delay instance. On each input change, a random subset of bits keeps its old value for one more destination cycle. Without the plusarg the delay model passes its input through.

The define also zeroes the RMA and security-disable token and digest bits in efuse_token_processing at time 0, which have no reset and otherwise power up X, and routes OpenTitan assertion failures in the SEP prim_assert shim to $error on non-UVM runs.

Two model families it enables make simulation deliberately non-deterministic: prim_flop_2sync, its Adams Bridge equivalent, and the OCAH prim_sync3 / prim_sync4 cells insert prim_cdc_rand_delay ahead of the synchronizer, which +cdc_instrumentation_enabled=1 arms, and prim_lfsr and prim_double_lfsr randomize their default seed on simulators other than Verilator (+prim_lfsr_use_default_seed=1 pins it). Two runs of one test are therefore not identical unless the simulator seed is fixed; record the seed with any failure you intend to reproduce.

Chapter 15. SYNTHESIS

SYNTHESIS selects the implementation view: simulation-only SVA, $readmemh preload, procedural delays, and similar non-synthesizable paths are omitted. Slang lint, Verilator lint, and Yosys set it (OCAH_FLOW_COMMON_DEFINES, or +define+SYNTHESIS=1 for Verilator lint). The Bender synth view emits +define+SYNTHESIS=1 on its filelist, so a synthesis compile that consumes the filelist's defines (below) gets it without tool setup; Synopsys tools also predefine the name. Native DV does not set it, except the SMC dual target, which lists it next to SMC_DUAL. The open formal path (sby / read_slang) keeps it defined. Licensed formal backends leave it unset.

Chapter 16. EMULATION

EMULATION selects the emulation view of OCAH RTL: synthesizable RTL for a hardware emulator (Palladium, Zebu, and similar) that keeps the debug hooks an emulator can map into hardware. Set it on the emulator compile, and set neither SIMULATION nor SYNTHESIS alongside it:

```
+define+EMULATION
```

Compiled in	Compiled out
Concurrent and immediate assertions, cover and assume: first-party, OpenTitan, Adams Bridge, Caliptra, and PULP macros.	Procedural delays. ASSERT_INIT_NET and its equivalents keep the check and drop the #1ps settle.
Elaboration-time parameter checks.	Random CDC delay models and randomized seeds.
$display, $error, and $fatal debug messages.	Testbench handshake initial blocks, and the testbench tasks in the i3c axi_if interface.
final blocks, and plusarg hooks outside the SIMULATION set, such as +show_mem_paths.	DPI exports, and the SIMULATION plusargs with their ROM and shadow-register file preload.
$readmemh initialization from a memory's MemInitFile parameter.	

EMULATION takes precedence over SYNTHESIS in first-party RTL and in the vendored assertion headers, so a compile where the tool predefines SYNTHESIS still gets the view above. Generated RTL (the SMC CPU cluster and the register blocks) and vendored logic outside the assertion headers still follow SYNTHESIS, so a compile that sets both loses their elaboration checks and debug messages. First-party RTL gives EMULATION precedence over SIMULATION too, but vendored CDC delay models and seed randomization follow SIMULATION whatever else is set.

The vendor aliases need the force-include or alias expansion described at the top of this chapter; without it, PULP assertions follow SYNTHESIS and ASSERTS_OFF instead.

In the emulation view the entropy source ring oscillator is a zero-delay combinational loop. The emulator compile must break or model it.

The emulation view has no testbench preloads. OCAH4CORECluster_rom_ext is a prim_rom with an empty MemInitFile, so load the SMC CPU ROM image through the emulator's memory backdoor. Fuse sense reads the bank through the adopter's SHIM, which the emulation must model.

Do not set SYNTHESIS in order to keep emulator debug hooks; SYNTHESIS strips assertions as well. Do not set PULP_FPGA_EMUL as a substitute: that name is an FPGA clock-mux path.

Chapter 17. Tool predefined names

Simulators and frontends define these themselves. Restate them only on that tool's compile line. Do not put them in a Bender filelist shared with other tools, and do not add a Bender target named `verilator` (flist-plus would emit `+define+TARGET_VERILATOR` into that shared `.f`).

Define	Role
`VERILATOR`	Verilator's predefine. First-party and OpenTitan SVA leave their include-assert macros undefined.
`VCS`	VCS predefine. Some PULP AXI files branch on it.
`XCELIUM`	Xcelium's predefine. Native DV uses this tool.
`XSIM`	Vivado simulator. PULP assertion macros also treat it as off.
`QUESTA`	QuestaSim's predefine. Some PULP AXI files branch on it.
`YOSYS`	Auto-defined by yosys-slang. The Yosys `prim_assert.sv` shim undefines it around the vendored header when `SYNTHESIS` is also set.

Chapter 18. Native DV extras

An adopter that copies the native profile sees these in addition to `SIMULATION`. They are not required to elaborate OCAH RTL for silicon.

Define	Role
`RANDOM=0`, `RANDOMIZE_MEM_INIT`, `RANDOMIZE_REG_INIT`, `RANDOMIZE_DELAY=0.02`	Chisel/FIRRTL random-init hooks on the generated SMC cluster. Native DV zero-fills rather than randomizing.
`UVM`	SV-UVM framework overlay. Assertion headers report through `uvm_report_error`. Adds the UVM harness block to shared testbench tops. Do not set it on a cocotb compile.
`COMMON_CELLS_ASSERTS_OFF`	Strips inline assertion blocks inside PULP `common_cells` modules. Native DV sets it on Verilator only, from the shared Verilator flags in `hw/common/dv/configs/profiles/native.toml`; VCS and Xcelium compile and run those blocks. It is independent of `ASSERTS_OFF`.
`ASSERTS_OFF`	Turns off PULP `OCAH_PULP_INC_ASSERT` macros. The SMC and DTP Verilator DUT flags and Verilator lint set it; VCS and Xcelium native compiles do not.

Chapter 19. Derived assertion gates

Do not set these. Headers define them from the view and tool names above.

Define	Defined when
OCAH_DEBUG_LIVE	First-party debug checks and messages. ocah_assert.svh defines it when EMULATION is set or SYNTHESIS is absent.
OCAH_INC_ASSERT	First-party concurrent SVA. ocah_assert.svh defines it when OCAH_DEBUG_LIVE is defined and VERILATOR and NO_OCAH_ASSERT are both absent.
OCAH_OT_INC_ASSERT	OpenTitan-derived SVA: off under VERILATOR, otherwise on unless SYNTHESIS is set without EMULATION (YOSYS forces it on). The Adams Bridge and Caliptra assertion gates give EMULATION the same precedence over SYNTHESIS.
OCAH_PULP_INC_ASSERT	PULP OCAH_PULP_ASSERT* macros, unless ASSERTS_OFF, SYNTHESIS, or XSIM is set. ASSERTS_OVERRIDE_ON, which EMULATION implies, forces it on.
NO_OCAH_ASSERT	Optional escape: drops first-party SVA only. No OCAH flow sets it.

No define gates the first-party parameter and width checks. OCAH_ASSERT_STATIC evaluates them at elaboration in every view and tool, Verilator and synthesis included, so a configuration that fails one does not elaborate.

Chapter 20. Vendor aliases

`ocah_vendor_defines.svh` defines these from the OCAH names. Do not set the vendor column.

OCAH name	Vendor alias
`SYNTHESIS`	`TARGET_SYNTHESIS`
`SIMULATION`	`ABR_SIMULATION`
`EMULATION`	`ASSERTS_OVERRIDE_ON`
`VERILATOR`	`TARGET_VERILATOR`
`XSIM`	`TARGET_XSIM`

Chapter 21. Adopter overlay hooks

No checked-in config sets these. An adopter overlay (`run_dv.py --overlay`) may append them.

Define	Role
`OCAH_JTAG_VENDOR_IF`	Nests a commercial JTAG verification interface.
`OCAH_AXI_VENDOR_IF`	Nests a commercial AXI verification interface.

Chapter 22. Subsystem defines

Each subsystem graph reads the view, tool and assertion names above as this chapter describes. The sections below cover the defines the rest of the chapter does not: two shared cells, then each subsystem in turn.

22.1. Shared cells

Two cells in `hw/common/ocah_prim`, compiled into every subsystem graph, read a define of their own. Leave both unset.

Define	Reason
`GATER_CHK`	In the simulation view, `prim_clk_gater_hysteresis` expands `export_dpi_get` to export its state over DPI. OCAH does not define that macro, so setting `GATER_CHK` breaks the compile.
`SIM_APB_ARB`	Adds a test-master APB port set to `prim_apb_arb`. No OCAH module instantiates `prim_apb_arb`.

22.2. SMC

The SMC graph adds three groups: first-party simulation hooks, the generated CPU cluster macros, and the i3c-core configuration.

22.2.1. Simulation hooks

The `SIMULATION` plusargs for the CPU ROM and the eFuse shadow registers are listed under `SIMULATION`. The SMC RTL also reads these view names: `smc_4core_cpu` prints scratch RAM initialization progress when neither `SYNTHESIS` nor `EMULATION` is set.

The `prim_flop_2sync`, `prim_sync3` and `prim_sync4` synchronizers instantiate `prim_cdc_rand_delay` in the simulation view, but no `smc` Bender target lists `prim_cdc_rand_delay.sv`. Add it to any `SIMULATION` compile of the `smc` filelist; SMC DV lists it as an extra source in `hw/sys/smc/dv/smc_sim_cfg.toml`.

The SMC DV `dual` target sets `SYNTHESIS` next to `SMC_DUAL`, so the CPU clusters in that build get no time-zero initialization (CPU cluster). `SMC_DUAL` is read only by `hw/sys/smc/dv/tb/tb_top.sv`, where it selects the two-instance testbench.

22.2.2. CPU cluster

The Chipyard-generated cluster in `hw/sys/smc/rtl/smc_cpu/chipyard_generated_files/4core` reads the FIRRTL macros in the table below. It ignores `EMULATION`. The generated code defines `ENABLE_INITIAL_REG_` and `ENABLE_INITIAL_MEM_` only when `SYNTHESIS` is absent, and those gate every time-zero initializer, so a `SYNTHESIS` compile reads none of the initialization macros.

Define	Effect
`RANDOMIZE_REG_INIT`	Assigns every cluster register from `RANDOM` at time zero. Without it the registers hold X until their first clocked write.
`RANDOMIZE_MEM_INIT`	Assigns every generated memory array from `RANDOM` at time zero. Without it a read before the first write returns X.
`RANDOM`	The value both initializers assign. Defaults to `$random`; `RANDOM=0` makes initialization a deterministic zero-fill.
`RANDOMIZE_DELAY`	Delay before the initializers run, in simulator time units. Defaults to `0.002`. Verilator ignores it.
`INIT_RANDOM`	Code placed at the start of every generated initializer, for a custom seeding scheme. Defaults to empty.

Define	Effect
FIRRTL_BEFORE_INITIAL, FIRRTL_AFTER_INITIAL	Code placed before and after every generated initial block. Defaults to empty.
ASSERT_VERBOSE_COND	Expression that gates the $error message of each generated assertion. Defaults to 1.
STOP_COND	Expression that gates the $fatal that follows each generated assertion failure. Defaults to 1.

Each generated module also defines a default for PRINTF_COND, but the cluster contains no printf, so that macro has no effect.

Do not set RANDOMIZE, INIT_RANDOM_PROLOG_, ENABLE_INITIAL_REG_, ENABLE_INITIAL_MEM_, PRINTF_COND_, ASSERT_VERBOSE_COND_ or STOP_COND_. The generated code derives them from the macros above.

22.2.3. i3c-core configuration

vendor/chipsalliance/i3c-core/upstream/src/i3c_defines.svh fixes the i3c-core configuration: the AXI host interface (I3C_USE_AXI) and its widths, controller and target support, loopback disabled, and the FIFO, DAT and DCT depths. Change a value in that header, not on the command line. The header leaves these upstream options unset:

Define	Effect when set
I3C_USE_AHB	Selects an AHB host interface instead of AXI.
AXI_ID_FILTERING	Adds AXI ID filtering, which needs NUM_PRIV_IDS and changes the i3c parameter and port lists.
CALIPTRA_AXI_SUB_EX_EN	Adds exclusive-access tracking to the AXI subordinate.
IBI_FIFO_EXT_SIZE	Multiplies the HCI and TTI IBI FIFO depths by 8.
DISABLE_INPUT_FF	Removes the two-flop synchronizers on the SCL and SDA inputs.

22.3. DTP

The dtp graph reads no define of its own. Two parts of it respond to the view names:

- jtag2axi asserts AXI valid stability and address and data pairing on its manager port when OCAH_DEBUG_LIVE is defined.
- The prim_flop_2sync, prim_sync3 and prim_sync4 synchronizers instantiate prim_cdc_rand_delay in the simulation view. The Bender simulation target adds prim_cdc_rand_delay.sv to the dtp graph, so generate a simulation filelist with -t dtp -t simulation.

22.4. SEP

The SEP graph adds five groups: the Adams Bridge block-selection defines, the VeeR EL2 core configuration, first-party view checks, the key manager's defines, and testbench-side DV names.

22.4.1. Adams Bridge

The Adams Bridge source group of the sep Bender target sets these three defines, and flist-plus emits them ahead of the source list. Every flow that uses the sep target gets them.

Define	Effect
SEP_ABR_EN	Includes the Adams Bridge post-quantum engine: sep_crypto instantiates sep_crypto_abr_wrapper, and sep_ip_integration builds the ABR SRAMs. Without it, sep_crypto terminates the ABR AXI aperture and the key manager's ABR key bus with DECERR subordinates, and sep_ip_integration drives the ABR memory response to zero.
CALIPTRA	Adds the Caliptra Key Vault ports to abr_top, which sep_abr_kv_shim binds. The vendored RTL requires it. sep_crypto raises an elaboration $error when SEP_ABR_EN is set without CALIPTRA.
TECH_SPECIFIC_ICG	Removes the behavioural latch gate from abr_icg.sv. The overlay abr_clk_gate instantiates prim_clock_gating in its place.

Adams Bridge also reads RV_FPGA_SCA, which adds FPGA side-channel trigger outputs to abr_top, and FCOV, which adds a covergroup to its AHB subordinate. No OCAH flow sets either.

22.4.2. VeeR EL2 core

The RV_* names that configure the VeeR EL2 core come from the snapshot header vendor/chipsalliance/Cores-VeeR-EL2/overlay/snapshots/sep/common_defines.vh, which the sep_el2 Bender target puts on the include path. Change the core configuration by regenerating the snapshot with the veer command recorded in its header, not with a command-line define.

The snapshot enables user mode (RV_USER_MODE), Smepmp (RV_SMEPMP) and the AXI4 bus interface (RV_BUILD_AXI4). It leaves RV_BUILD_AHB_LITE, RV_ASSERT_ON, RV_FPGA_OPTIMIZE, RV_PHYSICAL, RV_CLOCKGATE, RV_LOCKSTEP_ENABLE and RV_LOCKSTEP_REGFILE_ENABLE unset.

RV_LOCKSTEP_ENABLE also changes SEP RTL. With it, sep_cpu connects lockstep_ctrl_i and lockstep_status_o to the lockstep ports of el2_veer_wrapper. Without it, sep_cpu ignores lockstep_ctrl_i and holds lockstep_status_o at zero. The sep_cpu port list is the same either way.

The sep_el2 Bender target sets TECH_SPECIFIC_EC_RV_ICG, so beh_lib.sv uses the snapshot's user_clock_gate cell rather than its own behavioural clock gate.

The core also reads GTLSIM, which zero-initializes its RAM models for gate-level simulation, and DUMP_BTB_ON, which prints branch predictor configuration and a trace of BTB events. No OCAH flow sets either.

22.4.3. First-party view checks

- tlul_rsp_intg_gen asserts that incoming response integrity and user bits are zero, where its parameters declare them zero, when OCAH_DEBUG_LIVE is defined.
- In the simulation view, entropy_ring_stage_wrappers adds a #1 delay to each ring oscillator stage so the ring oscillates. SYNTHESIS and EMULATION drop the delay, so a zero-delay simulation of either view has no entropy source oscillation.

22.4.4. Key manager

Define	Effect
OCAH_KM_VROM	Decodes the virtual ROM window, 0x1000_0000–0x1000_FFFF, in picorv32_wrapper. Key manager CPU accesses to the window leave on the vrom_mem_* nets instead of the AXI-Lite crossbar. The window is executable with KMCSR.SRAM_EXEC_MODE in either state and is exempt from the ROM lockout. An access completes only when the simulation drives vrom_mem_ready; otherwise the key manager CPU stalls. No OCAH flow sets it, SEP DV included.

Without OCAH_KM_VROM, the window is an unmapped key manager address. A load or store answers DECERR and sets

KMCSR.IRQ_STATUS.AXI_DECERR, and an instruction fetch sets KMCSR.IRQ_STATUS.EXEC_VIOLATION.

WARNING	OCAH_KM_VROM is simulation-only. Elaboration fails with an $error when SYNTHESIS or EMULATION is also defined, because nothing outside a simulation drives vrom_mem_ready.

The key manager's picorv32 core reads FORMAL, which compiles its embedded assert and restrict properties, and the riscv-formal names (RISCV_FORMAL, RISCV_FORMAL_ALTOPS, RISCV_FORMAL_BLACKBOX_REGS, RISCV_FORMAL_BLACKBOX_ALU), the DEBUG* trace names and the PICORV32_TESTBUG_* fault injections. Leave these unset outside a picorv32 formal flow.

22.4.5. DV testbench names

SEP DV sets these for hw/sys/sep/dv/tb/tb_top.sv and its CPU stub. No DUT file reads them.

Define	Effect
SEP_JTAG_AXIL_LIVE	Drives the OTP debug AXI-Lite port from the testbench instead of tying it off.
SEP_SMN_INBOUND_AXI_LIVE	Drives the SMN inbound AXI port from the testbench instead of tying it off.
SEP_SMC_MEM_MODEL	Routes the SMC address region to a testbench memory model. Set on the rom_boot target.
SEP_CPU_STUB	Compiles hw/sys/sep/dv/shims/cpu/sep_cpu_stub.sv in place of sep_cpu, so the build contains no VeeR core. The lsu_stub_all_live target sets it and removes sep_cpu.sv from the filelist.

The SEP DV Verilator flags set neither VERILATOR nor ASSERTS_OFF. Verilator predefines the first; without the second, the PULP assertion macros expand.

Chapter 23. Synthesis filelist defines

The Bender `synth` view is the authoritative source for the defines a synthesis compile needs. Generate the filelist from that view — or use the filelist shipped in a release package, which is generated the same way:

```
bender script flist-plus <block targets> -t synth > <block>.f
```

The filelist carries the defines as `define` lines paired with the sources they select: `SYNTHESIS=1`, and the package switches that choose between behavioural models and technology-cell overlays in vendored RTL. Consume the `define` lines from the filelist rather than restating them on the tool command line: the set changes as the tree does, and the filelist pairs each switch with the overlay sources it selects.

WARNING	A compile that drops the `define` lines analyzes and elaborates without error. The failure is silent mis-selection: vendored RTL builds its behavioural models instead of the technology-cell overlays, and optional blocks compile out of the hierarchy. After elaboration, confirm the netlist instantiates the primitive cells and the block structure you expect.

The filelist also lists `ocah_vendor_defines.svh`, but vendored packages precede it in compile order, so it does not replace the force-include described at the top of this chapter.

23.1. Synopsys Design Compiler and Fusion Compiler

`analyze` rejects a filelist as a native argument (VER-294), and `analyze -define` covers only names given on that command. Hand the filelist to the VCS-compatible reader, which parses its `define` and `incdir` lines:

```
analyze -format sverilog -vcs "-f <block>.f"
```

These tools predefine `SYNTHESIS`; when the filelist restates it, Design Compiler keeps its own value and warns (VER-1051). The value is 1 either way, so the warning is harmless.

23.2. Cadence Genus

Genus parses the same lines through its optionfile reader. Select SystemVerilog explicitly: `read_hdl -f` treats sources as Verilog-2001 regardless of the `hdl_language` attribute.

```
read_hdl -language sv -f <block>.f
```

Prim Cell Replacement and Don’t-Touch Constraints

OCAH primitives (`prim_buf`, `prim_and2`, `prim_clock_gating`, the `prim_flop_*sync*` family, and so on) ship as behavioral models with a reference technology mapping. Adopters replace them with cells from their own library.

Two rules govern those replacements. Both are the adopter’s responsibility — OCAH provides the instance names, but the names have no effect until the adopter’s synthesis and place-and-route flows constrain them.

Chapter 24. Naming Requirement

Every library cell instantiated inside a replacement primitive must have `d0nt_` in its instance name.

The marker is deliberately misspelled with a zero. `dont_touch` is a reserved keyword in several tools and a common substring in ordinary comments and signal names; `d0nt_` occurs only where the protection is intended, so a name-based match cannot pick up unrelated cells.

Clock-path cells take the extended prefix `d0nt_clk`, which selects the stricter rule below. The `d0nt_*` naming convention appears in the eFuse IP (`efuse_token_digest_sha256`, `efuse_triple_redundant_comparator`) as the primary example. The adopter is expected to carry the same convention into any technology-mapped primitive wrapper they deliver. The intended name assignments are:

Instance name	Rule	Primitive
`d0nt_clkgate`	Don't touch	`prim_clock_gating`
`d0nt_clkbuf`	Don't touch	`prim_clock_buf`
`d0nt_clkinv`	Don't touch	`prim_clock_inv`
`d0nt_clkmux`	Don't touch	`prim_clock_mux2`
`d0nt_clknand`	Don't touch	`prim_clock_nand2`
`d0nt_clkxor`	Don't touch	`prim_xor2`
`d0nt_stdbuf`	Size only	`prim_buf`
`d0nt_stdinv`	Size only	`prim_flop_*sync_*` reset inverter
`d0nt_an*`	Size only	`prim_and2`, `prim_and3`
`d0nt_ao222`	Size only	`prim_ao222`
`d0nt_wrap_sync`	Size only	`prim_flop_*sync_*` synchronizer flops

Keep the same names when substituting your own cells. Adopters who rename must apply the equivalent constraints to whatever names they use.

Chapter 25. Constraint Requirement

In synthesis, apply `set_size_only` to every cell matching `d0nt_`, then `set_dont_touch` to every cell matching `d0nt_clk`. Every step after synthesis needs `size_only` alone — on all `d0nt_` cells, clock cells included.

```
# Order matters: d0nt_clk* also matches d0nt_*, so the clock rule is applied
# second and overrides size-only for those cells.
set_size_only  [get_cells -hierarchical -filter "full_name =~ *d0nt_*"]
set_dont_touch [get_cells -hierarchical -filter "full_name =~ *d0nt_clk*"]
```

`size_only` keeps the instance in the netlist — the tool may not delete, merge, absorb, restructure, or buffer through it — while still allowing it to be swapped for another library cell of the same logic function, typically to change drive strength or threshold-voltage flavor for timing, area, and EM closure. That is the intent for every replacement cell: these primitives exist because the structure matters, not because the logic function could not be inferred. Synthesis will otherwise remove them — an inverter pair that cancels, redundant AND gates that always agree, a buffer driving a single load are all dead weight to an optimizer.

Clock cells need the stronger `dont_touch` because that swap is exactly the problem. Under `size_only` the tool treats the logic function as the contract and every other property of the cell as free to change. On the clock path the function is the least of the reasons the cell was chosen. Clock buffers, inverters, muxes, and gaters are picked for electrical characteristics the function does not capture — matched rise and fall times above all, and with them duty-cycle preservation, balanced input-to-output delay, and low delay variation across PVT. A functionally identical cell pulled from the general-purpose library guarantees none of that. The netlist still says "inverter", functional simulation still passes, and the duty-cycle distortion shows up only in silicon or in a timing run late enough to be expensive.

`dont_touch` removes the tool's discretion: the cell the designer selected is the cell that reaches the netlist. Sizing and insertion-delay decisions on the clock path belong to clock tree synthesis, which owns them explicitly, rather than to a logic-optimization pass making a substitution it believes is free.

25.1. After Synthesis

The `dont_touch` on `d0nt_clk*` is a synthesis-only constraint. Once synthesis has handed off the netlist, release the clock cells to `size_only` and carry that — and nothing stronger — through placement, clock tree synthesis, routing, and optimization:

```
# Post-synthesis: uniform size_only, clock cells included.
set_dont_touch [get_cells -hierarchical -filter "full_name =~ *d0nt_clk*"] false
set_size_only  [get_cells -hierarchical -filter "full_name =~ *d0nt_*"]
```

The stricter rule exists to stop logic optimization from choosing the cell. Downstream, clock cell sizing is CTS's job: it cannot balance skew or close transition violations with the cells it needs to resize frozen, and freezing them there buys nothing. Keeping `size_only` in place still prevents deletion, merging, and restructuring for the rest of the flow.

Restrict the physical flow to the library's clock-cell list so a resize stays inside the same clock cell family. Sizing a `CLKBUF_D4` up to a `CLKBUF_D8` keeps the matched rise and fall behavior the cell was chosen for; substituting a general-purpose buffer of equivalent drive does not.

Additional requirements:

- **Carry `size_only` through PnR.** The `dont_touch` is synthesis-only, but `size_only` is not. A cell protected in synthesis and left entirely unprotected downstream is not protected.
- **Verify the match is non-zero.** A typo in the filter, or a flow that renames instances before the constraint is read, silently protects nothing. Check the collection size and error out if it is empty.
- **Preserve the names.** Uniquification, flattening, and name mapping must not strip the `d0nt_` substring. If your flow renames instances, derive the collections before flattening and carry them forward.
- **Restore protection after ECOs.** Flows that relax preservation constraints for clock-tree or filler ECOs — clearing `eco_honor_dont_touch`, for example — must restore `size_only` before any subsequent optimization pass.
- **Check the final netlist,** not just the post-synthesis one.

Flows other than Design Compiler and Fusion Compiler must apply the equivalent `size_only` and `dont_touch` attributes to the same collections.

Chapter 26. Simulation

These constraints have no functional effect and no simulation impact. RTL simulation compiles the technology-independent primitive models (`not(synth)`). Library cells are not on that filelist, so a simulator never elaborates them.

Confirming that the protected cells survived is therefore a gate-level netlist check. RTL simulation cannot detect their loss.

CDC Timing Constraints

OCAH ships synthesis constraints that bound the enumerated synchronization structures in the SMC, SEP, DTP and SMU. Independent CDC sign-off is still required for the assembled SoC. This section explains what those constraints do, how they are produced, and what an adopter has to change when integrating OCAH into their own flow and cell library.

To give the different systems a unified view of how the constraints are defined, these constraints are collected with symlinks under `integration/constraints` using a common hierarchy, described below. If the collateral inventory changes, run `scripts/update_integration_symlinks.py` to regenerate the index.

The constraints for each system build on shared code, linked at `integration/constraints/shared`. This consists of five files: `clock_periods.tcl` (which sets the target clock periods every block reads), `io_delay_budgets.tcl` (which sets the fraction of a period each kind of port reserves outside the block), `async_clock_groups.tcl` (which defines clock groups with default bounds), `cdc_max_delay_procs.tcl` (which defines procs for emitting CDC constraints) and `hier_reuse_procs.tcl` (which lets a parent replay a block's constraints under an instance prefix).

The constraints for each system can use this shared code and are visible through symlinks at `integration/constraints/<block>/synth`. This directory contains the following files:

- `constraints.sdc`: Composes the block's timing intent from the files below and from the shared code, in the order they are listed here. It carries no constraints of its own beyond the caveats recorded in its header.
- `<block>_clocks.sdc`: The block's clock definitions: the `create_clock` stampings on its clock ports and the generated clocks derived from them. Reads the periods from `clock_periods.tcl`.
- `<block>_clock_groups.sdc`: The asynchronous clock groups, declared with `-allow_paths` so the per-instance bounds below are not masked by a false path.
- `<block>_io_delays.sdc`: The input and output delays on the block's ports, sized from `io_delay_budgets.tcl`, which the file sources itself. The CDC/RDC sign-off run reads the same three files, without the max-delay layer below (see CDC/RDC sign-off collateral).
- `<block>_cdc_max_delay.tcl`: This defines periods for generated clocks and any hand-written overrides and then sources the next script.
- `<block>_cdc_max_delay_generated.tcl`: This auto-generated file defines maximum delays for instances of synchronization structures in the system. It expects to be sourced by a file that has already sourced `cdc_max_delay_procs.tcl` to define the procs that it uses.

Two blocks carry an extra file. SEP adds `sep_entropy_clocks.sdc`, whose ripple-divider clocks are built from the RTL hierarchy and are skipped when the entropy source is blackboxed. SMC adds `smc_gpio_io_delays.sdc` for the GPIO padring; SMU passes the same padring through, so its `smu_io_delays.sdc` sources that file rather than keeping a second copy of it.

Chapter 27. Why crossings are bounded rather than timed

A clock-domain crossing has no fixed phase relationship, so it cannot be timed against a capture edge. Its correctness still depends on delay, though: every CDC element assumes the data reaches the receiver within a bounded time.

OCAH therefore keeps the crossings visible to the tool and gives each one an explicit upper bound, sized from the clock that captures it. This doesn’t assert any phase relationship. It only states how much delay the crossing tolerates while its CDC element’s guarantee still holds.

Chapter 28. What is constrained

Any CDC element has at least one capture clock. For every CDC element whose type is listed below, the generated constraints give the element its own `set_max_delay`. This is sized from the capture clock, as read from the netlist. When the capture clock is muxed from several sources, the bound must hold for whichever source the mux selects at runtime; as such, the shortest period among the mux inputs is used.

The elements covered are `prim_flop_2sync`, `prim_sync3/4` and their reset variants, the pulp `sync` shim, `prim_sync_reset`, `prim_sync3_pulse`, `prim_fifo_async`, `avsbus_async_fifo`, `cdc_fifo_gray`, `axi_cdc`, `prim_reg_cdc`, `prim_sync_reqack_data`, `prim_sync_data_autohs`, `cdc_2phase`, `dmi_wrapper` and Rocket `AsyncQueue`.

The bound depends on what protects the crossing:

Kind of path	Bound	Reason
Synchronizer data input	`0.5 * T_dst`	Leaves half a destination period of settling margin before the capture edge the synchronizer relies on.
Protocol-protected data	`1.0 * T_dst`	Conservative. The requirement is set by the pointer synchronizer's depth — see below.
Two-phase handshake nets	`min(T_src, T_dst)`	`cdc_2phase` runs the handshake in both directions, so the tighter domain governs. This is the module's own stated requirement.

`T_dst` rather than `min(T_src, T_dst)` in the general case because the gating synchronizer is clocked by the destination, so its settling window is `S * T_dst` and `T_src` does not enter the guarantee. This makes slow-to-fast and fast-to-slow equally safe.

28.1. Why protocol-protected data still needs a bound

The payload of a gray-pointer FIFO or a req/ack handshake never passes through a synchronizer — only the pointer or request does. So this is not a metastability constraint, and it is reasonable to ask why such a path needs a bound at all, or why a single blanket bound covering every crossing in the design would not be enough.

The bound is needed because the protocol guarantees **ordering**, not **arrival**. The read side will not look at an entry until the write pointer has crossed, which takes `SYNC_STAGES` destination cycles, where `SYNC_STAGES` is the depth of the pointer synchronizer. Nothing in the protocol ensures the data has physically arrived within that window; that is a timing property, and only a `max_delay` states it. Miss it and the consumer samples the entry's previous contents. There is no violation report and no metastability event, just wrong data.

So the requirement is `data path < SYNC_STAGES * T_dst`. It is not a fixed number of nanoseconds: it scales with the clock that captures the crossing, and the destination periods within one design span an order of magnitude.

Destination period	Guarantee at 3 stages
1 ns	3 ns
5 ns	15 ns
10 ns	30 ns

That is why the bound is applied per instance rather than as one global value. A single blanket `max_delay` would have to be loose enough to be feasible on the slowest domains. But this would permit several times more delay than a fast crossing's pointer scheme tolerates. To ensure that fast crossings are properly constrained, each crossing is sized from its own capturing clock.

The blanket default applied by the clock-group layer (see The two layers) deliberately applies this looser constraint. For slow domains, it is already sufficient and an explicit per-instance constraint is redundant. It is the fast crossings that need a per-instance constraint.

`1.0 * T_dst` is therefore conservative by a factor of two to three. It is kept simple deliberately, but if wide payloads such as AXI data will not close then raising `cdc_data_delay` to `2.0 * T_dst` stays inside a three-stage guarantee with a cycle to spare. An adopter whose synchronizer cells use a different depth should recompute from their own `SYNC_STAGES` rather than inheriting this factor.

Enumeration and clock resolution run once, offline, against an elaborated design, and their output is an explicit per-block file of constraint calls that is reviewed and checked in. The flow itself does no discovery. This matters to an integrator because those files are written against the OCAH hierarchy and OCAH's clock names. If you instantiate a block as delivered, you do not need to regenerate or hand-edit them: set `::cdc_hier_prefix` to where the block sits in your hierarchy, and optionally `::cdc_clock_alias` to map OCAH clock names onto your own. Regeneration is needed only when the block is reconfigured in a way that changes which CDC elements exist, since no prefix can conjure constraints for elements the file never listed (see Integration checklist item 2).

Chapter 29. The two layers

The per-instance bounds only take effect on top of a correctly configured clock-group layer. That layer decides whether the tool considers inter-domain paths at all, so it must be established first; the per-instance exceptions then refine it.

Layer 1 is `set_async_clock_groups`, defined in `flows/synth/constraints/async_clock_groups.tcl` and reached from each block's `<block>_clock_groups.sdc` through `cdc_apply_async_groups` (in the same file), which resolves each domain's membership and passes the block's `-exclude` pairs through. It declares the asynchronous groups **with** `-allow_paths` and applies a loose 20 ns / -20 ns default to every inter-group clock pair.

IMPORTANT	`-allow_paths` is a prerequisite, not a refinement. In DC and PrimeTime exception priority, `set_false_path` outranks `set_max_delay` regardless of which is more specific, so a bare `set_clock_groups -asynchronous` silently masks **every** per-instance constraint. There is no warning; the exceptions simply have no effect.

The 20 ns default is what makes `-allow_paths` safe to enable. Once paths are allowed, every crossing without a specific constraint would otherwise be timed against a meaningless launch-capture relationship and fail. The default bounds them loosely so that only deliberately tightened paths appear as real requirements. It is emitted per ordered clock pair, never group-wide, because a 20 ns bound inside a group would mask genuine setup failures on synchronous paths.

Layer 2 is the per-instance `set_max_delay` output of the three steps above, with two deliberate carve-outs:

- **Paths from primary inputs revert to the default.** A synchronizer input reachable from a top-level port carries that port's `set_input_delay`, which is a budget figure rather than real logic and can alone exceed the tightened bound. Only the parent block sees both ends, so there is nothing to tighten at block level.
- **Same-clock instances are skipped.** A two-clock element wired to one clock is not a crossing, and applying `set_max_delay -ignore_clock_latency` there would replace a proper setup check with a flat budget that discards skew and uncertainty. These are emitted commented out. OCAH has five, where OpenTitan AES, KMAC and OTBN instantiate `prim_sync_reqack_data` on EDN and OTP interfaces that are separate domains upstream but share a clock in SEP.

Chapter 30. CDC/RDC sign-off collateral

The same clock, group and I/O-delay files that synthesis reads are the clock truth for static CDC/RDC sign-off, and the rest of that sign-off collateral is checked in beside them, so an adopter can run their own tool against the reviewed constraints and waivers rather than reconstructing them.

- `hw/sys/<block>/cdc/<block>.cdc_rdc.tcl` is the entry point. It sources the block's files in dependency order: `hier_reuse_procs.tcl`, `clock_periods.tcl` and the `<block>_{clocks,clock_groups,io_delays}.sdc` siblings (without the CDC max-delay layer `constraints.sdc` adds), `<block>.resets.tcl` (the reset tree and its assertion sequences), `<block>.case_analysis.tcl` (the functional pins for DFT and strap ports), `<block>.static_signals.tcl` (quasi-static signals, CDC only), the shared type-level setup in `flows/cdc/cdc_rdc_setup.tcl` (synchronizer and verified-IP modules, gray annotations) and the block's instance-level `<block>.cdc_rdc_setup.tcl`.
- `hw/sys/<block>/cdc/<block>.vccdc*.waiver.tcl` and `hw/sys/<block>/rdc/<block>.vcrdc*.waiver.tcl` hold the reviewed waivers, one file for Tenstorrent-owned RTL and, where needed, an `.opensource_ip` file for third-party IP. Each waiver carries its rationale in `-comment {}`.
- `flows/synth/constraints/hier_reuse_procs.tcl` wraps the tool commands these files use so they can be replayed under a hierarchy prefix.

The files name the leaves of the generic primitives (`q_d`/`q_ddd` synchroniser stages, `u_sync_1`/`u_sync_2` two-stage synchronisers, `q_o` flops, `en_latch` clock gates); `flows/cdc/README.md` lists them. Run the sign-off against the generic primitive set or re-anchor the collateral to the swapped names.

Two variables select how the files are read:

- `::cdc_app` is `cdc` (default) or `rdc`; the entry skips CDC-only files for `rdc`.
- `::cdc_scenario` is `synth` (default) or `functional`. The functional scenario is the sign-off run: the case-analysis files pin the DFT and strap ports, so `cdc_pinned_port_delay` skips the I/O delays on those ports.

`hw/sys/smu/cdc/smu.cdc_rdc.tcl` shows how a parent replays a child: `cdc_begin_block <prefix> {clock aliases} {reset aliases}`, source the child's entry, `cdc_end_block`. Port clocks, I/O delays, port resets and the child's own group declaration are block-top gated and skipped under a prefix; internal constraints re-anchor through `cdc_inst`, and generated clocks registered with `cdc_group_extra` land in the parent's single `cdc_apply_async_groups`. Waiver filter fields carry a `${PREFIX}` token for the same reason. `flows/cdc/README.md` summarises the layout.

Chapter 31. What is covered

Block	CDC elements constrained	Exceptions emitted
SMC	365	412
SEP	18	21
DTP	250	270
SMU	758	838

SMU is a flat build containing the other three, so its instance paths carry `u_smc/`, `gen_sep.u_sep/` and `u_dtp/` prefixes. The SMU row has more CDC elements than the sum of the other three rows because the SMU instantiates a larger DTP. The DTP contains three JTAG2AXI bridges in the SMU configuration and only two when it is on its own. These configurations give the DTP 375 and 250 elements, respectively.

Chapter 32. Integration checklist

Five things must change, three should be checked. None of them requires editing per-instance data by hand. Every change is either a one-line policy edit or a regeneration.

32.1. 1. Repoint the constraints at the pin of the cell inside the prim wrapper

This is the most important change.

As shipped, the procs target the prim wrapper's own port, `prim_sync3/d_i`. That port exists only as long as the wrapper survives as a hierarchical boundary, and a prim cell is ordinary RTL: nothing stops synthesis from ungrouping it and dissolving its ports.

WARNING	Do not rely on the wrapper port surviving. Anchor the constraint to the pin of the cell instantiated **inside** the wrapper. When you swap in your own synchronizer, that cell is a real library cell or macro — a leaf that cannot be flattened — so its pins persist through optimisation regardless of what happens to the RTL hierarchy around it.

```
proc set_cdc_max_delay_prim_sync3 { inst dst_clk {delay {}} } {
    cdc_emit "prim_sync3 $inst" [cdc_sync_delay $dst_clk $delay] to \
        {} [get_pins "$inst/u_my_sync_cell/D" -quiet]
}
```

`$inst` stays the wrapper instance path the generator recorded; you extend it down to the leaf. If your cell is instantiated in a generate loop or per bit, the path needs a glob. Note that generate scopes are separated by `.`, not `/`:

```
[get_pins "$inst/g_bits*.u_my_sync_cell/D" -quiet]
```

Anchoring this way makes hierarchy preservation a safety margin rather than a load-bearing dependency.

If you keep the prim wrappers unchanged, the shipped port names work as-is, but your flow must then preserve those wrapper boundaries. A `destination empty` warning appearing after `compile` where there was none at constraint-load time means a boundary was dissolved.

If you go further and replace a wrapper with a module of your own name, the enumeration that produced `<block>_cdc_max_delay_generated.tcl` knows nothing about it and those crossings end up with no per-instance bound at all. Nothing warns — an element that was never listed cannot report itself missing. Add the calls for it to `<block>_cdc_max_delay.tcl` by hand; the procs take an instance path and the capturing clock regardless of the module they were named after.

32.2. 2. Point the files at your hierarchy, or regenerate

The checked-in `<block>_cdc_max_delay_generated.tcl` files are written against the OCAH hierarchy, with paths relative to the block's own top and clocks named as OCAH names them. Two ways to make them apply to your design.

If you instantiate the block as delivered, reuse the files and set two variables. Both default to no-ops, so leaving them alone reproduces the shipped behaviour exactly.

```
# where the block sits in your hierarchy, with a trailing slash
set ::cdc_hier_prefix "u_chiplet/u_mgmt/u_smc/"

# optional: map an OCAH clock name onto the clock you actually drive it with
set ::cdc_clock_alias(SMCCLK) MY_SYS_CLK
```

The prefix is prepended to every instance path. The alias is applied wherever a clock name reaches the tool, so OCAH's names become labels in your design. You can also skip the alias and supply the period directly with `set ::cdc_clock_period(SMCCLK) <ps>`, which is enough for the synchronizer constraints; the alias additionally matters for the data-path constraints, which name the destination clock as a timing endpoint and so need it to exist.

If you reconfigure the block, regenerate instead. Changing parameters that add or remove CDC elements makes the shipped file stale in the silent direction — the constraints that should exist for new elements simply will not, and nothing warns. The prefix does not help with that.

A wrong prefix fails loudly: every path resolves to nothing, so you get a warning per constraint against a zero exception count, and `SUMMARY 0 exceptions` is itself reported as a setup failure.

Regeneration is also the right answer whenever a parent build contains the block. Instance counts there can differ from the standalone build, so a parent's file is not the child's file with a prefix.

32.3. 3. Set the default bound

The 20 ns / -20 ns figures are a starting point. Both are `info exists`-guarded in `flows/synth/constraints/async_clock_groups.tcl`, so setting them before that file is sourced overrides without editing it. `set_async_clock_groups` also accepts per-call overrides, which is the cleaner way to give one block a different default:

```
set_async_clock_groups { {CLK_A} {CLK_B} } -max_delay 8000 -min_delay -8000
```

Two constraints on the choice. The default must exceed every per-instance bound you tighten, because it also governs paths from primary inputs into synchronizers; if it is tighter than `0.5 * T_dst` it becomes the binding constraint there. And tighter is not automatically better: this is a net for crossings you have not enumerated, so set it loose enough that those do not fail and rely on layer 2 for the paths that matter.

The minimum is negative deliberately. With `-allow_paths` a crossing has no meaningful hold relationship, and a zero floor would report hold violations where hold is not a real requirement.

32.4. 4. Declare your own clock groups

Each block's `<block>_clock_groups.sdc` calls `cdc_apply_async_groups` with its list of domain names. A domain's group is its clock plus every generated clock registered against it with `cdc_group_extra`, clocks absent from the session are dropped, and the call reaches `set_async_clock_groups` with any `-exclude` pairs passed through. Both calls have to come after every `create_clock` and `create_generated_clock` they are meant to cover. Call `set_async_clock_groups` directly when you need globs or per-call bounds; each inner list is one asynchronous group:

```
set_async_clock_groups {
    {CLK_A CLK_A_DERIVED_*}
    {CLK_B}
    {JTAG_TCK}
} -exclude {{PATTERN_X* PATTERN_Y*}}
```

`-exclude` takes pairs of patterns and suppresses that part of the cross product, for clock pairs that already have a `-logically_exclusive` or `-physically_exclusive` relationship that `-allow_paths` would contradict. Omit it unless you hit a `TIM-184` conflict.

32.5. 5. Declare periods for generated clocks

A generated clock exposes no readable `period` attribute in Design Compiler, so any generated clock that clocks a CDC element needs an explicit entry in `<block>_cdc_max_delay.tcl`:

```
set ::cdc_clock_period(MY_DIV_CLK) [expr {4 * $clock_periods(MY_MASTER_PERIOD)}]
```

Omitting one is loud — a `no period for clock` warning per affected element — so a clean run means you have them all. Keep them in step with your `create_generated_clock` divide factors; nothing cross-checks them.

32.6. Also check

Hierarchy preservation. The reference flow applies `set_ungroup false` and `set_boundary_optimization false` to the CDC designs before `compile`. How much this matters depends on item 1: with leaf cell pins it is a safety margin, with wrapper ports it is load-bearing. Either way, your flow needs the equivalent, and it needs to name your cells rather than OCAH's.

Delay factors. `0.5 * T_dst` and `1.0 * T_dst` are policy, not physics. If your methodology wants more settling margin, change `cdc_sync_delay` and `cdc_data_delay` in `flows/synth/constraints/cdc_max_delay_procs.tcl`. Per-instance overrides are also available without editing procs: every proc takes an optional trailing delay in picoseconds, and re-stating a call in `<block>_cdc_max_delay.tcl` after the `source` wins.

Tool differences. Two Design Compiler specific choices to revisit under PrimeTime. `get_attribute <pin> clocks` is used because `get_clocks -of_objects` is PrimeTime-only, and generated clocks may expose a readable `period` in PrimeTime, making item 5 unnecessary.

Chapter 33. Confirming it worked

Load the constraints and check the summary:

```
INFO: cdc_max_delay: SUMMARY <n> exceptions, 0 warnings
```

Zero warnings is necessary but not sufficient. A pin pattern that matches nothing warns, but one that matches the **wrong** thing does not. Two stronger checks:

- **Predict the count.** Multiply each proc's number of emitted exceptions by its number of calls and compare against the summary. A mismatch localises the failure, because the size of the shortfall identifies which proc did not fire.
- **Confirm the precedence.** Layer 2 is on pins and layer 1 on clocks, and the design assumes the more specific specification wins. Verify once in your tool:

```
report_timing -to [get_pins <a synchronizer>/d_i] -nworst 1
```

The `max_delay` line must show your `0.5 * T_dst` value rather than the default. If it shows the default, layer 2 is inert and nothing else in this section applies.

`SUMMARY 0 exceptions` is treated as a setup failure and warns as such. It means the constraints did not load, not that the design has no crossings.

Chapter 34. Scope and limitations

The 20 ns default is loose by design. Against a 1 ns clock it permits twenty clock periods of delay into a synchronizer input. This is a safety net that keeps `-allow_paths` usable, not a real constraint; any crossing that matters belongs in layer 2.

Enumeration works from a fixed list of module names, so a hand-written two-flop synchronizer that uses none of them will not be found. Structural detection of crossings is what a static CDC tool does, and this flow does not attempt it. These constraints time crossings already known to exist; they are not a substitute for CDC signoff.

Because the instance list is generated offline, removals are loud but additions are silent. A stale instance path warns, but a synchronizer added without regenerating produces no warning: the constraint never exists, the path falls back to the default, and it passes timing. Regenerate and diff the result against what is checked in whenever the RTL changes.